



**ESRG**

EMBEDDED  
SYSTEMS  
RESEARCH GROUP

**University of Minho**  
School of Engineering

MSc in Industrial Electronics Engineering and Computers  
**Embedded Systems**

---

# ShivAT C Compiler

Project Report

---



**Project made by**  
Class of 25/26

**Project supervised by**  
Professor Adriano Tavares, PhD

# Contents

---

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>List of Figures</b>                                                  | <b>5</b>  |
| <b>List of Tables</b>                                                   | <b>7</b>  |
| <b>List of Listings</b>                                                 | <b>8</b>  |
| <b>1 Introduction</b>                                                   | <b>10</b> |
| 1.1 Contextualization . . . . .                                         | 10        |
| 1.2 Problem Statement . . . . .                                         | 10        |
| 1.3 Motivation . . . . .                                                | 10        |
| 1.4 Requirements . . . . .                                              | 11        |
| 1.4.1 Functional Requirements . . . . .                                 | 11        |
| 1.4.2 Non-Functional Requirements . . . . .                             | 11        |
| 1.5 Constraints . . . . .                                               | 11        |
| 1.5.1 Technical Constraints . . . . .                                   | 11        |
| 1.5.2 Non-Technical Constraints . . . . .                               | 12        |
| <b>2 Analysis</b>                                                       | <b>13</b> |
| 2.1 Compiler Architectures . . . . .                                    | 13        |
| 2.2 Intermediate Representations: 3AC, GIMPLE, SSA . . . . .            | 13        |
| 2.3 Register Allocation Algorithms . . . . .                            | 14        |
| 2.4 Compilers for Small / Embedded Targets . . . . .                    | 15        |
| 2.5 Assemblers and Retargetable Toolchains . . . . .                    | 15        |
| <b>3 Design</b>                                                         | <b>17</b> |
| 3.1 The Compilation Pipeline . . . . .                                  | 17        |
| 3.1.1 Lexer: Lexical Analysis . . . . .                                 | 17        |
| 3.1.2 Parser: Syntax Analysis . . . . .                                 | 18        |
| 3.1.3 Semantic Analysis . . . . .                                       | 19        |
| 3.1.3.1 Two-Pass Architecture . . . . .                                 | 19        |
| 3.1.3.2 Scope and Symbol Table Design . . . . .                         | 20        |
| 3.1.3.3 Type-System Design . . . . .                                    | 21        |
| 3.1.3.4 AST Annotation Design . . . . .                                 | 22        |
| 3.1.3.5 Diagnostics Design . . . . .                                    | 23        |
| 3.1.3.6 Pass 1 Design: Symbol Binding . . . . .                         | 23        |
| 3.1.3.7 Pass 2 Design: Type Inference and Constraint Checking . . . . . | 24        |
| 3.1.3.8 Subset-C Support . . . . .                                      | 24        |
| 3.1.4 Intermediate Representation . . . . .                             | 24        |
| 3.1.4.1 Design Choice: GIMPLE-Style 3-Address Code . . . . .            | 24        |
| 3.1.4.2 IR Type System . . . . .                                        | 25        |
| 3.1.4.3 Module / Function / Block / Instruction Hierarchy . . . . .     | 26        |
| 3.1.4.4 The <code>ir_instr_t</code> Quadruple . . . . .                 | 26        |
| 3.1.4.5 Operand Kinds . . . . .                                         | 26        |
| 3.1.4.6 Opcode Catalogue . . . . .                                      | 26        |
| 3.1.4.7 IR Lowering Overview . . . . .                                  | 26        |
| 3.1.4.8 Aggregate Layout . . . . .                                      | 27        |

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 3.1.5    | Register Allocator . . . . .                                         | 27        |
| 3.1.5.1  | Liveness Analysis . . . . .                                          | 28        |
| 3.1.5.2  | Interference Graph . . . . .                                         | 28        |
| 3.1.5.3  | Pre-colouring . . . . .                                              | 28        |
| 3.1.5.4  | Chaitin–Briggs Colouring with George Coalescing . . . . .            | 29        |
| 3.1.5.5  | Move Coalescing (George Test) . . . . .                              | 29        |
| 3.1.5.6  | Simplification . . . . .                                             | 30        |
| 3.1.5.7  | Colour Selection . . . . .                                           | 30        |
| 3.1.5.8  | Alias Propagation . . . . .                                          | 30        |
| 3.1.5.9  | Spill Rewriting . . . . .                                            | 30        |
| 3.1.5.10 | Convergence and Termination . . . . .                                | 31        |
| 3.1.6    | Code Generator . . . . .                                             | 32        |
| 3.1.6.1  | Target ISA Overview . . . . .                                        | 32        |
| 3.1.6.2  | Register Conventions and the <code>t3</code> Reservation . . . . .   | 32        |
| 3.1.6.3  | Frame Layout . . . . .                                               | 33        |
| 3.1.6.4  | Per-Opcode Emission Strategy . . . . .                               | 33        |
| 3.1.6.5  | CALL Setup with Parallel Copy . . . . .                              | 34        |
| 3.1.6.6  | Stack-Passed Parameter Loading . . . . .                             | 34        |
| 3.1.7    | Runtime Library . . . . .                                            | 34        |
| 3.1.7.1  | Why Software Arithmetic . . . . .                                    | 34        |
| 3.1.7.2  | Helpers . . . . .                                                    | 35        |
| 3.1.7.3  | Algorithms . . . . .                                                 | 35        |
| 3.1.7.4  | Branch-Mnemonic Caveat . . . . .                                     | 35        |
| 3.1.7.5  | Performance Bounds . . . . .                                         | 35        |
| 3.2      | Assembler Pipeline . . . . .                                         | 35        |
| 3.2.0.1  | High-Level Architecture . . . . .                                    | 35        |
| 3.2.1    | Preprocessor Design . . . . .                                        | 36        |
| 3.2.2    | Step 1: Parsing and Intermediate Representation Generation . . . . . | 36        |
| 3.2.2.1  | Symbols Table . . . . .                                              | 37        |
| 3.2.2.2  | Intermediate Representation Design . . . . .                         | 38        |
| 3.2.3    | Step 2: Object Code Generation . . . . .                             | 39        |
| <b>4</b> | <b>Implementation</b> . . . . .                                      | <b>40</b> |
| 4.1      | Frontend . . . . .                                                   | 40        |
| 4.1.1    | Lexical Analysis . . . . .                                           | 40        |
| 4.1.1.1  | Keywords and Identifiers . . . . .                                   | 40        |
| 4.1.1.2  | Numeric and Literal Handling . . . . .                               | 40        |
| 4.1.1.3  | Comment Handling . . . . .                                           | 41        |
| 4.1.1.4  | Preprocessor Directives . . . . .                                    | 41        |
| 4.1.1.5  | Line Tracking and Error Handling . . . . .                           | 41        |
| 4.1.2    | Syntax Analysis . . . . .                                            | 41        |
| 4.1.2.1  | AST Construction . . . . .                                           | 41        |
| 4.1.2.2  | Expression Grammar and Precedence . . . . .                          | 42        |
| 4.1.2.3  | Semantic Values . . . . .                                            | 42        |
| 4.1.2.4  | Error Handling . . . . .                                             | 42        |
| 4.1.2.5  | Dangling-Else Resolution . . . . .                                   | 43        |
| 4.1.3    | Semantic Analysis . . . . .                                          | 43        |
| 4.1.3.1  | Two-Pass Semantic Traversal . . . . .                                | 43        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 4.1.3.2  | Arena Allocator Implementation . . . . .                          | 43        |
| 4.1.3.3  | Symbol Table Implementation . . . . .                             | 44        |
| 4.1.3.4  | Type-System Implementation . . . . .                              | 44        |
| 4.1.3.5  | Diagnostic Infrastructure Implementation . . . . .                | 45        |
| 4.1.3.6  | Modifiable Lvalue Check . . . . .                                 | 45        |
| 4.1.3.7  | Control-Flow Validation . . . . .                                 | 46        |
| 4.1.4    | Intermediate Representation . . . . .                             | 47        |
| 4.1.4.1  | Module / Function / Block / Instruction Hierarchy . . . . .       | 47        |
| 4.1.4.2  | The <code>ir_instr_t</code> Quadruple . . . . .                   | 47        |
| 4.1.4.3  | IR Lowering . . . . .                                             | 48        |
| 4.1.4.4  | IR Dumps and Verification . . . . .                               | 51        |
| 4.2      | Backend . . . . .                                                 | 52        |
| 4.2.1    | Register Allocation . . . . .                                     | 52        |
| 4.2.1.1  | Liveness Analysis . . . . .                                       | 52        |
| 4.2.1.2  | Interference Graph . . . . .                                      | 53        |
| 4.2.1.3  | Pre-colouring . . . . .                                           | 55        |
| 4.2.1.4  | Chaitin–Briggs Colouring with George Coalescing . . . . .         | 56        |
| 4.2.1.5  | Spill Rewriting . . . . .                                         | 57        |
| 4.2.2    | Code Generation . . . . .                                         | 58        |
| 4.2.2.1  | Prologue and Epilogue Emitter . . . . .                           | 58        |
| 4.2.2.2  | Two-Address Arithmetic Emitters . . . . .                         | 59        |
| 4.2.2.3  | Comparison and Branch Synthesis . . . . .                         | 60        |
| 4.2.2.4  | CALL Setup: Parallel-Copy Implementation . . . . .                | 61        |
| 4.2.3    | Runtime Library . . . . .                                         | 61        |
| 4.3      | Assembler . . . . .                                               | 62        |
| 4.3.1    | Preprocessor – <code>abi.m4</code> . . . . .                      | 62        |
| 4.3.2    | Lexer – <code>lex.l</code> . . . . .                              | 63        |
| 4.3.2.1  | Pattern Definitions . . . . .                                     | 63        |
| 4.3.2.2  | Identifiers, Registers, and Special Tokens . . . . .              | 63        |
| 4.3.2.3  | Branch Condition Tokens . . . . .                                 | 63        |
| 4.3.3    | Parser – <code>parser.y</code> . . . . .                          | 64        |
| 4.3.3.1  | Token Declarations . . . . .                                      | 64        |
| 4.3.3.2  | RI-Format Instruction Construction . . . . .                      | 64        |
| 4.3.4    | Symbol Table – <code>symbol_table.c</code> . . . . .              | 65        |
| 4.3.4.1  | Hash Function . . . . .                                           | 65        |
| 4.3.4.2  | Symbol Insertion and Collision Handling . . . . .                 | 65        |
| 4.3.5    | Step 2 – Code Generator – <code>code_generator.c</code> . . . . . | 66        |
| 4.3.5.1  | Instruction Encoding with Symbol Resolution . . . . .             | 66        |
| 4.3.6    | Concluding Details . . . . .                                      | 66        |
| <b>5</b> | <b>Implementation Validation</b> . . . . .                        | <b>67</b> |
| 5.1      | Test Suite Inventory . . . . .                                    | 67        |
| 5.2      | Snapshot Mechanism . . . . .                                      | 67        |
| 5.3      | CI Scripts . . . . .                                              | 68        |
| 5.4      | End-to-End Walk-Through (Linear) . . . . .                        | 69        |
| 5.5      | End-to-End Walk-Through (Control Flow) . . . . .                  | 70        |
| 5.6      | End-to-End Walk-Through (Forced Spill) . . . . .                  | 71        |
| 5.7      | Negative Tests for Unsupported Features . . . . .                 | 72        |

|          |                                                                   |            |
|----------|-------------------------------------------------------------------|------------|
| 5.8      | Compiled Feature Demo . . . . .                                   | 73         |
| 5.9      | Two-Dimensional Array . . . . .                                   | 74         |
| 5.10     | Stack-Passed Arguments . . . . .                                  | 76         |
| 5.11     | Nested Calls and Link-Register Save . . . . .                     | 77         |
| 5.12     | Software Division via Runtime Helpers . . . . .                   | 78         |
| 5.13     | Short-Circuit Boolean Logic . . . . .                             | 80         |
| 5.14     | Quantitative Evaluation . . . . .                                 | 85         |
| 5.15     | Generated Code Size . . . . .                                     | 85         |
| 5.16     | Register Allocation Statistics . . . . .                          | 85         |
| 5.17     | Coalescing and Spilling . . . . .                                 | 86         |
| 5.18     | Compilation Throughput . . . . .                                  | 87         |
| 5.19     | Cross-Compiler Comparison . . . . .                               | 88         |
| 5.20     | Leaf-Function Fast Path: Implementation and Measurement . . . . . | 89         |
| 5.21     | Limitations . . . . .                                             | 90         |
| 5.22     | Threats to Validity . . . . .                                     | 91         |
| 5.23     | Complete End-to-End Assembler Validation . . . . .                | 91         |
| 5.24     | Target Testing . . . . .                                          | 92         |
| <b>6</b> | <b>Conclusion and Future Work</b>                                 | <b>98</b>  |
| <b>A</b> | <b>Diagnostic Code Reference</b>                                  | <b>101</b> |
| <b>B</b> | <b>IR Opcode Reference</b>                                        | <b>103</b> |
| <b>C</b> | <b>gr0040 ABI Quick Reference</b>                                 | <b>104</b> |

# List of Figures

---

|      |                                                                                                                                                                                                                                                                                                                      |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1  | End-to-end compilation pipeline. Adapted from [1, Fig. 1.6]. . . . .                                                                                                                                                                                                                                                 | 17 |
| 3.2  | Flowcharts fore Node Creation and Children addition . . . . .                                                                                                                                                                                                                                                        | 19 |
| 3.3  | Rooted hierarchy of lexical scopes used during semantic analysis. Each scope stores its own hash table of symbols and a pointer to its enclosing parent scope. .                                                                                                                                                     | 21 |
| 3.4  | GCC's IR structure, reproduced from [2]. The IR in this project is a simplified version of GIMPLE, with no SSA form. . . . .                                                                                                                                                                                         | 25 |
| 3.5  | The five-stage register-allocation pipeline. . . . .                                                                                                                                                                                                                                                                 | 27 |
| 3.6  | Interference graph built from the running example. . . . .                                                                                                                                                                                                                                                           | 28 |
| 3.7  | The pre-colouring stage identifies the call-live set. This forces some virtual registers onto specific physical registers ( $K = 4$ callee-saved registers) as per the ABI's constraints. . . . .                                                                                                                    | 29 |
| 3.8  | Simplification: low-degree nodes are removed and pushed onto the select stack; when no low-degree node exists, a potential spill is removed instead. . . . .                                                                                                                                                         | 30 |
| 3.9  | Colour selection: the first colour not in the forbidden set is chosen according to the preference order. The color represents the physical register assigned to the virtual register, and edges represent interferences. If no color is available, the node is marked for spilling. . . . .                          | 31 |
| 3.10 | Spill rewriting: the chosen vreg is spilled to a new stack slot, with a store after its definition and a load before every use. . . . .                                                                                                                                                                              | 31 |
| 3.11 | The code generator translates the register-allocated IR into gr0040 machine code, handling instruction selection, register assignment, and frame management according to the target ISA and ABI conventions. . . . .                                                                                                 | 32 |
| 3.12 | Assembler overview. . . . .                                                                                                                                                                                                                                                                                          | 36 |
| 3.13 | Step 1 Overview. . . . .                                                                                                                                                                                                                                                                                             | 37 |
| 3.14 | Symbols table flowchart for inserting, adding and setting a symbol, as well as computing the hash value . . . . .                                                                                                                                                                                                    | 38 |
| 4.1  | Interference-graph dump for the running example. . . . .                                                                                                                                                                                                                                                             | 55 |
| 4.2  | Pre-colouring output: return source forced to <b>r1</b> (highlighted). . . . .                                                                                                                                                                                                                                       | 55 |
| 4.3  | Worked spill example for a vreg whose live range crosses many calls. . . . .                                                                                                                                                                                                                                         | 58 |
| 5.1  | Source vs. generated-assembly LOC for ten representative tests, sorted left-to-right by source LOC. Data from <code>assets/eval/code_size.csv</code> . . . . .                                                                                                                                                       | 86 |
| 5.2  | Total virtual-register count and interference-graph edges per test, sorted by source LOC. Data from <code>assets/eval/regalloc_stats.csv</code> . . . . .                                                                                                                                                            | 86 |
| 5.3  | Median compile time vs. source LOC across the test corpus (twenty iterations per test) on two platforms. Data from <code>assets/eval/compile_time.csv</code> (macOS) and <code>assets/eval/compile_time_linux.csv</code> (Ubuntu container). . . . .                                                                 | 87 |
| 5.4  | Generated asm-LOC with and without the leaf-function fast path. Fourteen tests are shown, sorted by source size; the remaining tests in the corpus do not benefit (they are non-leaf or already minimal). Data: <code>assets/eval/code_size_pre_leaf.csv</code> and <code>assets/eval/code_size.csv</code> . . . . . | 90 |
| 5.5  | Waveform capture at the prologue of the <code>main</code> function. . . . .                                                                                                                                                                                                                                          | 96 |
| 5.6  | Waveform capture at the epilogue of the <code>sum</code> function. . . . .                                                                                                                                                                                                                                           | 96 |
| 5.7  | Waveform at variable initialisation, showing register values of <code>t0</code> and <code>t3</code> . . . . .                                                                                                                                                                                                        | 97 |

|     |                                                                                                                         |    |
|-----|-------------------------------------------------------------------------------------------------------------------------|----|
| 5.8 | Waveform showing <b>a0</b> and <b>a1</b> during execution of the loop summation between <b>x</b> and <b>y</b> . . . . . | 97 |
| 5.9 | Realtime ILA waveform, showing <b>a0</b> incrementing by <b>y</b> (+2). . . . .                                         | 97 |

# List of Tables

---

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |     |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 3.1 | Lexical categories recognised by the lexer. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                  | 17  |
| 3.2 | Trade-offs between constructing the symbol table inside parser actions versus using a dedicated semantic pass. . . . .                                                                                                                                                                                                                                                                                                                                               | 20  |
| 3.3 | Selected Pass 1 semantic rules. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                              | 23  |
| 3.4 | Supported subset of the language after semantic analysis. . . . .                                                                                                                                                                                                                                                                                                                                                                                                    | 24  |
| 3.5 | Mapping from semantic <code>type_t</code> to <code>ir_type_t</code> . The IR type itself is sign-agnostic; unsignedness of arithmetic is recovered at lowering time from the operand semantic types and selects the appropriate opcode (e.g. <code>SHRU</code> vs <code>SHRS</code> , <code>LTU</code> vs <code>LTS</code> ). . .                                                                                                                                    | 25  |
| 3.6 | Pre-colour assignments. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                      | 29  |
| 3.7 | gr0040 register conventions. <code>t3</code> is excluded from the allocator preference list and reserved exclusively for the code generator. . . . .                                                                                                                                                                                                                                                                                                                 | 32  |
| 3.8 | Runtime helpers. Each function follows the project ABI: arguments in <code>a0</code> , <code>a1</code> ; result in <code>a0</code> ; <code>a0–a2</code> and <code>t0–t2</code> clobbered (caller-saved); <code>t3</code> is reserved scratch. . .                                                                                                                                                                                                                    | 35  |
| 3.9 | Fields of the <code>statement_t</code> structure. . . . .                                                                                                                                                                                                                                                                                                                                                                                                            | 39  |
| 5.1 | Test file inventory. Backend tests = <code>RegisterAllocation/</code> (24) + <code>IR_checks/</code> (14) = 38. . . . .                                                                                                                                                                                                                                                                                                                                              | 67  |
| 5.2 | Project-goal-to-evidence mapping. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                            | 85  |
| 5.3 | Instructions emitted by our compiler versus SDCC 4.5.0, AVR-GCC 9.5.0 (Homebrew <code>avr-gcc@9</code> ; <code>atmega328p</code> , <code>-Os</code> ) and MSP430-GCC 9.3.1.11 ( <code>msp430g2553</code> , <code>-Os</code> ). Lower is better. The two GCC toolchains aggressively dead-code-eliminate unused locals and constant-fold trivial returns, which is why the <code>linear/iftest/looptest</code> columns collapse to two or three instructions. . . . . | 88  |
| 5.4 | Aggregate effect of the leaf-function fast path. . . . .                                                                                                                                                                                                                                                                                                                                                                                                             | 90  |
| 6.1 | Hand-written source-line counts (excluding generated <code>lex.yy.c</code> and <code>parser.tab.c</code> ). . .                                                                                                                                                                                                                                                                                                                                                      | 98  |
| A.1 | Complete diagnostic code reference. The full text and example trigger for every code are reproduced in <code>docs/semantic-analysis/SEMANTIC_CHECK_MATRIX.md</code> . .                                                                                                                                                                                                                                                                                              | 102 |
| B.1 | Complete IR opcode reference. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                | 103 |
| C.1 | gr0040 ABI register summary. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                 | 104 |

# Listings

---

|      |                                                                                                                                                                                                                                                                  |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1  | Keyword and identifier handling. . . . .                                                                                                                                                                                                                         | 40 |
| 4.2  | Numeric and literal token handling. . . . .                                                                                                                                                                                                                      | 40 |
| 4.3  | Block comment handling using start conditions. . . . .                                                                                                                                                                                                           | 41 |
| 4.4  | Preprocessor directive rejection. . . . .                                                                                                                                                                                                                        | 41 |
| 4.5  | AST construction during parsing. . . . .                                                                                                                                                                                                                         | 41 |
| 4.6  | Expression precedence cascade. . . . .                                                                                                                                                                                                                           | 42 |
| 4.7  | Semantic value propagation via Bison. . . . .                                                                                                                                                                                                                    | 42 |
| 4.8  | Precedence-based dangling-else resolution. . . . .                                                                                                                                                                                                               | 43 |
| 4.9  | Fast-path bump allocator from <code>Semantic/arena.c</code> . . . . .                                                                                                                                                                                            | 43 |
| 4.10 | FNV-1a hash function used in symbol tables. . . . .                                                                                                                                                                                                              | 44 |
| 4.11 | Builtin type singletons. . . . .                                                                                                                                                                                                                                 | 44 |
| 4.12 | Diagnostic emission wrapper. . . . .                                                                                                                                                                                                                             | 45 |
| 4.13 | Modifiable-lvalue validation. . . . .                                                                                                                                                                                                                            | 45 |
| 4.14 | Control-flow enforcement for break and continue. . . . .                                                                                                                                                                                                         | 46 |
| 4.15 | IR module hierarchy. . . . .                                                                                                                                                                                                                                     | 47 |
| 4.16 | The <code>ir_instr_t</code> quadruple, condensed from <code>IR/ir.h</code> . . . . .                                                                                                                                                                             | 47 |
| 4.17 | Replacing direct union reads with opcode-checked accessor macros in <code>IR/ir.c</code><br>and <code>CodeGen/codegen.c</code> . . . . .                                                                                                                         | 47 |
| 4.18 | <code>binop_opcode</code> maps the AST operator and a signedness flag onto a concrete IR<br>opcode. Multiplication, division and modulo deliberately return <code>IR_OP_INVALID</code><br>– the caller must check <code>needs_runtime_call</code> first. . . . . | 48 |
| 4.19 | Predicate for runtime-helper substitution. . . . .                                                                                                                                                                                                               | 49 |
| 4.20 | If-statement lowering schema. . . . .                                                                                                                                                                                                                            | 50 |
| 4.21 | Source for the if-lowering example ( <code>test_files/RegisterAllocation/iftest.c</code> ). . . . .                                                                                                                                                              | 51 |
| 4.22 | IR text for a simple linear program (excerpt). . . . .                                                                                                                                                                                                           | 51 |
| 4.23 | IR for <code>iftest.c</code> . Note the <code>NEQ 0</code> that coerces <code>cond</code> to <code>i1</code> before the conditional<br>branch, and the four blocks of the diamond. . . . .                                                                       | 51 |
| 4.24 | Worklist body for the iterative dataflow. . . . .                                                                                                                                                                                                                | 52 |
| 4.25 | Liveness output for <code>linear.c</code> (excerpt). . . . .                                                                                                                                                                                                     | 53 |
| 4.26 | Interference-graph construction ( <code>CodeGen/RegAlloc/interference.c</code> , condensed). . . . .                                                                                                                                                             | 54 |
| 4.27 | Preference-ordered colour selection from <code>CodeGen/RegAlloc/regalloc.c</code> . . . . .                                                                                                                                                                      | 56 |
| 4.28 | Register allocation for <code>linear.c</code> . . . . .                                                                                                                                                                                                          | 56 |
| 4.29 | Spill cost, from <code>CodeGen/RegAlloc/spill.c</code> . . . . .                                                                                                                                                                                                 | 57 |
| 4.30 | Spill rewrite for <code>force_spill.c</code> (the only vreg of cost 0.40 was spilled). . . . .                                                                                                                                                                   | 57 |
| 4.31 | Prologue emitter from <code>CodeGen/codegen.c</code> . . . . .                                                                                                                                                                                                   | 58 |
| 4.32 | Three-to-two-address translation rules: the commutative emitter <code>emit_binop_rr</code><br>and its non-commutative counterpart <code>emit_binop_rr_nc</code> , from <code>CodeGen/codegen.c</code> . . . . .                                                  | 59 |
| 4.33 | Comparison and branch synthesis: <code>cmp_to_branch</code> maps an IR comparison op-<br>code to a branch mnemonic plus an operand-swap flag, from <code>CodeGen/codegen.c</code> . . . . .                                                                      | 60 |
| 4.34 | Parallel-copy algorithm, described inline in <code>CodeGen/codegen.c</code> . . . . .                                                                                                                                                                            | 61 |
| 4.35 | Register aliases and stack macros in <code>abi.m4</code> . . . . .                                                                                                                                                                                               | 62 |
| 4.36 | Macro expansion example. . . . .                                                                                                                                                                                                                                 | 62 |
| 4.37 | Flex pattern definitions used by the lexer. . . . .                                                                                                                                                                                                              | 63 |
| 4.38 | Identifier and symbol handling in the lexer. . . . .                                                                                                                                                                                                             | 63 |
| 4.39 | Branch condition tokens. . . . .                                                                                                                                                                                                                                 | 64 |

|      |                                                                                                                                                                                                                                                                                                                                                               |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.40 | Bison token and union declarations. . . . .                                                                                                                                                                                                                                                                                                                   | 64 |
| 4.41 | RI-format instruction handling with forward references. . . . .                                                                                                                                                                                                                                                                                               | 64 |
| 4.42 | Hash function used for symbol lookup. . . . .                                                                                                                                                                                                                                                                                                                 | 65 |
| 4.43 | Insertion with separate chaining for collision handling. . . . .                                                                                                                                                                                                                                                                                              | 65 |
| 4.44 | Instruction encoding with label resolution in the second pass. . . . .                                                                                                                                                                                                                                                                                        | 66 |
| 5.1  | Body of <code>scripts/test_compile_all.sh</code> . . . . .                                                                                                                                                                                                                                                                                                    | 67 |
| 5.2  | Live output of <code>scripts/test_compile_all.sh</code> with no failing tests. . . . .                                                                                                                                                                                                                                                                        | 68 |
| 5.3  | The C source. . . . .                                                                                                                                                                                                                                                                                                                                         | 69 |
| 5.4  | Final assembly emitted for <code>linear.c</code> . . . . .                                                                                                                                                                                                                                                                                                    | 69 |
| 5.5  | Register allocation for <code>iftest.c</code> . With 17 vregs and a low maximum simultaneous live set, no spilling is needed; <code>%v10</code> and <code>%v16</code> are pre-coloured to <code>r1</code> because they feed the two RET terminators. . . . .                                                                                                  | 70 |
| 5.6  | Final assembly emitted for <code>iftest.c</code> (excerpt: <code>bb0</code> is the test, <code>bb1</code> is the then-arm, <code>bb3</code> is the else-arm). Note the BNE <code>t0, r0</code> pattern that branches when the condition is non-zero – the codegen synthesises the <code>neq 0-then-branch</code> pair from the IR's predicate. . . . .        | 70 |
| 5.7  | Spill rewrite for <code>force_spill.c</code> . <code>%v0</code> (cost 0.40, 2 references on a degree of 5) is the cheapest spill candidate; the rewrite stores it to <code>%slot0</code> after its definition and reloads it (as <code>%v8</code> ) just before its use, leaving four call-live vregs that fit comfortably into <code>r8–r11</code> . . . . . | 71 |
| 5.8  | Rejection test for function pointers ( <code>test_unsupported_funcptr.c</code> ). . . . .                                                                                                                                                                                                                                                                     | 72 |
| 5.9  | Rejection test for struct copying ( <code>test_unsupported_struct_copy.c</code> ). . . . .                                                                                                                                                                                                                                                                    | 72 |
| 5.10 | Rejection test for array initializers ( <code>test_unsupported_aggregate_init.c</code> ). . . . .                                                                                                                                                                                                                                                             | 72 |
| 5.11 | Terminal output demonstrating the explicit rejection of unsupported C features. . . . .                                                                                                                                                                                                                                                                       | 72 |
| 5.12 | Struct field read and write ( <code>use_point</code> ). . . . .                                                                                                                                                                                                                                                                                               | 73 |
| 5.13 | Assembly for <code>use_point</code> : leaf prologue and GEP-based field addressing. . . . .                                                                                                                                                                                                                                                                   | 73 |
| 5.14 | 2-D array allocation and indexed store/load ( <code>sum_corners</code> ). . . . .                                                                                                                                                                                                                                                                             | 74 |
| 5.15 | Assembly for <code>sum_corners</code> : row-stride multiply via SLL and stack-passed fourth argument. . . . .                                                                                                                                                                                                                                                 | 74 |
| 5.16 | Five-argument function: <code>d</code> and <code>e</code> overflow past the three register arguments and are passed on the stack. . . . .                                                                                                                                                                                                                     | 76 |
| 5.17 | Assembly for <code>five_args</code> : prologue loads of two stack-passed parameters. . . . .                                                                                                                                                                                                                                                                  | 76 |
| 5.18 | Nested call: result of <code>h</code> must survive across the call to <code>g</code> . . . . .                                                                                                                                                                                                                                                                | 77 |
| 5.19 | Assembly for <code>nested_calls</code> : non-leaf prologue with <code>lr</code> save and call-live value threading. . . . .                                                                                                                                                                                                                                   | 77 |
| 5.20 | Signed and unsigned division lowered to runtime-library calls. . . . .                                                                                                                                                                                                                                                                                        | 78 |
| 5.21 | Assembly for <code>signed_div</code> and <code>unsigned_div</code> : division lowered to <code>__divs</code> helper calls. . . . .                                                                                                                                                                                                                            | 78 |
| 5.22 | Compound conditions with <code>&amp;&amp;</code> and <code>  </code> (short-circuit evaluation). . . . .                                                                                                                                                                                                                                                      | 80 |
| 5.23 | Assembly for <code>cond_logic</code> : short-circuit <code>&amp;&amp;</code> and <code>  </code> encoded as multi-block boolean-slot sequences. . . . .                                                                                                                                                                                                       | 80 |
| 5.24 | Example assembly source file. . . . .                                                                                                                                                                                                                                                                                                                         | 91 |
| 5.25 | Step 1 output: symbol table and IR dump. . . . .                                                                                                                                                                                                                                                                                                              | 92 |
| 5.26 | Step 2 output: 16-bit hex object file. . . . .                                                                                                                                                                                                                                                                                                                | 92 |
| 5.27 | Accumulating sum with overflow reset ( <code>main</code> ). . . . .                                                                                                                                                                                                                                                                                           | 92 |
| 5.28 | Assembly output for <code>sum</code> and <code>main</code> (custom 16-bit RISC ISA). . . . .                                                                                                                                                                                                                                                                  | 93 |

# Chapter 1

## Introduction

---

### 1.1 Contextualization

---

The gr0040 16-bit RISC softcore developed in the previous semester [3] is a fully functional processor capable of booting, executing interrupt-driven programs, and achieving timing closure on a Zybo Z7-10 at 50 MHz. However, development is currently limited to assembly-level programming.

This project addresses that limitation by providing a complete Subset-C compiler targeting the gr0040 architecture, using the same ABI and integrating with the existing two-pass assembler. The compiler enables high-level program development for the platform while preserving full compatibility with the existing hardware and toolchain.

A new compiler was chosen instead of retargeting existing toolchains such as LLVM, GCC, or SDCC due to the architectural characteristics of the gr0040 ISA. In particular, its two-address instruction format and lack of hardware multiply/divide operations would require substantial backend modification regardless of the chosen framework. Developing a dedicated compiler also ensures explicit control over each compilation stage, which is essential in a hardware–software co-design environment where processor and compiler design evolve together. The resulting system forms a self-contained toolchain, with conceptual grounding in standard compiler design principles [1].

### 1.2 Problem Statement

---

The gr0040 processor lacks a high-level language toolchain. Writing programs in raw assembly is error-prone, time-consuming, and impractical beyond small programs. The problem is to design and implement a complete, self-contained C compiler that: accepts a well-defined subset of C11 [4] with deterministic diagnostics; produces gr0040 assembly conformant to the project ABI; handles real register pressure arising from the absence of hardware multiply/divide and the limited register file; and is reproducible and extensible by future students.

### 1.3 Motivation

---

Existing general-purpose compiler toolchains are poorly suited to the gr0040 ISA. Its two-address format, lack of hardware multiply/divide, and limited register file require substantial backend adaptation, meaning that retargeting LLVM, GCC, or SDCC would still involve significant low-level work.

Beyond technical constraints, the project is also motivated by pedagogy. Implementing a compiler from scratch makes each compilation stage explicit, exposing the full contract between frontend, IR, and backend rather than hiding it behind existing infrastructure. This is particularly valuable in a hardware–software co-design setting, where compiler structure is influenced directly by processor design decisions and vice versa.

Finally, developing the compiler alongside the processor enables tight integration with the ABI and toolchain, ensuring that both evolve together as a coherent system rather than as independently designed components.

## 1.4 Requirements

---

### 1.4.1 Functional Requirements

We set five concrete goals at the start of the project. Every goal is testable against either the in-tree test corpus (Section 5) or the quantitative measurements in the Evaluation chapter (Section 5.14).

- **Accept Subset-C.** Accept a substantial subset of C11 (the surface defined in Section 3.1.3.8) and reject all unsupported constructs with deterministic, code-namespaced diagnostics (SEM### / IR###).
- **Ensure ABI conformance.** Generate gr0040 assembly compatible with the project's two-pass assembler without modification, including correct register usage, stack layout, and call sequences as defined by the ABI specification (Appendix C).
- **Implement register allocation.** Implement a five-stage register allocator (liveness analysis, interference graph construction, pre-colouring, Chaitin–Briggs colouring with George conservative coalescing, and iterative spill rewriting) that converges in a finite number of iterations.
- **Achieve test-suite parity.** Pass 100 % of the in-tree regression suite (currently 38 tests covering IR validation and register allocation).
- **Provide a reproducible toolchain.** Deliver a self-contained toolchain (compiler binary, runtime helpers, regression harness, and this report) that can be built, tested, and extended without external dependencies beyond `flex`, `bison`, `gcc`, and `make`.

### 1.4.2 Non-Functional Requirements

- **Reproducibility.** The compiler shall build from source on Linux with no dependencies beyond `flex`, `bison`, `gcc`, and `make`.
- **Diagnostic quality.** Every rejected program shall produce at least one diagnostic with a unique code that test scripts can match programmatically.
- **Extensibility.** Each compilation stage shall expose a well-defined contract so that optimisation passes and new language features can be added without modifying existing stages.

## 1.5 Constraints

---

### 1.5.1 Technical Constraints

- The target ISA is fixed: gr0040 16-bit RISC, two-address arithmetic, no hardware multiplier or divider, word-addressed BRAM.
- The generated assembly must be consumable by the project's existing two-pass assembler without modification.
- The compiler must be written in C, with Flex for lexical analysis and Bison for parsing.
- The register file provides 16 registers, of which 10 are allocatable (excluding `r0`, `fp`, `sp`, `lr`, `gp`, and the codegen-reserved `t3`).

### 1.5.2 Non-Technical Constraints

- The project timeline is one semester, which precludes implementing machine-independent optimisation passes (constant folding, dead-code elimination, copy propagation).
- The compiler must be understandable by future students; code clarity is a priority alongside performance.
- No floating-point hardware is available on the gr0040; software-float support is deferred to future work.

# Chapter 2

## Analysis

---

This chapter surveys the compiler-engineering landscape that frames our design decisions. We are by no means the first to write a C compiler, nor the first to target a small embedded ISA. Where existing work could plausibly have been retargeted instead of being reinvented, we explain why we wrote our own. The Dragon Book [1] is the textbook reference throughout; Cooper and Torczon [5], Appel [6] and Loudon [7] are alternative entry points to the same material.

### 2.1 Compiler Architectures

---

The three compilers most often cited as reference points – GCC, LLVM and TCC – cover the design space neatly.

**GCC** is a multi-pass, multi-IR compiler whose front end lowers C through several intermediate forms: high *Tree*, *GIMPLE* (high then low, three-address, CFG-based), *SSA* on top of *GIMPLE* for optimisation, and finally *RTL*, a near-machine register-transfer language that drives target-specific backends [2]. The architecture supports dozens of ISAs through retargetable RTL machine descriptions; backends are expensive to write but optimisation passes are reused across all of them.

**LLVM** unifies the middle end around a single, strict-SSA intermediate representation – LLVM IR – which serves as the exchange format between language-specific front ends (Clang for C/C++, Rustc, Swift, etc.) and target-specific code generators [8]. Strict SSA simplifies most optimisation passes but constrains the register allocator: the standard LLVM allocator relies on the *chordal* structure of SSA-derived interference graphs, which a non-SSA IR (such as ours) does not provide.

**TCC** [9] is the opposite extreme: a single-pass compiler with no IR at all. It emits machine code directly during parsing via syntax-directed translation. This sacrifices essentially all optimisation in exchange for compile-time speed; TCC is the de facto fastest C compiler in existence.

This compiler positions itself between GCC and TCC. Like GCC, it has a separate IR (a typed three-address CFG; see Section 3.1.4) so that the back end can work on a clean abstraction. Like TCC, the IR is *not* in SSA form – avoiding the complexity of  $\phi$ -node insertion and dominance-frontier computation when no optimisation passes will ever consume them. The result is closer in spirit to GCC’s *GIMPLE before the SSA pass*, which is exactly what we cite as our model.

### 2.2 Intermediate Representations: 3AC, GIMPLE, SSA

---

The Dragon Book [1, Ch. 6] presents intermediate representations along two principal axes: tree-based (concrete / abstract syntax trees) versus linear (three-address code, P-code, LLVM bytecode), and within the linear family, SSA versus non-SSA [1, §6.2].

Three-address code (3AC) limits each instruction to at most one operator on the right-hand side, with up to two source operands and one destination [1, §6.2]. Aho et al. list three concrete representations: *quadruples* (operator, dst, src0, src1, all explicitly stored), *triples* (the destination is implicit in the instruction’s index), and *indirect triples* (an array of pointers into

a triple pool). Our IR is in quadruple form, which makes the data structure simple but slightly less compact than triples; we chose it for ease of inspection during debugging (every instruction is self-describing, see Listing 4.16).

P-code [1, §6.2.4] is the historical alternative – a zero-address stack-based encoding that produces compact bytecode at the cost of frequent stack manipulation operations. We considered and rejected it: the gr0040 has a register file, not a hardware stack-machine, so a stack IR would need lowering twice (to 3AC, then to two-address physical instructions) instead of once.

SSA – the Static Single Assignment form of Cytron et al. [10] – assigns each variable exactly once, with  $\phi$ -functions at control-flow merges to disambiguate values flowing in from different predecessors. SSA dramatically simplifies many global optimisation passes, but its construction requires computing dominance frontiers and the resulting interference graphs are chordal, which assumes an SSA-aware register allocator. Because this project implements zero optimisation passes (Section ??) and uses Chaitin–Briggs (which works on *any* interference graph), the cost of converting to SSA is not amortised over any benefit. We therefore stayed with non-SSA GIMPLE-style 3AC.

## 2.3 Register Allocation Algorithms

---

The register-allocation literature splits into two main camps: *graph colouring* (Chaitin and successors) and *linear scan*.

**Chaitin (1981)** [11] introduced the seminal algorithm: build an interference graph, attempt to colour it with  $K$  colours where  $K$  is the number of physical registers, spill nodes that cannot be coloured, and iterate until convergence. The original Chaitin allocator is *pessimistic* – a node that cannot be coloured during simplification is unconditionally spilled.

**Briggs (1992)** [12] added *optimistic colouring*: nodes pushed onto the select stack as “potential spills” might still be colourable when neighbours are popped first, because their live colours could leave a free slot. The improvement typically halves the number of spills on the same input. This is the algorithm we implement; Briggs’s PhD dissertation also contributed the cost-per-degree spill heuristic that `spill.c` uses.

**George and Appel (1996)** [13] proposed *Iterated Register Coalescing*, which interleaves move coalescing with simplification rather than running them in sequence. Iterated coalescing is more aggressive (eliminates more copies) but can occasionally make worse spill choices because it merges nodes before their full degree is known. We use the *conservative* variant of George’s coalescing test (a copy is merged only when the merged node is provably below  $K$ -degree across the entire merge neighbourhood) but run it once at the start, not interleaved – a deliberate compromise that keeps the allocator simpler and easier to debug.

**Linear scan** (Poletto and Sarkar 1999) [14] is the alternative paradigm. It treats the program as a single linear sequence and assigns colours by scanning live intervals in order, allocating a fresh register at each interval start and freeing it at each end. Linear scan is  $O(N \log N)$ , far faster than the  $O(N^2)$  worst case of graph colouring, and is the allocator of choice in JIT compilers (HotSpot, V8 baseline, LLVM’s fast-path) where compile-time matters more than code quality. The trade-off documented in [14] – linear scan produces  $\sim 10\%$  worse code than graph colouring and lacks a coalescing path – led us to graph colouring even at the cost of a more complex implementation.

## 2.4 Compilers for Small / Embedded Targets

---

Few existing compilers target 16-bit or smaller microcontrollers, which makes positioning our work straightforward.

**SDCC** [15] (the Small-Device C Compiler) is the most mature C compiler for 8-bit and 16-bit micros. It supports the 8051, Z80, AVR, STM8, MCS51, MOS6502 and similar ISAs through a retargetable backend. SDCC's IR is a custom three-address form ("iCode") and its allocator is a heuristic graph colouring allocator. SDCC is a far larger and more mature project than ours and could in principle be retargeted to the gr0040, but the backend rewrite would be comparable in effort to writing our compiler from scratch – with the disadvantage that the resulting SDCC port would no longer be a teaching artefact.

**AVR-GCC** [16] is GCC's official backend for the 8-bit AVR family (Arduino's microcontroller). It demonstrates that GCC's RTL infrastructure scales *down* to 8-bit register files, but the backend definition (`avr.md`, `avr.c`) is a multi-thousand-line description of every addressing mode and instruction.

**MSP430-GCC** [17] is GCC's backend for Texas Instruments' MSP430 – a 16-bit RISC architecture with a register file and a flat condition-flag word, structurally the closest cousin of the gr0040 we found in the wild. The MSP430 has hardware multiply (a co-processor on most variants) but no divide, so its runtime library includes `__divhi3` and similar helpers analogous to our `__divs` / `__divu`. The runtime contract – helpers receive arguments in fixed registers, return in a fixed register, clobber a defined set – is conceptually identical to the one we describe in Section 3.1.7.

**Tiger**, the pedagogic compiler that accompanies Appel's *Modern Compiler Implementation in C* [6], is the closest scholarly relative of this project. Like ours, Tiger has a non-SSA IR, a Chaitin-style allocator with iterated coalescing, and a textbook frame-layout convention. Where Tiger differs is in its target: a hypothetical RISC machine with an idealised ABI, not a real FPGA softcore. We borrow Appel's vocabulary throughout (frame abstraction, activation records, parallel-copy CALL setup) but ground every concept in the actual gr0040.

## 2.5 Assemblers and Retargetable Toolchains

---

Unlike compilers, assemblers are comparatively simple pieces of software: they translate a textual assembly representation into the binary encoding expected by the processor. Nevertheless, assemblers still span a significant design space, particularly regarding retargetability, symbol resolution, and object-file support.

Traditional assembler literature distinguishes between *single-pass* and *multi-pass* assemblers. A single-pass assembler emits code immediately while parsing source lines, requiring forward references to be resolved conservatively or via fixup tables. Multi-pass assemblers instead perform at least one additional traversal after symbol collection, simplifying label resolution and producing cleaner implementations at the cost of slightly longer assembly time. Our assembler follows the classical *two-pass* design described in compiler textbooks [1, 7].

The GNU assembler (**GAS**) is the standard backend assembler used by both GCC and LLVM toolchains. GAS supports dozens of ISAs through target-specific opcode tables and object-file descriptions, integrating tightly with linkers and debuggers via the ELF format. While highly portable and mature, GAS assumes the existence of a complete systems toolchain and operating-system environment, making it substantially more complex than required for a small FPGA softcore such as the gr0040.

**NASM** represents a more lightweight approach. Designed primarily for x86, it provides a clean syntax, explicit control over binary layout, and relatively modular instruction encoding logic. NASM demonstrates that a practical assembler can remain compact while still supporting macros, relocations and multiple output formats. Our assembler adopts a similarly direct approach: instructions are parsed into compact internal representations and encoded through explicit per-opcode routines rather than a large declarative machine-description system.

Assemblers for embedded systems are often tightly coupled to the surrounding compiler infrastructure. For example, the assemblers used by SDCC [15] and AVR-GCC [16] are responsible not only for instruction encoding but also for handling architecture-specific relocation records, startup code conventions and runtime integration. In contrast, our assembler targets a deliberately minimal environment: flat binary output, no linker stage, and a fixed memory layout defined by the FPGA platform itself.

For these reasons, implementing a custom assembler was ultimately simpler than adapting an existing one. Retargeting GAS or another general-purpose assembler would require reproducing significant amounts of target-description infrastructure, whereas the gr0040 ISA is sufficiently small that a dedicated implementation remains both manageable and pedagogically valuable. The resulting assembler is a cross two-pass assembler whose structure mirrors the educational goals of the compiler itself: transparency and simplicity take priority over generality.

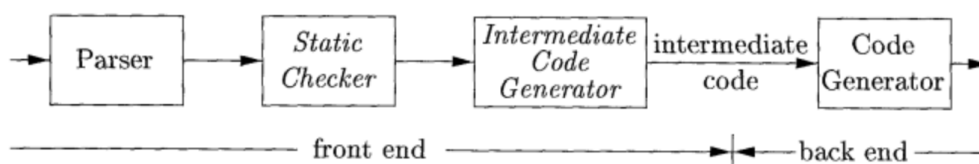
# Chapter 3

## Design

This chapter presents the overall design of the ShivAT toolchain, covering both the C compiler and the assembler. It describes the major architectural decisions and trade-offs across each stage of the compilation and assembly pipelines — from lexical analysis and parsing, through IR generation and register allocation, to final assembly and machine-code emission.

### 3.1 The Compilation Pipeline

A compiler is characterized as a system software, since it consists of a variety of programs that support the operation of a computer [18, Ch. 1]. The compiler is a classic multi-pass design [1, Ch. 1]: the front-end builds a typed AST, the middle-end lowers it to a typed three-address IR, and the back-end allocates physical registers and emits assembly text. Figure 3.1 shows the stages and the artefacts each stage hands to the next.



**Figure 3.1:** End-to-end compilation pipeline. Adapted from [1, Fig. 1.6].

#### 3.1.1 Lexer: Lexical Analysis

The lexical analysis stage is responsible for transforming raw C source code into a stream of tokens that can be consumed by the parser. The design follows the classical regular-expression-based scanner model described in the Dragon Book, where lexical patterns are specified using regular expressions and compiled into a deterministic finite automaton.

The lexer recognises all lexical categories required by the supported subset of C11, which can be grouped as follows:

| Category     | Examples                                                                   | Purpose                 |
|--------------|----------------------------------------------------------------------------|-------------------------|
| Keywords     | <code>int</code> , <code>if</code> , <code>return</code>                   | Language structure      |
| Identifiers  | <code>foo</code> , <code>var_name</code>                                   | User-defined symbols    |
| Literals     | integers, floats, chars, strings                                           | Constant values         |
| Operators    | <code>+</code> , <code>&lt;&lt;</code> , <code>==</code> , <code>+=</code> | Computation             |
| Punctuation  | <code>;</code> , <code>{</code> , <code>(</code> , <code>,</code>          | Grammar structure       |
| Comments     | <code>//</code> , <code>/* */</code>                                       | Ignored during parsing  |
| Preprocessor | <code>#define</code> , <code>#undef</code>                                 | Recognised but rejected |

**Table 3.1:** Lexical categories recognised by the lexer.

A key design requirement is that each token may carry an associated semantic value (via a shared union such as `yylval`), allowing the parser to operate directly on typed data rather than raw strings. This reduces complexity in later stages of the compiler pipeline.

The lexer enforces the *maximal munch* rule to resolve ambiguities between overlapping operators (e.g. `<=` vs `<` vs `<`). Line tracking is maintained to support precise error reporting, and block comments are removed using a dedicated lexical state.

Preprocessor directives are intentionally not fully supported. Instead, they are detected early and rejected to avoid introducing a macro-expansion phase into the compiler pipeline.

### 3.1.2 Parser: Syntax Analysis

The parser is based on an LALR(1) bottom-up parsing strategy as described in the Dragon Book. Its main role is to take the token stream produced by the lexer and verify that it conforms to the grammar rules of the language while simultaneously constructing an Abstract Syntax Tree (AST) representation of the program.

A central design decision is that the AST is built incrementally during reductions, meaning that every grammar rule directly contributes to the construction of the final program tree. This avoids the need for a separate tree-building pass and ensures a tight coupling between syntactic structure and semantic representation.

The grammar is organised in a layered structure that mirrors the ISO C11 specification [4]:

- **Translation Unit Layer** – represents a complete program as a sequence of global declarations and function definitions.
- **Declaration Layer** – handles variable declarations, function signatures, and type specifiers.
- **Statement Layer** – includes control-flow constructs such as `if`, `while`, `for`, and block statements.
- **Expression Layer** – implements operator precedence and associativity using a hierarchical cascade of non-terminals.

Operator precedence and associativity are encoded directly in the grammar using Bison precedence declarations. This avoids ambiguity in expressions and ensures deterministic parse trees that follow ISO C11 semantics. Classical ambiguities such as the dangling-else problem are resolved using precedence rules, ensuring that each program has a unique syntactic interpretation.

A key design requirement is that AST construction happens during parsing, rather than as a separate phase. Each grammar reduction directly produces a node in the tree, ensuring a single-pass construction of the program representation.

The AST is composed of nodes, where each node represents a syntactic construct of the language. Each node contains:

- A **node type** identifying the syntactic construct (e.g. function, statement, expression, operator).
- A **node value** storing optional data such as identifiers, constants, or operator symbols.
- A **type hint** used when type information is locally available.
- A pointer to the **first child** representing the hierarchical structure of the AST.
- A pointer to the **next sibling** used to represent multiple children in a linear chain.
- A **source line number**, used for error reporting and debugging.

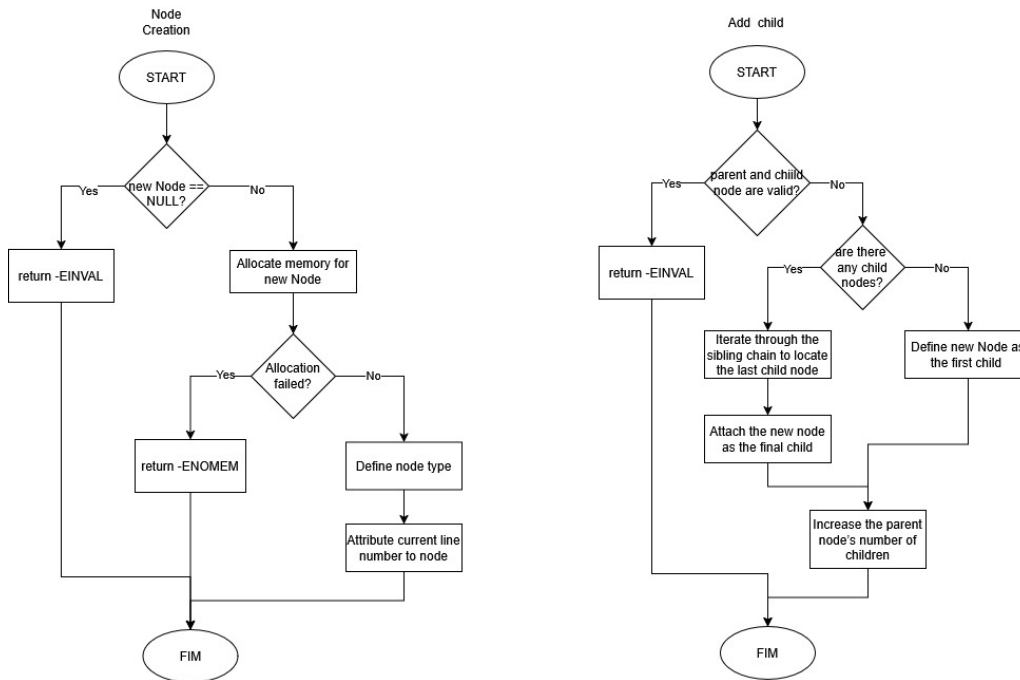
The AST follows a *left-child / right-sibling representation*, which allows each node to have an arbitrary number of children while keeping a simple pointer-based structure. This design avoids dynamic array overhead and ensures efficient memory usage while preserving full structural flexibility.

Node creation occurs at specific grammar reductions. When a rule is matched, a new node is allocated for the corresponding syntactic construct, and its children are attached according to the grammatical structure. Children are always inserted in a deterministic order, preserving the semantics of the original program.

The sibling mechanism is used to represent sequential constructs such as statement lists inside blocks, parameter lists in function declarations, and declaration sequences at global scope. This allows linear program structures to be naturally mapped onto linked sibling chains within the AST.

This separation between hierarchical structure (parent-child links) and sequential structure (sibling links) simplifies later compiler stages such as semantic analysis and code generation, where tree traversal patterns remain uniform and predictable.

The resulting AST is therefore both memory-efficient and structurally expressive, allowing complex C programs to be represented using a single unified node model that can be traversed recursively.



**Figure 3.2:** Flowcharts fore Node Creation and Children addition

### 3.1.3 Semantic Analysis

#### 3.1.3.1 Two-Pass Architecture

The semantic-analysis stage is responsible for validating the meaning of the program after syntactic correctness has already been established by the parser. Its role is to enrich the AST with semantic information, verify language constraints, and prepare the program representation for IR lowering and code generation.

The compiler adopts a two-pass semantic architecture inspired by the attribute-evaluation model described in the Dragon Book. The design separates declaration analysis from type

inference, allowing semantic information to be propagated through the AST in a structured and predictable way.

The first pass performs symbol binding and scope construction using a preorder traversal of the AST. Since declarations are encountered before their uses, identifiers can be registered into the symbol table as soon as they appear.

The second pass performs type inference and semantic constraint checking using a hybrid traversal strategy. Expression-level types are inferred bottom-up, while contextual information such as loop state and current function return type flows top-down through the tree.

This architecture approximates inherited and synthesised attributes without implementing a formal attribute-grammar engine.

Some semantic information naturally flows downward through the AST:

- Current scope
- Current function return type
- Loop nesting depth
- Switch nesting depth

Other information is synthesised upward:

- Expression result types
- Operator result types
- Lvalue/rvalue classification
- Constant-expression evaluation

Separating semantic analysis into two passes improves modularity and keeps parsing independent from semantic policy.

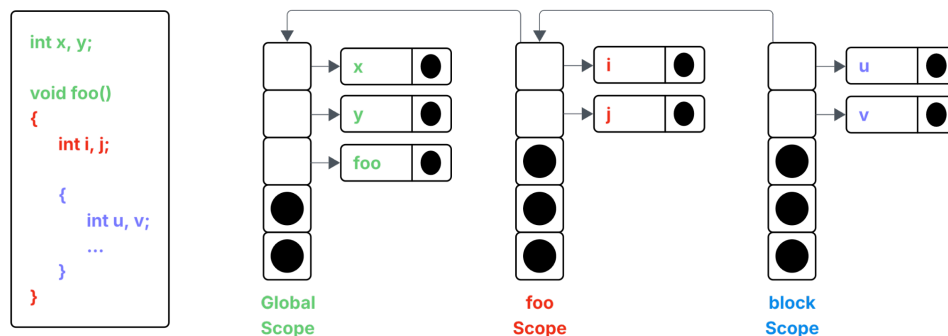
|                      | ST built in the parser                                                                                                                     | ST built in Pass 1 (chosen)                                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Advantages</b>    | <ul style="list-style-type: none"><li>• No extra traversal</li><li>• Immediate semantic errors</li><li>• Lower constant overhead</li></ul> | <ul style="list-style-type: none"><li>• Better modularity</li><li>• Easier debugging</li><li>• Cleaner parser grammar</li></ul>            |
| <b>Disadvantages</b> | <ul style="list-style-type: none"><li>• Semantic/parser coupling</li><li>• Harder maintenance</li><li>• Reduced readability</li></ul>      | <ul style="list-style-type: none"><li>• Extra traversal</li><li>• Slight runtime overhead</li><li>• Delayed semantic diagnostics</li></ul> |

**Table 3.2:** Trade-offs between constructing the symbol table inside parser actions versus using a dedicated semantic pass.

### 3.1.3.2 Scope and Symbol Table Design

The semantic analyser uses a hierarchical scope system that mirrors the lexical nesting structure of the source program. Each scope represents a distinct declaration region and stores all identifiers visible within that lexical context.

Scopes are organised as a rooted tree where every scope maintains a pointer to its enclosing parent scope. This structure allows identifier lookup to follow standard lexical-scoping rules by searching outward through progressively enclosing scopes until a matching declaration is found.



**Figure 3.3:** Rooted hierarchy of lexical scopes used during semantic analysis. Each scope stores its own hash table of symbols and a pointer to its enclosing parent scope.

The active scope chain is maintained through a scope stack during AST traversal. Entering a function body, compound block, or loop creates a new scope, while leaving the construct restores the previous scope.

Each symbol-table entry stores semantic information associated with an identifier:

- Identifier name
- Declared type
- Storage-class information
- Function signature information
- Source location
- Definition state

The symbol table distinguishes between redeclaration and shadowing. Redeclaring an identifier in the same scope is illegal, while declaring an identifier with the same name in a nested scope is permitted and treated as shadowing.

The internal implementation uses hash tables to provide near-constant lookup performance while traversing large ASTs.

### 3.1.3.3 Type-System Design

The semantic analyser defines a canonical recursive type system used throughout semantic analysis and IR lowering.

Types are represented using the `type_t` structure, which supports both builtin and recursively derived types.

Builtin categories include:

- `char`
- `short`
- `int`
- `long`

- `float`
- `double`
- `void`

Derived types include:

- Pointers
- Arrays
- Functions
- Structs
- Unions
- Enums

The type system is recursive by construction. For example, a pointer to an array of integers is represented as nested type objects where each derived type references another underlying type.

Builtin types are represented as singleton objects. Every reference to a builtin type points to the same canonical instance, making equality checks efficient.

Type equality is structural and recursive rather than nominal. Two types are considered equal if their structure and recursively referenced subtypes are equivalent.

The semantic layer also acts as the bridge between multiple internal type representations:

- `NodeType_t` in the parser
- `VarType_t` parser hints
- `type_t` semantic types
- `ir_type_t` IR-level types

#### 3.1.3.4 AST Annotation Design

The semantic analyser augments AST nodes with semantic metadata during analysis. Rather than modifying the AST structure itself, semantic information is stored in a dedicated annotation table indexed by node pointer.

Each annotation record may contain:

- Inferred semantic type
- Bound symbol-table entry
- Lvalue/rvalue classification
- Constant-expression state
- Address-taken information
- Code-generation restrictions

This separation keeps the parser-generated AST independent from the semantic layer while still allowing later stages to retrieve semantic information efficiently.

The annotation table is heavily used during IR lowering, where type information and value-category information become necessary for code generation.

### 3.1.3.5 Diagnostics Design

The compiler implements a structured diagnostic subsystem shared by all semantic-analysis stages.

Diagnostics are represented as linked records containing:

- Diagnostic code
- Severity level
- Source location
- Formatted diagnostic message

Severity levels are divided into:

- NOTE
- WARNING
- ERROR

Diagnostic codes are namespaced according to subsystem:

- SEM### for semantic-analysis diagnostics
- IR### for IR-lowering diagnostics

This structured approach allows automated test scripts to match exact diagnostic identifiers instead of relying on fragile free-form text.

### 3.1.3.6 Pass 1 Design: Symbol Binding

Pass 1 performs symbol registration and scope construction over the AST. The traversal follows preorder semantics so that declarations are always encountered before their uses.

Function declarations receive additional semantic checks, including:

- Prototype compatibility
- Duplicate-definition detection
- Function-signature consistency

Nested scopes are created dynamically while traversing blocks and control-flow constructs.

The semantic rules enforced during Pass 1 are summarised in Table 3.3.

| Code   | Severity | Rule                                              |
|--------|----------|---------------------------------------------------|
| SEM002 | error    | redeclaration in same scope                       |
| SEM003 | note     | shadowing in nested scope is allowed              |
| SEM004 | error    | prototype / definition signature mismatch         |
| SEM005 | error    | duplicate function definition                     |
| SEM007 | error    | <code>inline</code> invalid on object declaration |
| SEM063 | error    | duplicate enum member                             |
| SEM064 | error    | invalid tag redefinition                          |

**Table 3.3:** Selected Pass 1 semantic rules.

### 3.1.3.7 Pass 2 Design: Type Inference and Constraint Checking

Pass 2 validates expression correctness and enforces language-level semantic constraints.

The traversal combines inherited contextual information with synthesised expression information. Child expressions are analysed before their parent operators so that operator typing rules can be evaluated using fully inferred operand types.

The semantic analyser enforces rules for:

- Arithmetic operators
- Bitwise operators
- Assignment legality
- Function calls
- Return statements
- Aggregate member access
- Control-flow legality

The analyser also classifies expressions as lvalues or rvalues. This is necessary to validate assignments, increment/decrement operators, and address-taking operations.

Control-flow legality is enforced using inherited context counters such as loop depth and switch depth.

Some language constructs are semantically valid but unsupported by the backend. These constructs are marked as code-generation blockers so that later compiler stages can emit clean diagnostics rather than generating partial code.

### 3.1.3.8 Subset-C Support

| Feature                                  | Parser-accepted | Semantic-accepted | IR-lowerable |
|------------------------------------------|-----------------|-------------------|--------------|
| int, char, short, long                   | ✓               | ✓                 | ✓            |
| float, double                            | ✓               | ✓                 | ×            |
| Pointers, arrays, structs, unions, enums | ✓               | ✓                 | ✓            |
| Struct copy / pass-by-value              | ✓               | ✓                 | ×            |
| Function pointers / indirect call        | ✓               | ✓                 | ×            |
| Variadic functions                       | ✓               | ×                 | —            |
| All control-flow constructs              | ✓               | ✓                 | ✓            |
| All operators of §6.5                    | ✓               | ✓                 | ✓            |

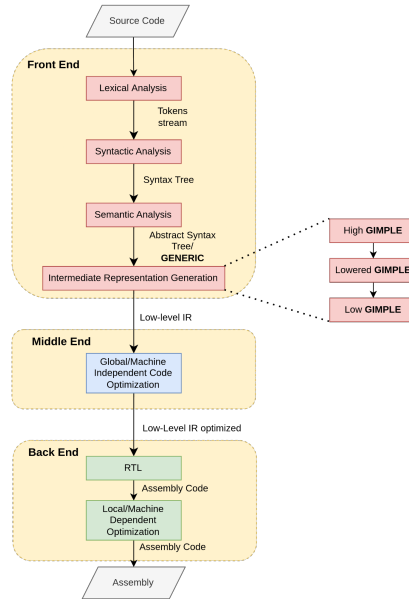
**Table 3.4:** Supported subset of the language after semantic analysis.

## 3.1.4 Intermediate Representation

### 3.1.4.1 Design Choice: GIMPLE-Style 3-Address Code

The IR is a typed three-address code (3AC) organised as a control-flow graph (CFG) of basic blocks. The choice mirrors GCC’s GIMPLE [2] rather than LLVM’s SSA [8]: every value lives in a virtual register, but the form is not in static-single-assignment form, so there are no  $\phi$  nodes. Control-flow merges are handled through stack slots (memory) instead. The trade-off is documented in [6, Ch. 19]: SSA would simplify some optimisations but require a chordal-friendly

allocator that the project does not need; conversely, the GIMPLE-like form is much easier to inspect and reason about by reading the IR dumps.



**Figure 3.4:** GCC’s IR structure, reproduced from [2]. The IR in this project is a simplified version of GIMPLE, with no SSA form.

The 3AC choice itself is the textbook recommendation of Dragon Chapter 6 [1, §6.2]: every instruction has at most one operator on the right-hand side, which keeps data dependencies clear and makes the back-end’s job tractable. Instructions are stored in *quadruple* form (op, dst, src0, src1) following [1, §6.2.3].

### 3.1.4.2 IR Type System

The IR’s type system is intentionally flatter than the semantic type system, because the back-end ultimately speaks in machine widths. The canonical mapping is shown in Table 3.5.

| Semantic type     | IR type        | Width                      |
|-------------------|----------------|----------------------------|
| char, signed char | i8             | 1 byte                     |
| short, int        | i16            | 2 bytes                    |
| long int          | i32            | 4 bytes                    |
| T* pointer        | ptr            | 2 bytes                    |
| T[N] array        | array          | $N \cdot \text{sizeof}(T)$ |
| struct / union    | struct / union | sum / max of fields        |
| void              | void           | —                          |
| boolean predicate | i1             | 1 bit (logical)            |

**Table 3.5:** Mapping from semantic `type_t` to `ir_type_t`. The IR type itself is sign-agnostic; unsignedness of arithmetic is recovered at lowering time from the operand semantic types and selects the appropriate opcode (e.g. SHRU vs SHRS, LTU vs LTS).

The boolean type `i1` is the result type of every comparison instruction and the operand type of conditional branches; it is never materialised in memory.

### 3.1.4.3 Module / Function / Block / Instruction Hierarchy

An IR program is an `ir_module_t` containing zero or more globals and one or more functions. Each function is a linked list of basic blocks, each block is a linked list of instructions, and each block ends with exactly one terminator (GOTO, BRANCH, SWITCH, RET). The four-level hierarchy is illustrated by the structure diagram in Figure 3.4.

### 3.1.4.4 The `ir_instr_t` Quadruple

Every IR instruction is a quadruple (*op*, *dst*, *src0*, *src1*), with a union of opcode-specific auxiliary data attached on the side, following Dragon §6.2.3 [1, §6.2.3]. The auxiliary union is necessary because some opcodes carry information that does not fit in the two source slots: CALL needs a callee name and an argument list, BRANCH needs two target labels, SWITCH needs a case table, and GEP needs the element width.

### 3.1.4.5 Operand Kinds

The `ir_value_t` structure is a tagged union with five variants:

- `IR_VAL_VREG` – a virtual register `%vN`.
- `IR_VAL_SLOT` – a stack slot `%slotN` reserved for a local or a parameter that escaped to memory.
- `IR_VAL_GLOBAL` – a module-level symbol `@name`.
- `IR_VAL_IMM_INT` – an integer literal.
- `IR_VAL_LABEL` – a basic-block label, used as a terminator target.

### 3.1.4.6 Opcode Catalogue

The IR has 40 opcodes (excluding `IR_OP_INVALID`; this count includes MUL, DIVS, DIVU, MODS, MODU, which are always lowered to runtime-library CALLs and so never reach codegen as a real opcode), grouped in seven categories. The full reference table is in Appendix B; one representative per category, using canonical Dragon-style notation [1, §6.2.1]:

- **Constants & moves:** CONST, COPY.
- **Arithmetic:** ADD, SUB, AND, OR, XOR, SHL, SHRS, SHRU, NEG, NOT.
- **Comparisons** (always producing `i1`): EQ, NEQ, LTS, LES, GTS, GES, LTU, LEU, GTU, GEU.
- **Memory:** ADDR\_OF, LOAD, STORE, GEP.
- **Casts:** ZEXT, SEXT, TRUNC, BITCAST.
- **Control:** GOTO, BRANCH, SWITCH, RET.
- **Calls:** CALL (the only call opcode; see Section 4.1.4.3 for runtime-helper calls).

### 3.1.4.7 IR Lowering Overview

The lowering pipeline translates the semantic AST into IR through three specialised phases: declaration lowering, expression lowering, and statement lowering. A shared mutable context carries the current module, function, basic block, control-flow stack, and error counter through all phases.

Declaration lowering handles globals (which become `ir_global_t` entries) and locals (which become `ir_slot_t` reservations on the function’s frame). Expression lowering covers identifier reads, binary operations with operand widening, assignment semantics, and the casting machinery. A critical design decision — imposed by the absence of hardware multiply/divide on the gr0040 — is *runtime-helper substitution*: multiplication, division and modulo operators do not lower to a single IR opcode but to an IR CALL of the appropriate helper (`__mul`, `__divs`, `__divu`, `__mods`, `__modu`); see Section 3.1.7. This keeps the back-end honest: any instruction that escapes lowering as a real opcode is one that the assembler will emit literally. Statement lowering produces the well-known basic-block templates from Dragon Chapter 6.6 [1] for `if/else`, `while`, `do-while`, `for`, and `switch`.

### 3.1.4.8 Aggregate Layout

Field offsets (in 16-bit words) are computed during lowering by walking the AST declaration of the tagged type. Unions always place every field at offset 0. The IR carries only the tag name plus the offset; it does not need to model the layout further.

### 3.1.5 Register Allocator

The register allocator implements the five-stage Chaitin–Briggs design documented in [5, Ch. 13] and [1, §8.8], with the conservative coalescing test from [13]. The pipeline is driven from `main.c` as an iterative loop:

1. **Liveness**: compute `live_in`, `live_out`, `live_after` for every basic block and instruction.
2. **Interference**: build the undirected interference graph and the move-list for coalescing.
3. **Pre-colouring**: pin parameter, return-value and call-result virtual registers to mandatory physical registers, and identify the *call-live* set whose elements must use callee-saved registers.
4. **Colouring** (Chaitin–Briggs with George coalescing): produce a colour assignment or mark spill candidates.
5. **Spill rewriting**: if any vreg failed to colour, rewrite the IR to spill the highest-cost candidate and restart the pipeline.

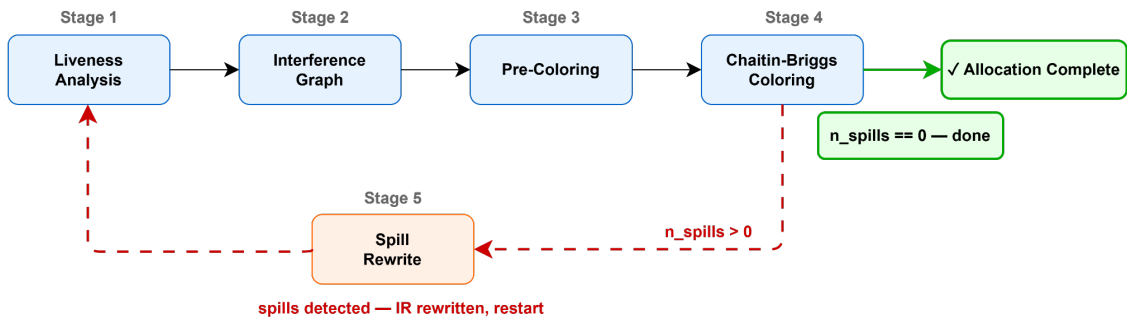


Figure 3.5: The five-stage register-allocation pipeline.

### 3.1.5.1 Liveness Analysis

Liveness is computed using the standard backward dataflow equations [1, §9.2.5]:

$$\text{live\_in}[B] = \text{use}[B] \cup (\text{live\_out}[B] - \text{def}[B]), \quad \text{live\_out}[B] = \bigcup_{S \in \text{succ}(B)} \text{live\_in}[S].$$

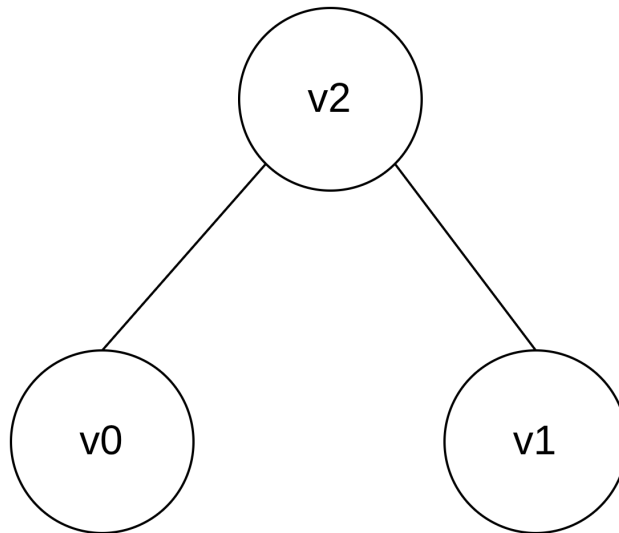
The fixed-point iteration terminates as soon as a full pass produces no change. Per-instruction `live_after` sets are then computed by a backward sweep within each block. The implementation uses a worklist rather than a naive sweep: blocks are enqueued in reverse-postorder for the first pass, and a block whose `live_in` changes re-enqueues all of its predecessors.

The trade-off versus linear-scan / live-interval analysis is as follows: live-interval is faster (one pass) but creates false interferences across branches and loops; iterative dataflow gives exact live sets at every program point and is what GCC and LLVM use.

### 3.1.5.2 Interference Graph

The interference graph is built by walking every instruction once: for every defined virtual register  $x$  and every  $y$  in `live_after` of that instruction, an edge  $x - y$  is added [11]. The data structure stores both an adjacency matrix (a triangular bitset for fast  $O(1)$  lookup) and per-node adjacency lists (for fast neighbour iteration).

A move-list is recorded in parallel: when the instruction is a `COPY` from  $y$  to  $x$ , the source is excluded from the `live_after` set used to add edges, so that  $x$  and  $y$  do not interfere by virtue of the copy itself; the pair is stored for coalescing in Stage 4.



**Figure 3.6:** Interference graph built from the running example.

### 3.1.5.3 Pre-colouring

The pre-colouring stage enforces calling-convention constraints from the ABI [3] *before* the colouring runs. The fixed assignments are shown in Table 3.6.

A second output of this stage is the *call-live set*: the virtual registers that are simultaneously live across at least one call instruction. Because registers `r1–r7` are caller-saved, a call-live vreg

| Case                        | Pinned register | Notes                 |
|-----------------------------|-----------------|-----------------------|
| Function parameters %v0–%v2 | r1–r3 (a0–a2)   | first three only      |
| Return-value source         | r1              | RET operand           |
| CALL result destination     | r1              |                       |
| CALL argument sources       | r1–r3           | up to 3 register args |

Table 3.6: Pre-colour assignments.

cannot reside in any of them, lest its value be clobbered by the call. The colouring stage will only consider r8–r11 (s0–s3) for the call-live set, with  $K = 4$  instead of the ordinary  $K = 10$ .

|                   |                               |
|-------------------|-------------------------------|
| <b>r0 (zero)</b>  | not writable                  |
| <b>r1 (a0/v0)</b> | arg0/ return value 0          |
| <b>r2 (a1/v1)</b> | arg1/ optional return value 1 |
| <b>r3 (a2)</b>    | arg2                          |
| <b>r4 (t0)</b>    | caller-saved temporary        |
| <b>r5 (t1)</b>    | caller-saved temporary        |
| <b>r6 (t2)</b>    | caller-saved temporary        |
| <b>r7 (t3)</b>    | caller-saved temporary        |
| <b>r8 (s0)</b>    | callee-saved                  |
| <b>r9 (s1)</b>    | callee-saved                  |
| <b>r10 (s2)</b>   | callee-saved                  |
| <b>r11 (s3)</b>   | callee-saved                  |
| <b>r12 (fp)</b>   | frame pointer                 |
| <b>r13 (sp)</b>   | stack pointer                 |
| <b>r14 (lr)</b>   | return address register       |
| <b>r15 (gp)</b>   | global pointer                |

**Figure 3.7:** The pre-colouring stage identifies the call-live set. This forces some virtual registers onto specific physical registers ( $K = 4$  callee-saved registers) as per the ABI’s constraints.

### 3.1.5.4 Chaitin–Briggs Colouring with George Coalescing

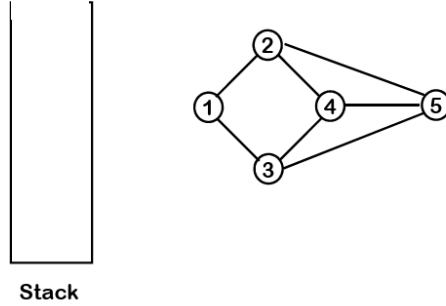
The colouring stage implements the canonical Chaitin–Briggs algorithm [11, 12] with George’s conservative move-coalescing test [13]. The trade-offs against linear scan [14] are: linear scan is  $O(N \log N)$  and simpler, but produces approximately 10% worse code, lacks coalescing, and over-spills when intervals have “holes”. The trade-offs against the more aggressive Iterated Coalescing of George–Appel are: iterated coalescing finds more coalescing opportunities, though it might occasionally make worse spill choices.

### 3.1.5.5 Move Coalescing (George Test)

For every move copy %a, %b: %a and %b are merged into a single node iff (i) they do not already interfere and (ii) for every high-degree neighbour  $n$  of %a,  $n$  already interferes with %b. The conservative test (only merge when colourability is provably preserved) is what makes the algorithm Chaitin–Briggs rather than George–Appel.

### 3.1.5.6 Simplification

A working copy of the graph is kept; nodes with current degree  $< K$  are pushed onto a LIFO “select stack” (Fig. 3.8) and removed from the graph (their neighbours’ working degrees decrease accordingly). When no low-degree node exists, the highest-degree non-precoloured node is pushed as a *potential spill* and similarly removed. The process continues until the graph is empty.



**Figure 3.8:** Simplification: low-degree nodes are removed and pushed onto the select stack; when no low-degree node exists, a potential spill is removed instead.

### 3.1.5.7 Colour Selection

Nodes are popped from the select stack in LIFO order. For each, the forbidden-colour set is gathered from already-coloured interfering neighbours, and the first colour not in the forbidden set is assigned following the preference order:

- Normal vregs: **r4–r6**, then **r8–r11**, then **r1–r3**. **r7** is reserved for codegen scratch (Section 3.1.6.2).
- Call-live vregs: **r8–r11** only.

If no colour is available, the node is marked `REGALLOC_SPILL` and the iterative loop will retry after Stage 5.

### 3.1.5.8 Alias Propagation

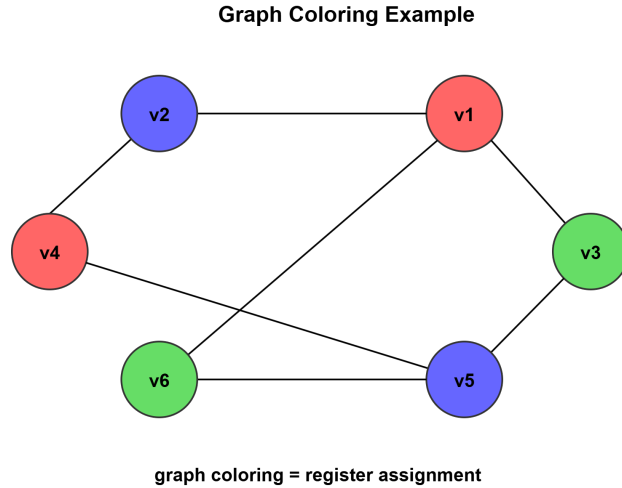
A coalesced node inherits its representative’s colour, propagating it back to every original COPY source/destination pair so that the copy can be eliminated.

### 3.1.5.9 Spill Rewriting

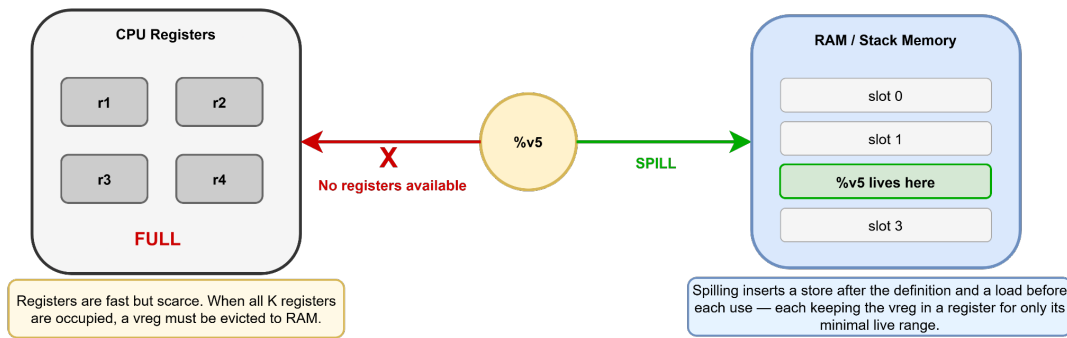
When Stage 4 leaves any `REGALLOC_SPILL` marks, the IR is rewritten. The cost of spilling a vreg is approximated by the heuristic

$$\text{cost}(v) = \frac{|\text{uses}(v)| + |\text{defs}(v)|}{\text{deg}(v)},$$

following the Chaitin formulation [11]; nodes with  $\text{deg}(v) = 0$  receive a very large cost (spilled last). The chosen candidate has its definition followed by an immediate STORE, and every use preceded by an immediate LOAD, so its live range shrinks to a single instruction (degree  $\leq 2$ , trivially colourable on the next iteration).



**Figure 3.9:** Colour selection: the first colour not in the forbidden set is chosen according to the preference order. The color represents the physical register assigned to the virtual register, and edges represent interferences. If no color is available, the node is marked for spilling.



**Figure 3.10:** Spill rewriting: the chosen vreg is spilled to a new stack slot, with a store after its definition and a load before every use.

### 3.1.5.10 Convergence and Termination

In practice, the pipeline converges in a single iteration on 34 of the 38 tests; the four exceptions are `force_spill.c`, `spill_call_chain.c` and `spill_pressure_basic.c` (the three synthetic spill stress tests in Section 5.17) and `super_test.c` (317 vregs across one function), each of which takes exactly one additional iteration.

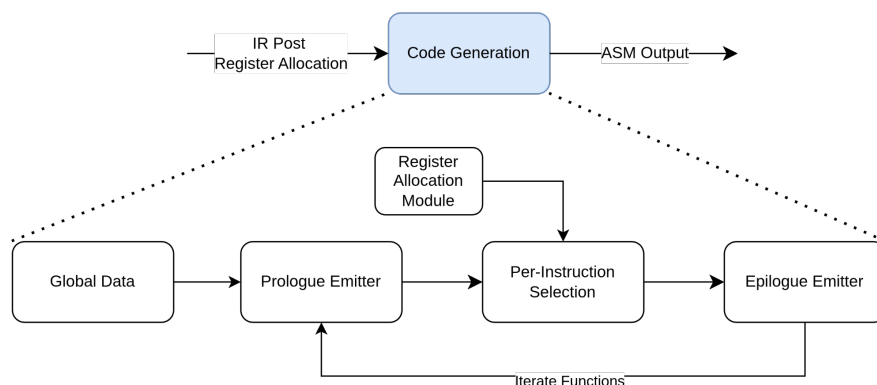
The loop terminates by the standard Chaitin–Briggs argument [5, 12]: each failed colouring round spills at least one virtual register, and the short-lived temporaries introduced to rewrite that register’s uses and defs have live ranges spanning at most a single instruction and so are trivially  $K$ -colourable. The set of vregs that can still be spilled therefore strictly shrinks each round, and an upper bound on the number of iterations is  $|V|$ . The Briggs argument relies on the invariant that reload temporaries introduced during spill rewriting cannot themselves be spilled; on programmes whose reload pattern violates this invariant, the current heuristic does not converge (see Section 5.22). This is a property of the present spill-cost choice rather than of Chaitin–Briggs itself.

### 3.1.6 Code Generator

#### 3.1.6.1 Target ISA Overview

The gr0040 is a 16-bit big-endian RISC processor documented in detail in the companion FPGA report [3]. The features relevant to the back-end are: a fixed 16-bit instruction word; sixteen general-purpose registers `r0–r15` (with `r0` hardwired to zero); five instruction formats (RR, RI, RRI, I12, BR); a flat condition-flag (PSR) status; and word-addressed BRAM. The instruction set is two-address for arithmetic (`op rd, rs ⇒ rd ← rd op rs`); the back-end is responsible for the 3-address  $\rightarrow$  2-address rewriting (Section 3.1.6.4).

Functions that make no calls take a leaf-function fast path that elides `PUSH(lr)/POP(lr)` (and, for the ultra-leaf case, the entire frame); the implementation and measured savings are covered in Section 5.20.



**Figure 3.11:** The code generator translates the register-allocated IR into gr0040 machine code, handling instruction selection, register assignment, and frame management according to the target ISA and ABI conventions.

#### 3.1.6.2 Register Conventions and the t3 Reservation

The ABI assigns roles as in Table 3.7.

| Register | ABI alias | Role                   | Allocatable?         |
|----------|-----------|------------------------|----------------------|
| r0       | zero      | hardwired zero         | no                   |
| r1       | a0        | arg 0 / return value   | yes (caller-saved)   |
| r2, r3   | a1, a2    | args 1, 2              | yes (caller-saved)   |
| r4–r6    | t0–t2     | temporaries            | yes (caller-saved)   |
| r7       | t3        | <i>codegen scratch</i> | <b>NO</b> – reserved |
| r8–r11   | s0–s3     | callee-saved           | yes                  |
| r12      | fp        | frame pointer          | no                   |
| r13      | sp        | stack pointer          | no                   |
| r14      | lr        | link register          | no                   |
| r15      | gp        | global pointer         | no                   |

**Table 3.7:** gr0040 register conventions. `t3` is excluded from the allocator preference list and reserved exclusively for the code generator.

`t3`’s reservation is essential. The codegen needs a register that is *guaranteed* never to hold a live virtual register, so that it can:

- materialise immediates and global addresses through `LI(t3, ...)` without colliding with any allocated vreg. Example: lowering `x = 1000` when `r2` holds `x` and every other allocatable register is also live emits `LI(t3, 1000); MOV(r2, t3)` – with no reserved scratch the materialisation has nowhere to land;
- save the second operand of a non-commutative in-place op (`SUB`, `SRL`, `SRA`) when the destination and the second source alias. Example: the IR triple `%v3 = %v1 - %v2` where the allocator assigns `%v3` and `%v2` to the same physical register `r4` lowers to `MOV t3, r4; MOV r4, r2; SUB r4, t3` – without `t3`, the rewrite to two-address form would clobber `%v2` before it is read;
- break parallel-copy cycles in `CALL` setup (Section 3.1.6.5). Example: a call site `f(a, b)` where `a` currently lives in `r2` (the slot `b` must occupy) and `b` in `r1` (the slot `a` must occupy) – the cycle `r1↔r2` is broken by spilling one end through `t3` (`MOV t3, r1; MOV r1, r2; MOV r2, t3`).

Without this reservation, every materialisation would have to push and pop a temporary on the stack; with it, those operations become a single `RR` instruction with zero stack traffic.

### 3.1.6.3 Frame Layout

The frame is full-descending and word-aligned, exactly as documented in [6, Ch. 6]:

```
+-----+ <- sp on entry
| saved fp    | <- fp+0 (PUSH(fp), then MOV(fp,sp) lands fp here)
| saved lr    | <- fp-1 (PUSH(lr) after MOV)
| saved s0..s3 | <- fp-2 .. fp-(1+cc) (cc = #callee-saved used)
| slot 0      | <- fp - (2 + cc)
| slot 1      | <- fp - (3 + cc)
| ...        |
| slot n-1    | <- fp - (1 + n + cc) = sp after allocation
+-----+
```

The prologue saves `fp`, establishes a new frame pointer with `MOV fp, sp`, saves `lr` and the actually-used callee-saved registers, and finally allocates the local slots with `ADDI sp, sp, #-n`. The epilogue undoes these in reverse order and ends with `RET`, which is the canonical jump-through-link encoding. The prologue and epilogue branch on a leaf-function flag: leaf functions skip `PUSH(lr)/POP(lr)` entirely, and ultra-leaf functions (no slots, no callee-saved,  $\leq 3$  params) skip the entire frame.

### 3.1.6.4 Per-Opcode Emission Strategy

For each IR opcode, the codegen invokes a dedicated emitter. The two non-trivial cases are described below; the rest follow the standard recipe of Dragon Chapter 8 [1, §8.6] adapted to the `gr0040`'s two-address ISA. Cooper–Torczon [5, Ch. 7] and Appel [6, Ch. 9] provide alternative treatments for the three-to-two-address rewriting and the comparison ladder.

**Two-Address Arithmetic** The ISA encodes `op rd, rs` as `rd ← rd op rs`. The IR's three-address `ADD %dst, %a, %b` therefore needs case analysis:

- if `dst = a`: emit `ADD dst, b`;

- if `dst = b` and `op` is commutative: emit `ADD dst, a;`
- otherwise: emit `MOV dst, a; ADD dst, b.`

For non-commutative operators (`SUB`, `SRL`, `SRA`) the special case `dst = b` but `dst  $\neq$  a` requires saving `b` to `t3` first: `MOV t3, b; MOV dst, a; SUB dst, t3.`

**Comparisons and Branches** The ISA does not directly encode every comparison: `GTS`, `GES`, `GTU`, `GEU` are missing. The codegen synthesises them by swapping operands (`a > b` becomes `b < a`) and the inequality `NEQ` is realised by swapping the true and false targets of `BEQ`.

### 3.1.6.5 CALL Setup with Parallel Copy

The `CALL` instruction is the most intricate emitter. The vregs that hold the arguments must end up in `a0–a2` (and the rest on the stack), but the source registers may form a permutation, possibly including cycles:

$$r1 \leftarrow a, \quad r2 \leftarrow r1, \quad r3 \leftarrow r2.$$

Issuing the moves naively would clobber `r1` before `r2` has read it. The codegen runs a three-phase parallel-copy algorithm [6, §19.6]:

1. **Topological order** – compute a directed graph of register-to-register copies and emit the moves in Kahn’s order so that every move’s source is read before any later move clobbers it.
2. **Cycle break** – the residual graph after topological emission consists exclusively of disjoint cycles. For each cycle, save the first register’s value into `t3`, perform the rotation around the cycle in destination order, and finish with a move from `t3` into the last destination.
3. **Immediate / global lift** – arguments that are immediates or globals (rather than vregs) are loaded last, when all the register sources have already been read.

Stack-bound arguments (parameter index  $\geq 3$ ) are written to the caller’s outgoing-argument area *before* the register copies, to avoid clobbering vreg sources that may also be needed for stack storage.

### 3.1.6.6 Stack-Passed Parameter Loading

The IR contract for parameters is asymmetric: register-passed parameters (index 0–2) are `STORED` to their slots in the function’s prologue IR, but stack-passed parameters (index  $\geq 3$ ) are **not** given an IR `STORE`. The reason is: if they were, the regalloc would have to model all of them as live-in simultaneously, which the IR has no construct for. Instead, the codegen prologue loads each stack-passed parameter directly via `t3`, placing the value in the parameter’s own slot so that any subsequent use behaves identically to a register-passed parameter.

## 3.1.7 Runtime Library

### 3.1.7.1 Why Software Arithmetic

The gr0040 datapath has no hardware multiplier and no hardware divider. Multiplication, division and modulo therefore cannot be expressed as a single ISA instruction; the IR lowering substitutes them with function calls to the runtime helpers `__mul`, `__divs`, `__divu`, `__mods`, `__modu`. The choice keeps the back-end simple (every IR `CALL` is a normal call, never a magical pseudo-instruction) and makes the cost of multi-cycle arithmetic visible at the call site.

### 3.1.7.2 Helpers

| Symbol              | Operands (a0, a1) | Result (a0)              |
|---------------------|-------------------|--------------------------|
| <code>__mul</code>  | two i16           | low 16 bits of product   |
| <code>__divu</code> | two unsigned i16  | quotient                 |
| <code>__modu</code> | two unsigned i16  | remainder                |
| <code>__divs</code> | two signed i16    | quotient (sign-handled)  |
| <code>__mods</code> | two signed i16    | remainder (sign-handled) |

**Table 3.8:** Runtime helpers. Each function follows the project ABI: arguments in `a0`, `a1`; result in `a0`; `a0`–`a2` and `t0`–`t2` clobbered (caller-saved); `t3` is reserved scratch.

### 3.1.7.3 Algorithms

The implementations are detailed in `docs/runtime/Runtime Library Specification.md`; the underlying algorithms are classical [19, §4.3]:

- `__mul` – shift-and-add: 16 iterations, each examining one bit of the multiplier and conditionally adding the multiplicand into a 16-bit accumulator.
- `__divu` – restoring long division: 16 iterations, each shifting the dividend high into the remainder, conditionally subtracting the divisor, and shifting a quotient bit.
- `__divs` / `__mods` – sign-normalise the operands, delegate to `__divu` or `__modu`, then re-apply the sign according to C99 truncation-toward-zero semantics.

### 3.1.7.4 Branch-Mnemonic Caveat

The gr0040 ISA’s BLEU mnemonic, despite the suffix, denotes a *strict* less-than on unsigned values; the  $\leq$  is named BLETU. The runtime helpers and the codegen comparison ladder both rely on this convention; changing one without the other silently breaks the system. The matter is flagged in the runtime spec and reiterated in the codegen spec.

### 3.1.7.5 Performance Bounds

Each helper terminates in  $O(W)$  inner iterations where  $W = 16$  is the word width. Multiplication, division and modulo on the gr0040 are therefore an order of magnitude slower than addition, and tight loops should be hand-written when this matters.

---

## 3.2 Assembler Pipeline

### 3.2.0.1 High-Level Architecture

The overall data flow of the assembler is shown in Figure 3.12. Source code first passes through macro pre-processing, then through the two-pass assembler core, and finally yields a hexadecimal object file. Two shared data structures – the *Opcodes Table* and the *Symbols Table* – bridge the two passes.

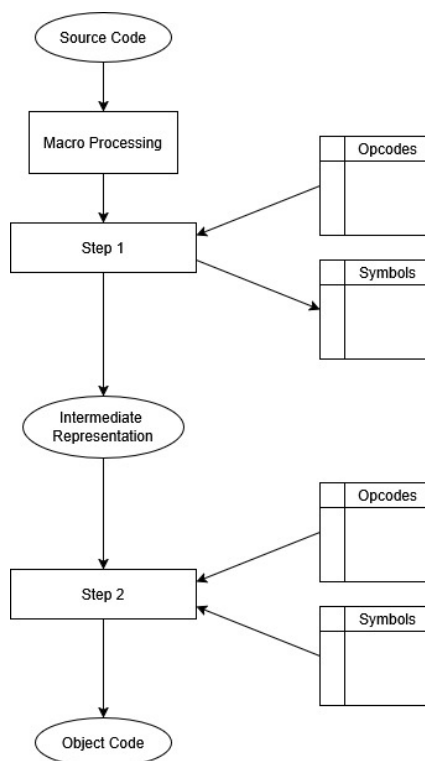


Figure 3.12: Assembler overview.

### 3.2.1 Preprocessor Design

Before assembly proper, the source file is processed by the `m4` macro processor, using and ABI definition file. This stage is responsible for:

- **Register Aliases (ABI)** – mapping human-friendly names (`a0`, `sp`, `lr`, ...) to their physical register numbers (`r1`, `r13`, `r14`, ...).
- **Stack Abstraction** – `PUSH` and `POP` macros expand into the necessary `ADDI/SW/LW` instruction sequences.
- **Pseudo-instructions** – e.g. `MOV(rd,rs)` expands to `ADDI rd, rs, #0`.
- **Function call helpers** and **interrupt support** macros.

The macro expansion is completely transparent to the assembler core: the lexer only ever sees expanded, standard assembly.

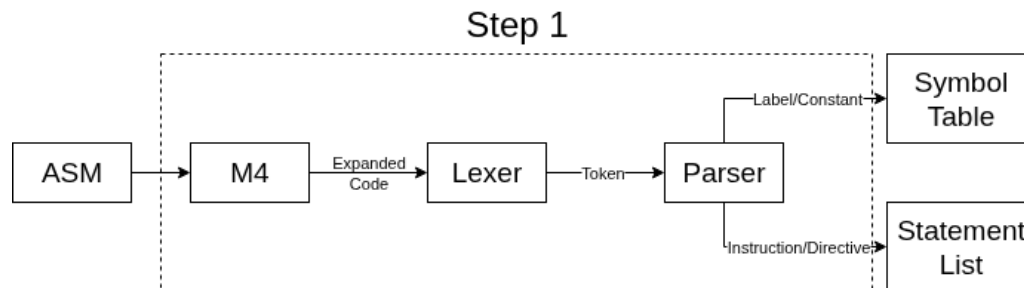
### 3.2.2 Step 1: Parsing and Intermediate Representation Generation

The first stage of the assembler is responsible for transforming raw assembly source code into a structured internal representation suitable for later processing. This stage performs lexical and syntactic analysis, symbol discovery, and initial instruction decoding.

During lexical analysis, the input file is scanned and broken into meaningful tokens such as labels, opcodes, registers, immediates, and directives. These tokens are then fed into the parser, which verifies the syntactic correctness of each instruction according to the grammar of the gr0040 assembly language.

As part of this stage, the assembler consults the opcode table to identify valid instructions and determine their expected operand formats. This allows early detection of invalid or mal-

formed instructions and ensures that each mnemonic is mapped to a well-defined internal opcode representation.



**Figure 3.13:** Step 1 Overview.

In parallel, a symbol table is constructed. Labels encountered in the code are recorded along with their corresponding (eventually resolved) addresses. At this stage, forward references are permitted; unresolved symbols are marked for later resolution in the second pass.

Once parsing is complete, the assembler generates an intermediate representation (IR) of the program. This IR encodes each instruction in a structured, machine-independent form, typically capturing the opcode, operand types, and placeholder fields for addresses or symbols. This representation serves as the bridge between parsing and final code emission, enabling clean separation between analysis and encoding phases.

### 3.2.2.1 Symbols Table

The symbol table is a central data structure in the first pass of the assembler. It is used to associate symbolic labels in the assembly source code with their corresponding memory addresses or instruction offsets. This mechanism allows the assembler to support forward and backward references, enabling labels to be used before they are defined in the source file.

Each time a label is encountered during parsing, the assembler either inserts a new entry into the symbol table or updates an existing unresolved entry. In the case of a first occurrence, the symbol is recorded along with its current program counter value. If the label was previously referenced but not yet defined, the stored placeholder is resolved by filling in the correct address once it becomes available.

The primary operations on the symbol table are insertion and lookup (Figure 3.14). Insertion occurs when a new label definition is encountered, while lookup is used whenever an instruction references a symbolic operand. Efficient lookup is particularly important, as symbols may be referenced many times throughout a program.

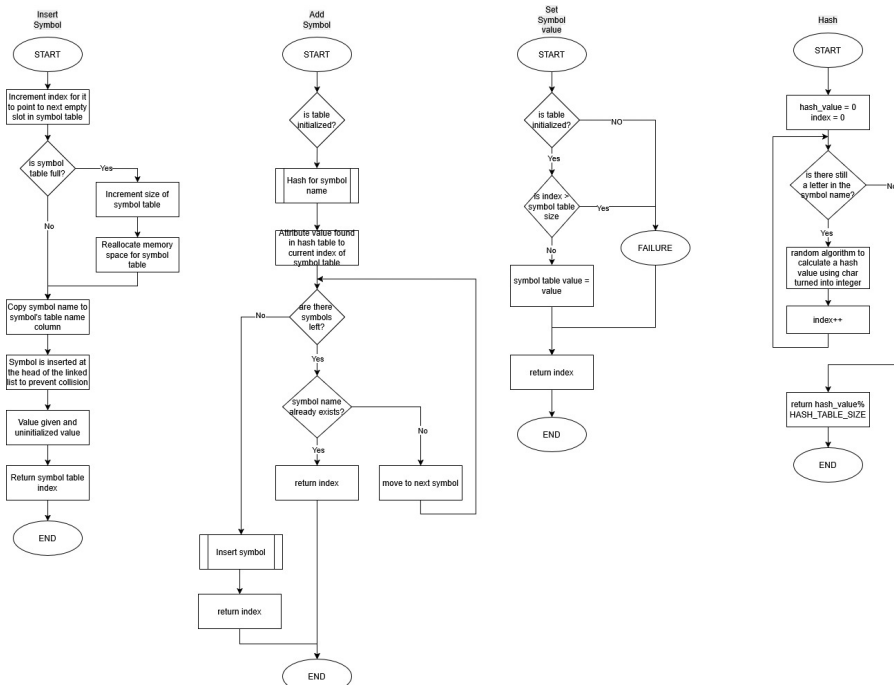
To support fast access, the symbol table is implemented using a hash table. This choice provides average-case constant time complexity  $O(1)$  for both insertion and lookup operations, which is especially beneficial for larger programs with many labels and references. Compared to linear data structures such as lists or arrays, a hash table significantly reduces the overhead of repeated symbol resolution and improves overall assembler performance.

In addition, the hash-based design naturally supports incremental construction during the first pass, allowing symbols to be added and resolved in a single traversal without requiring expensive rescanning of the source code.

To handle hash collisions, the symbol table uses separate chaining with linked lists. In this approach, each bucket of the hash table stores a linked list of entries that share the same hash value. When multiple symbols map to the same bucket, they are appended to the list rather

than overwriting existing entries.

This design ensures that collisions do not result in data loss while preserving the simplicity of the hash table structure. Although worst-case lookup time can degrade to  $O(n)$  within a single bucket, in practice a well-distributed hash function keeps lists short, maintaining near constant-time performance for both insertion and lookup operations.



**Figure 3.14:** Symbols table flowchart for inserting, adding and setting a symbol, as well as computing the hash value

### 3.2.2.2 Intermediate Representation Design

The IR is a dynamic array of `statement_t` records. Each record captures all data needed by Step 2 without re-reading the source. Table 3.9 summarises the fields.

**Table 3.9:** Fields of the `statement_t` structure.

| Field                 | Type                  | Description                                                                                 |
|-----------------------|-----------------------|---------------------------------------------------------------------------------------------|
| <code>opcode</code>   | <code>uint8_t</code>  | Identifies the instruction or directive family (0–15 for instructions, 50+ for directives). |
| <code>fn</code>       | <code>uint8_t</code>  | Function code distinguishing operations sharing an opcode (e.g. ADD vs SUB).                |
| <code>format</code>   | <code>char</code>     | Instruction format: 'R' (RR), 'I' (RI), 'M' (RRI), 'J' (I12), 'B' (BR), 'F' (fixed).        |
| <code>rd</code>       | <code>uint8_t</code>  | Destination register (0–15).                                                                |
| <code>rs</code>       | <code>uint8_t</code>  | Source register (0–15).                                                                     |
| <code>imm</code>      | <code>int16_t</code>  | Immediate value, branch offset, or symbol table index (for labels).                         |
| <code>cond</code>     | <code>uint8_t</code>  | Condition code for branch instructions.                                                     |
| <code>misc</code>     | <code>uint8_t</code>  | Interpretation flag: IMMEDIATE, LABEL, or LINK.                                             |
| <code>line_num</code> | <code>uint32_t</code> | Source line number for error reporting.                                                     |

The statement list grows dynamically via `realloc` when capacity is exhausted. The location counter is updated after each statement is committed, according to the instruction format: 2 bytes for all instructions, 4 bytes for `.word`, 1 byte for `.byte`, and no bytes for `.org/.equ`.

### 3.2.3 Step 2: Object Code Generation

The second pass of the assembler is responsible for transforming the intermediate representation (IR) into final machine-readable object code. This stage uses the previously constructed IR together with the opcode table and the fully resolved symbol table to produce the final binary encoding of the program.

During code generation, each statement is processed sequentially while a location counter is maintained to track the current memory address. As instructions and data directives are emitted, the location counter is updated to reflect their size, ensuring that all code is placed at the correct offsets in memory.

Each IR instruction is translated into its corresponding machine opcode using the opcode table, which defines the binary encoding and operand format for each instruction in the gr0040 architecture. When symbolic operands are encountered, such as labels used in branch or jump instructions, the assembler queries the symbol table to retrieve their resolved addresses before completing the final encoding.

Once symbol resolution and opcode translation are complete, the assembler produces a 16-bit hexadecimal representation of each instruction. The final output is a linear sequence of these encoded words, forming the complete object code ready for execution. The clear separation between the first pass (parsing and symbol collection) and the second pass (encoding and emission) keeps the design simple and ensures that symbol resolution and code generation remain cleanly decoupled.

# Chapter 4

## Implementation

---

This chapter documents the concrete implementation of each pipeline stage in order, including all data structures, algorithms, and code listings.

### 4.1 Frontend

---

#### 4.1.1 Lexical Analysis

The lexer is implemented using Flex in `lexer.l`. Flex generates a deterministic scanner from regular expressions and associated actions, which are compiled during the build process into C code.

##### 4.1.1.1 Keywords and Identifiers

Keywords are matched directly and return fixed token types. Identifiers are stored as strings and passed to the parser through the semantic value union.

```
1  "int"      { return TOKEN_INT;      }
2  "if"       { return TOKEN_IF;       }
3  "return"   { return TOKEN_RETURN;   }
4
5  {identifier} {
6      yyval.nodeData.sVal = strdup(yytext);
7      return TOKEN_ID;
8  }
```

**Listing 4.1:** Keyword and identifier handling.

##### 4.1.1.2 Numeric and Literal Handling

Numeric values are converted during lexical analysis so that the parser receives fully evaluated constants. This includes support for hexadecimal and decimal integers, floating-point numbers, and character literals with escape sequence handling.

```
1  {hexNumber} {
2      yyval.nodeData.dVal = strtol(yytext, NULL, 16);
3      return TOKEN_NUM;
4  }
5
6  {number} {
7      yyval.nodeData.dVal = strtol(yytext, NULL, 10);
8      return TOKEN_NUM;
9  }
```

**Listing 4.2:** Numeric and literal token handling.

#### 4.1.1.3 Comment Handling

Block comments are handled using Flex start conditions. When a comment is detected, the lexer enters a dedicated state in which all characters are ignored until the closing delimiter is encountered.

```
1  <INITIAL>"/*" { BEGIN(INCOMMENT); }
2
3  <INCOMMENT>{
4      "*/"      { BEGIN(INITIAL); }
5      "\n"      { line_number++; }
6      "[^*\n]+" { }
7      "*"       { }
8  }
```

**Listing 4.3:** Block comment handling using start conditions.

#### 4.1.1.4 Preprocessor Directives

Preprocessor directives are recognised but not supported in the full compilation pipeline. They are therefore rejected immediately after detection, ensuring that the compiler remains free of macro-expansion complexity.

```
1  "#define" [ \t]+ { read_pp_define();
2                      return TOKEN_ERROR; }
3
4  "#undef" [ \t]+ { read_pp_undef();
5                      return TOKEN_ERROR; }
```

**Listing 4.4:** Preprocessor directive rejection.

#### 4.1.1.5 Line Tracking and Error Handling

The lexer maintains a global line counter that is incremented on each newline. This enables precise error reporting in later compilation stages, especially during parsing and semantic analysis.

The parser is implemented using Bison in the file `parser.y`. Bison generates an LALR(1) parsing table and associated C code, which is compiled into the final compiler executable.

### 4.1.2 Syntax Analysis

#### 4.1.2.1 AST Construction

Each grammar rule constructs nodes of type `TreeNode_t` using helper functions such as `NodeCreate` and `NodeAddChild`. These functions ensure that all AST nodes are allocated consistently and linked into a left-child / right-sibling structure.

```
1  if_statement
2      : TOKEN_IF '(' expression ')' statement
3      {
4          NodeCreate(&($$.treeNode), NODE_IF);
```

```
5         NodeAddChild($$.treeNode, $3.treeNode);
6         NodeAddChild($$.treeNode, $5.treeNode);
7     }
8     | TOKEN_IF '(' expression ')' statement TOKEN_ELSE statement
9     {
10        NodeCreate(&($$.treeNode), NODE_IF);
11        NodeAddChild($$.treeNode, $3.treeNode);
12        NodeAddChild($$.treeNode, $5.treeNode);
13        NodeAddChild($$.treeNode, $7.treeNode);
14    }
15    ;
```

**Listing 4.5:** AST construction during parsing.

#### 4.1.2.2 Expression Grammar and Precedence

The expression grammar is implemented as a precedence cascade, where each level corresponds to a different operator precedence class. This ensures that the resulting AST correctly reflects ISO C11 evaluation rules.

```
1 expression
2   : assignment_expression
3   ;
4
5 assignment_expression
6   : logical_or_expression
7   | IDENTIFIER '=' assignment_expression
8   ;
```

**Listing 4.6:** Expression precedence cascade.

#### 4.1.2.3 Semantic Values

The parser uses a Bison union (`yylval`) to propagate semantic information from the lexer. This includes numeric values, identifiers, and AST node pointers. Each reduction passes structured data upward in the grammar tree, enabling direct AST construction during parsing.

```
1 %union {
2     int num;
3     struct TreeNode* treeNode;
4 }
5
6 %token <num> TOKEN_NUMBER TOKEN_REG TOKEN_IDENTIFIER
```

**Listing 4.7:** Semantic value propagation via Bison.

#### 4.1.2.4 Error Handling

The parser performs basic syntactic error detection through Bison’s built-in error recovery mechanisms. Invalid token sequences trigger error productions, allowing the compiler to report syntax errors with line-level precision while continuing parsing where possible.

#### 4.1.2.5 Dangling-Else Resolution

The classic dangling-else ambiguity is resolved using Bison precedence declarations. This ensures that each `else` is associated with the nearest unmatched `if`, following ISO C semantics.

```
1 %nonassoc LOWER_THAN_ELSE
2 %nonassoc TOKEN_ELSE
```

**Listing 4.8:** Precedence-based dangling-else resolution.

Overall, the parser acts as the structural backbone of the compiler frontend. It ensures syntactic correctness, constructs the AST in a single pass, and enforces language semantics such as operator precedence and control-flow binding before passing the program to semantic analysis and code generation.

### 4.1.3 Semantic Analysis

#### 4.1.3.1 Two-Pass Semantic Traversal

The implementation follows the two-pass architecture described above. Pass 1 performs symbol binding using preorder traversal, while Pass 2 performs type inference and semantic validation using postorder traversal.

#### 4.1.3.2 Arena Allocator Implementation

Memory allocation during semantic analysis is handled by a custom arena allocator located in `Semantic/arena.c`. This allocator uses a bump-pointer strategy to minimise allocation overhead and improve cache locality.

Two arenas are maintained:

- **Persistent arena:** stores symbols, types, and semantic structures that must persist across compiler stages.
- **Scratch arena:** used for temporary allocations and reset between passes.

The fast allocation path is shown below.

```
1 void *sem_arena_alloc(sem_arena_t *arena,
2                       size_t size, size_t alignment)
3 {
4     sem_arena_block_t *block;
5
6     block = arena->head;
7
8     if (block) {
9         uintptr_t base = (uintptr_t)(block->data + block->used);
10
11         uintptr_t aligned = (base + (alignment - 1u))
12                        & ~(uintptr_t)(alignment - 1u);
13
14         size_t padding = (size_t)(aligned - base);
15         size_t needed = padding + size;
16
17         if (needed <= (block->capacity - block->used)) {
```

```
18         block->used += needed;
19         return (void *)aligned;
20     }
21 }
22
23 /* slow path omitted for brevity */
24 }
```

**Listing 4.9:** Fast-path bump allocator from `Semantic/arena.c`.

#### 4.1.3.3 Symbol Table Implementation

The symbol table is implemented as a hierarchy of scopes, each containing a fixed-size hash table. Each scope points to its parent scope, forming a lexical tree structure.

The hash function used for symbol lookup is FNV-1a, chosen for its simplicity and low collision rate.

```
1  #define SCOPE_BUCKET_COUNT 127u
2
3  static size_t hash_name(const char *name)
4  {
5      uint32_t hash = 2166136261u;
6
7      if (!name) return 0u;
8
9      while (*name) {
10         hash ^= (unsigned char)*name;
11         hash *= 16777619u;
12         ++name;
13     }
14
15     return (size_t)(hash % SCOPE_BUCKET_COUNT);
16 }
```

**Listing 4.10:** FNV-1a hash function used in symbol tables.

Symbols are inserted at the head of each bucket chain to resolve collisions efficiently while maintaining constant-time average lookup.

#### 4.1.3.4 Type-System Implementation

The type system is implemented in `Semantic/type.c`. Builtin types are represented as static singleton objects to ensure structural identity.

```
1  static const type_t g_type_invalid = {
2      .kind = TYPE_INVALID,
3      .qualifiers = 0u};
4
5  static const type_t g_type_int = {
6      .kind = TYPE_BUILTIN,
7      .qualifiers = 0u,
8      .as = {.builtin = BUILTIN_INT}};
```

```
9
10 static const type_t g_type_void = {
11     .kind = TYPE_BUILTIN,
12     .qualifiers = 0u,
13     .as = {.builtin = BUILTIN_VOID}};
```

**Listing 4.11:** Builtin type singletons.

Derived types (pointers, arrays, functions, structs) are constructed recursively during semantic analysis.

Type comparison is structural, except for builtin types which use direct pointer equality due to singleton representation.

#### 4.1.3.5 Diagnostic Infrastructure Implementation

Diagnostics are stored in a central linked structure shared across all semantic passes. Each diagnostic entry contains a code, severity, source location, and formatted message.

The emission interface is implemented as a thin wrapper:

```
1 static void pass2_emit(pass2_state_t *state,
2                       const char *code,
3                       size_t line,
4                       const char *message)
5 {
6     if (!state || !state->ctx)
7         return;
8
9     (void)diag_emit(&state->ctx->diagnostics,
10                   code,
11                   DIAG_ERROR,
12                   line,
13                   0u,
14                   "%s",
15                   message);
16 }
```

**Listing 4.12:** Diagnostic emission wrapper.

This wrapper ensures safe emission and centralises all diagnostic logic through a single interface.

#### 4.1.3.6 Modifiable Lvalue Check

Assignments and increment/decrement operators require modifiable lvalues. This is enforced using the predicate below.

```
1 static int pass2_is_modifiable_lvalue(TreeNode_t *node,
2                                       pass2_state_t *state,
3                                       const type_t *type)
4 {
5     const sem_node_info_t *info =
6         pass2_get_node_info(state, node);
7 }
```

```
8     if (!pass2_is_lvalue_kind(info))
9         return 0;
10
11     if (!type)
12         return 0;
13
14     return (type->qualifiers & TYPE_QUAL_CONST) == 0u;
15 }
```

**Listing 4.13:** Modifiable-lvalue validation.

#### 4.1.3.7 Control-Flow Validation

Control-flow correctness is enforced using loop and switch nesting counters.

```
1  } else if (it->nodeType == NODE_BREAK) {
2
3      state->result.statement_count++;
4
5      if (state->loop_depth == 0u &&
6          state->switch_depth == 0u) {
7
8          pass2_emit(state, "SEM050",
9                      it->lineNumber,
10                     "break only valid inside loop or switch");
11      }
12
13      it = it->p_sibling;
14      continue;
15
16  } else if (it->nodeType == NODE_CONTINUE) {
17
18      state->result.statement_count++;
19
20      if (state->loop_depth == 0u) {
21
22          pass2_emit(state, "SEM051",
23                      it->lineNumber,
24                     "continue only valid inside loop");
25      }
26
27      it = it->p_sibling;
28      continue;
29  }
```

**Listing 4.14:** Control-flow enforcement for break and continue.

## 4.1.4 Intermediate Representation

### 4.1.4.1 Module / Function / Block / Instruction Hierarchy

The four-level hierarchy is implemented precisely as shown in Listing 4.15, declared in `IR/ir.h`.

```
ir_module_t
+-- ir_global_t*      (linked list)
+-- ir_function_list_t* (linked list)
    +-- ir_function_t*
        +-- ir_block_t* (linked list of basic blocks)
            +-- ir_instr_t* (linked list of instructions)
```

Listing 4.15: IR module hierarchy.

### 4.1.4.2 The `ir_instr_t` Quadruple

Listing 4.16 shows the structure declaration condensed from `IR/ir.h`.

```
1  struct ir_instr_s {           /* op dst src[0] src[1] */
2      ir_opcode_t op;
3      ir_value_t dst;           /* IR_VAL_NONE for stores/void calls/ret */
4      ir_value_t src[2];        /* up to two operands */
5
6      union {                   /* per-opcode auxiliary data */
7          struct { long width; } gep;
8          struct { char callee[128];
9                      ir_value_t args[IR_MAX_ARGS];
10                     unsigned arg_count;
11                     int is_void_call; } call;
12         struct { unsigned true_block, false_block; } branch;
13         struct { unsigned default_block;
14                     long case_values[IR_MAX_SWITCH_CASES];
15                     unsigned case_blocks[IR_MAX_SWITCH_CASES];
16                     unsigned case_count; } sw;
17     } as;
18
19     ir_instr_t *next;          /* linked list inside a block */
20 };
```

Listing 4.16: The `ir_instr_t` quadruple, condensed from `IR/ir.h`.

The union is a footgun: writing into `as.call.callee` for an instruction whose opcode is actually GEP silently corrupts `as.gep.width` (the first byte of the callee string overlaps the low byte of the width). The header file documents this and provides the `IR_GEP_*` / `IR_CALL_*` accessor macros that assert on the opcode at access time. As a hardening step, all remaining direct reads of the union in `IR/ir.c` and `CodeGen/codegen.c` were converted to use these macros; Listing 4.17 shows a representative before/after pair. The only exceptions are array-element accesses of the form `&ins->as.call.args[k]`, which require an lvalue address and are kept as direct accesses with an explanatory comment.

```
1  /* Before: direct access, no opcode guard */
```

```

2  fprintf(out, "goto bb%u\n", i→as.branch.true_block);
3  long width = ins→as.gep.width;
4  fprintf(out, "CALL(%s)\n", ins→as.call.callee);
5
6  /* After: macro access, asserts opcode at runtime */
7  fprintf(out, "goto bb%u\n", IR_BRANCH_TRUE(i));
8  long width = IR_GEP_WIDTH(ins);
9  fprintf(out, "CALL(%s)\n", IR_CALL_CALLEE(ins));

```

**Listing 4.17:** Replacing direct union reads with opcode-checked accessor macros in IR/ir.c and CodeGen/codegen.c.

#### 4.1.4.3 IR Lowering

**Driver** The entry point is `ir_lower_translation_unit(root, sem_ctx, name)` declared in IR/ir\_lower.h. It allocates the module, walks the `NODE_TRANSLATION_UNIT` children, and dispatches each child to the appropriate lowering helper. A shared mutable `ir_lower_ctx_t` (created on the stack and threaded by pointer) carries the current module, function, basic block, control-flow stack, and error counter.

**Declaration Lowering** IR/ir\_lower\_decl.c handles globals (which become `ir_global_t` entries with optional initialisers) and locals (which become `ir_slot_t` reservations on the function's frame). Parameter slots are also created here, with the special property that the first three parameters' values arrive in `a0–a2` (per the ABI), while parameters with index  $\geq 3$  arrive on the stack and are loaded directly by the prologue (see Section 3.1.6.6).

**Expression Lowering** IR/ir\_lower\_expr.c is the largest of the three lowering files. It implements identifier read (resolve symbol  $\rightarrow$  ADDR\_OF  $\rightarrow$  LOAD), binary operations with operand widening, C assignment semantics (lower RHS, compute LHS address, cast, STORE, return RHS), the C-mandated value of `++x` versus `x++`, and the casting machinery (ZEXT, SEXT, TRUNC, BIT-CAST).

The mapping from C operator (`OperatorType_t`) to IR opcode is not one-to-one: relational operators and right-shift split into signed and unsigned variants based on the operands' types, and multiplication / division / modulo do not have a direct opcode at all. Listing 4.18 shows the central dispatch.

```

1  static ir_opcode_t binop_opcode(OperatorType_t op, int is_unsigned)
2  {
3      switch (op) {
4          case OP_PLUS:                return IR_OP_ADD;
5          case OP_MINUS:               return IR_OP_SUB;
6
7          case OP_MULTIPLY:
8          case OP_DIVIDE:
9          case OP_MODULE:               /* modulo */
10             return IR_OP_INVALID;    /* lowered to runtime call */
11
12          case OP_BITWISE_AND:         return IR_OP_AND;
13          case OP_BITWISE_OR:          return IR_OP_OR;
14          case OP_BITWISE_XOR:         return IR_OP_XOR;

```

```

15     case OP_LEFT_SHIFT:         return IR_OP_SHL;
16     case OP_RIGHT_SHIFT:        return is_unsigned ? IR_OP_SHRU : IR_OP_SHRS;
17     case OP_EQUAL:              return IR_OP_EQ;
18     case OP_NOT_EQUAL:          return IR_OP_NEQ;
19     case OP_LESS_THAN:          return is_unsigned ? IR_OP_LTU : IR_OP_LTS;
20     case OP_LESS_THAN_OR_EQUAL: return is_unsigned ? IR_OP_LEU : IR_OP_LES;
21     case OP_GREATER_THAN:       return is_unsigned ? IR_OP GTU : IR_OP_GTS;
22     case OP_GREATER_THAN_OR_EQUAL: return is_unsigned ? IR_OP GEU : IR_OP_GES;
23     default:                    return IR_OP_ADD;
24 }
25 }

```

**Listing 4.18:** `binop_opcode` maps the AST operator and a signedness flag onto a concrete IR opcode. Multiplication, division and modulo deliberately return `IR_OP_INVALID` – the caller must check `needs_runtime_call` first.

Listing 4.19 shows the predicate used to decide on runtime-helper substitution; the matching emitter constructs an `CALL` instruction with the two operands as arguments.

```

1  /* MUL/DIV/MOD have no hardware instruction on this 16-bit RISC.
2   * They are lowered to runtime-library calls so that liveness, precolour
3   * and regalloc see real call boundaries (caller-saved clobber). */
4  static int needs_runtime_call(OperatorType_t op, int is_unsigned,
5                               const char **out_callee)
6  {
7      switch (op) {
8          case OP_MULTIPLY: *out_callee = "__mul"; return 1;
9          case OP_DIVIDE:  *out_callee = is_unsigned ? "__divu" : "__divs"; return 1;
10         case OP_MODULE:   /* modulo */
11                         *out_callee = is_unsigned ? "__modu" : "__mods"; return 1;
12         default:          return 0;
13     }
14 }

```

**Listing 4.19:** Predicate for runtime-helper substitution.

The short-circuit operators `&&` and `||` follow a similar discipline: they are lowered as control-flow diamonds. A fresh stack slot is allocated to hold the result, the LHS is evaluated and tested, the RHS is conditionally evaluated, both arms write 1 or 0 into the slot, and the merge block reloads the slot.

**Statement Lowering** `IR/ir_lower_stmt.c` lowers control-flow constructs by producing the well-known basic-block templates (Dragon Chapter 6.6 [1]):

- **if/else:** condition  $\rightarrow$  `BRANCH` to `then/else`, then  $\rightarrow$  `GOTO merge`, else  $\rightarrow$  `GOTO merge`.
- **while:** header (test)  $\rightarrow$  `BRANCH body / merge`, body  $\rightarrow$  `GOTO header`.
- **do-while:** body  $\rightarrow$  test (`BRANCH body / merge`); body executes at least once.
- **for:** init in entry block, then a *while* on the condition with the step appended at the end of the body. The IR keeps the `NODE_FOR` information so that `continue` jumps to the step rather than to the condition.

- **switch**: a jump table is the default lowering, with a comparison-chain fallback when the case-count exceeds the table budget; the `switch` union member of `ir_instr_t` carries the table.

A fixed-size control-flow stack (`IR_CTRL_STACK_MAX = 64`) holds the `break` and `continue` targets; deeply nested loops beyond 64 levels would require enlarging it.

Listing 4.20 shows the if-lowering schema, condensed from `IR/ir_lower_stmt.c`. The pattern is the standard four-block diamond from Dragon §6.6 [1, §6.6]: a decision block ending in `BRANCH`, a then-arm block, an optional else-arm block, and a merge block. The two arms are sealed back to the merge block with `GOTO`; if either arm already terminated (with `RET` or a nested `BRANCH`), the seal is recorded but does not emit an instruction.

```

1  case NODE_IF:
2  {
3      const TreeNode_t *cond_node = s->p_firstChild;
4      const TreeNode_t *then_node = cond_node ? cond_node->p_sibling : NULL;
5      const TreeNode_t *else_node = then_node ? then_node->p_sibling : NULL;
6
7      if (!cond_node || !then_node) {
8          ir_diag(lctx, "IR001", s->lineNumber, "malformed if statement");
9          break;
10     }
11
12     /* Evaluate condition in current block, normalise it to i1. */
13     ir_value_t cond = ir_lower_expr(lctx, cond_node, &cond_type);
14     ir_value_t pred = coerce_to_pred(lctx, cond);
15
16     /* Pre-allocate the structural blocks. */
17     ir_block_t *then_block = ir_new_block(lctx);
18     ir_block_t *merge_block = ir_new_block(lctx);
19     ir_block_t *else_block = else_node ? ir_new_block(lctx) : NULL;
20
21     /* Seal the current block with a branch on `pred`. */
22     ir_seal_branch(lctx, pred,
23                   then_block->id,
24                   else_block ? else_block->id : merge_block->id);
25
26     /* Lower the then-arm. */
27     lctx->cur_block = then_block;
28     ir_lower_single_stmt(lctx, then_node);
29     ir_seal_goto(lctx, merge_block->id);
30
31     /* Lower the else-arm (if present) and seal it to the merge. */
32     if (else_block) {
33         lctx->cur_block = else_block;
34         ir_lower_single_stmt(lctx, else_node);
35         ir_seal_goto(lctx, merge_block->id);
36     }
37
38     lctx->cur_block = merge_block;
39     break;
40 }
```

**Listing 4.20:** If-statement lowering schema.

The lowering of **while** reuses the same structural moves with a header-block ( $\rightarrow$  branch) plus a body-block ( $\rightarrow$  goto header); **do-while** swaps the order so the body executes once before the test; **for** unwraps to a **while** with the step appended inside the body.

#### 4.1.4.4 IR Dumps and Verification

The IR text format is printed by the compiler at the start of each function. Listing 4.22 shows the dump for a simple linear program, and Listing 4.23 shows the four-block diamond produced by the if-lowering for `iftest.c` (Listing 4.21).

```

1  int main() {
2      int cond = 1;
3      int val = 1;
4
5      if (cond) {
6          int t1 = val + 1;
7          return t1;
8      } else {
9          int t2 = val - 1;
10         return t2;
11     }
12 }
```

**Listing 4.21:** Source for the if-lowering example (`test_files/RegisterAllocation/iftest.c`).

```

1  func @main() → i16 {
2      slot %slot1 : i16 ; b
3      slot %slot0 : i16 ; a
4
5  bb0:
6      %v0 = addr_of %slot0
7      *%v0 = 42
8      %v1 = addr_of %slot1
9      %v2 = addr_of %slot0
10     %v3 = *%v2
11     %v4 = %v3 add 1
12     *%v1 = %v4
13     %v5 = addr_of %slot1
14     %v6 = *%v5
15     ret %v6
16
17  bb1:
18     ret void
19 }
```

**Listing 4.22:** IR text for a simple linear program (excerpt).

```

1  func @main() → i16 {
```

```
2  slot %slot3 : i16 ; t2
3  slot %slot2 : i16 ; t1
4  slot %slot1 : i16 ; val
5  slot %slot0 : i16 ; cond
6
7  bb0:
8      %v0 = addr_of %slot0
9      *%v0 = 1
10     %v1 = addr_of %slot1
11     *%v1 = 1
12     %v2 = addr_of %slot0
13     %v3 = *%v2
14     %v4 = %v3 neq 0
15     if %v4 goto bb1 else bb3
16
17  bb1:
18     %v5 = addr_of %slot2
19     %v6 = addr_of %slot1
20     %v7 = *%v6
21     %v8 = %v7 add 1
22     *%v5 = %v8
23     %v9 = addr_of %slot2
24     %v10 = *%v9
25     ret %v10
26
27  bb2:
28     ret void
29
30  bb3:
31     ; ... else-arm equivalent to bb1, but with `sub 1` ...
32     ret %v16
33 }
```

**Listing 4.23:** IR for `iftest.c`. Note the `NEQ 0` that coerces `cond` to `i1` before the conditional branch, and the four blocks of the diamond.

The `bb2` merge block is emitted even though both arms return, to maintain IR well-formedness; it receives a `ret void` terminator.

## 4.2 Backend

---

### 4.2.1 Register Allocation

#### 4.2.1.1 Liveness Analysis

`CodeGen/RegAlloc/liveness.c` implements the backward dataflow equations using a worklist. Blocks are enqueued in reverse-postorder for the first pass, and a block whose `live_in` changes re-enqueues all of its predecessors. Listing 4.24 shows the body of the worklist loop.

```
1  while (wl_head != wl_tail) {
2      unsigned id = wl[wl_head++ % nb];
3      in_wl[id] = 0;
4  }
```

```

5     block_liveness_t *bl = &liv→blk[id];
6
7     /* new_live_out = union of live_in[succs] */
8     vreg_set_clear(&new_live_out);
9     for (unsigned k = 0; k < cfg[id].n_succs; k++) {
10         unsigned sid = cfg[id].succ_ids[k];
11         vreg_set_union_into(&new_live_out, &liv→blk[sid].live_in);
12     }
13     /* new_live_in = use[B] U (new_live_out - def[B]) */
14     vreg_set_copy(&new_live_in, &new_live_out);
15     vreg_set_minus_into(&new_live_in, &bl→def);
16     vreg_set_union_into(&new_live_in, &bl→use);
17
18     /* If live_in changed: commit and re-enqueue every predecessor. */
19     if (!vreg_set_equal(&new_live_in, &bl→live_in)) {
20         vreg_set_copy(&bl→live_in, &new_live_in);
21         vreg_set_copy(&bl→live_out, &new_live_out);
22         for (unsigned k = 0; k < cfg[id].n_preds; k++) {
23             unsigned pid = cfg[id].pred_ids[k];
24             if (!in_wl[pid]) {
25                 wl[wl_tail++ % nb] = pid;
26                 in_wl[pid] = 1;
27             }
28         }
29     } else {
30         vreg_set_copy(&bl→live_out, &new_live_out);
31     }
32 }

```

Listing 4.24: Worklist body for the iterative dataflow.

Listing 4.25 shows the liveness output for the running example.

```

=== liveness: @main ===
bb0:
  use      = {}
  def      = {%v0, %v1, %v2, %v3, %v4, %v5, %v6}
  instructions:
    [ 0]    %v0 = addr_of %slot0
           live_after = {%v0}
    [ 3]    %v2 = addr_of %slot0
           live_after = {%v1, %v2}
    [ 5]    %v4 = %v3 add 1
           live_after = {%v1, %v4}
    [ 9]    ret %v6
           live_after = {}

```

Listing 4.25: Liveness output for linear.c (excerpt).

#### 4.2.1.2 Interference Graph

CodeGen/RegAlloc/interference.c builds the graph in a single pass over all instructions. The triangular bitset is allocated to hold  $n(n-1)/2$  bits with one `calloc`. The inner loop walks the `live_after` set as a bitset using `__builtin_ctzll` to find each set bit, keeping the constant

factor down. Listing 4.26 reproduces the body of `ifg_build`.

```

1  ifg_t *ifg_build(const ir_function_t *func, const ir_liveness_t *liv)
2  {
3      /* ... allocate triangular bitset + adjacency lists ... */
4
5      /* Walk instructions and add interference edges. */
6      unsigned instr_idx = 0;
7      for (const ir_block_t *b = func->entry; b; b = b->next) {
8          for (const ir_instr_t *ins = b->head; ins; ins = ins->next) {
9              const vreg_set_t *la = &liv->instr[instr_idx].live_after;
10             instr_idx++;
11
12             unsigned d;
13             if (!instr_get_def(ins, &d)) continue;
14
15             /* Move exception: exclude src from live_after so      *
16              * that (dst, src) do not interfere purely due to this *
17              * copy. Record the pair for later coalescing.      */
18             unsigned move_src = (unsigned)-1, move_dst;
19             if (instr_is_move(ins, &move_dst, &move_src))
20                 ifg_record_move(g, move_dst, move_src);
21
22             /* Add edge (d, x) for every vreg x live after this    *
23              * instruction with x ≠ d (and x ≠ move_src).      */
24             unsigned nw = (la->cap + 63) / 64;
25             for (unsigned w = 0; w < nw; w++) {
26                 uint64_t word = la->words[w];
27                 while (word) {
28                     unsigned bit = __builtin_ctzll(word);
29                     unsigned x = w * 64 + bit;
30                     word &= word - 1;
31                     if (x == d || x == move_src) continue;
32                     ifg_add_edge(g, d, x);
33                 }
34             }
35         }
36     }
37     return g;
38 }
```

**Listing 4.26:** Interference-graph construction (CodeGen/RegAlloc/interference.c, condensed).

```

└─ [2] STAGE 2: INTERFERENCE GRAPH
=== interference: @main ===
nodes: 7
edges: 3
edge list:
  %v1 -- %v2
  %v1 -- %v3
  %v1 -- %v4
adjacency:
  %v0 (deg=0) {}
  %v1 (deg=3) {%v2, %v3, %v4}
  %v2 (deg=1) {%v1}
  %v3 (deg=1) {%v1}
  %v4 (deg=1) {%v1}
  %v5 (deg=0) {}
  %v6 (deg=0) {}

```

Figure 4.1: Interference-graph dump for the running example.

#### 4.2.1.3 Pre-colouring

CodeGen/RegAlloc/precolor.c enforces the ABI constraints described in Section 3.1.5.3 before the colouring runs. The pre-colour assignments follow Table 3.6 directly; the call-live set is computed as a by-product of this pass.

```

└─ [0] IR
func @main() -> i16 {
  slot %slot1 : i16 ; b
  slot %slot0 : i16 ; a

bb0:
  %v0 = addr_of %slot0
  *%v0 = 42
  %v1 = addr_of %slot1
  %v2 = addr_of %slot0
  %v3 = *%v2
  %v4 = %v3 add 1
  *%v1 = %v4
  %v5 = addr_of %slot1
  %v6 = *%v5
  ret %v6

```

Figure 4.2: Pre-colouring output: return source forced to r1 (highlighted).

## 4.2.1.4 Chaitin–Briggs Colouring with George Coalescing

CodeGen/RegAlloc/regalloc.c implements the colouring algorithm. Listing 4.27 shows the preference-ordered colour selection in full. `r7` is excluded from the preference list because it is reserved as a codegen scratch register (Section 3.1.6.2), making correctness a construction-time guarantee rather than a runtime check.

```

1  /* r7 (t3) is reserved exclusively for the codegen as a scratch.
2  * Reserving one register out of regalloc's reach makes the codegen
3  * scratch correctness-preserving by construction.                               */
4  static const phys_reg_t pref_normal[] = {
5      PHYS_R4, PHYS_R5, PHYS_R6,
6      PHYS_R8, PHYS_R9, PHYS_R10, PHYS_R11,
7      PHYS_R1, PHYS_R2, PHYS_R3
8  };
9  static const phys_reg_t pref_callee[] = {
10     PHYS_R8, PHYS_R9, PHYS_R10, PHYS_R11
11 };
12
13 int          cl      = precolor_is_call_live(pc, v);
14 const phys_reg_t *pref = cl ? pref_callee : pref_normal;
15 unsigned      nregs = cl
16                ? sizeof(pref_callee) / sizeof(*pref_callee)
17                : sizeof(pref_normal) / sizeof(*pref_normal);
18 phys_reg_t chosen = REGALLOC_SPILL;
19
20 for (unsigned pi = 0; pi < nregs; pi++) {
21     phys_reg_t c = pref[pi];
22     if (!(forbidden & (uint16_t)(1u << c))) {
23         chosen = c;
24         break;
25     }
26 }
27 r->color[v] = chosen;

```

**Listing 4.27:** Preference-ordered colour selection from CodeGen/RegAlloc/regalloc.c.

Listing 4.28 shows the colour assignment produced for the running example. Note how `%v6` is [precolored] to `r1` (because it is the return-value operand) while the rest of the graph receives `r4`, the first preference for normal vregs.

```

regalloc for @main (7 vregs):
  %v0  →  r4    [allocated]
  %v1  →  r5    [allocated]
  %v2  →  r4    [allocated]
  %v3  →  r4    [allocated]
  %v4  →  r4    [allocated]
  %v5  →  r4    [allocated]
  %v6  →  r1    [precolored]
spills: none

```

**Listing 4.28:** Register allocation for `linear.c`.

## 4.2.1.5 Spill Rewriting

CodeGen/RegAlloc/spill.c implements the spill rewrite. Listing 4.29 shows the cost computation; the `deg = 0` branch is the special case for an isolated vreg (very rare in practice, but the formula would otherwise divide by zero).

```

1  unsigned ei = 0;
2  for (unsigned v = 0; v < ra->n_vregs; v++) {
3      if (ra->color[v] != REGALLOC_SPILL) continue;
4
5      unsigned udc = count_use_def(func, v);
6      unsigned deg = ifg_degree(g, v);
7
8      sr->entries[ei].vreg          = v;
9      sr->entries[ei].vtype         = find_vreg_type(func, v);
10     sr->entries[ei].use_def_count = udc;
11     sr->entries[ei].degree        = deg;
12     sr->entries[ei].cost          = (deg > 0)
13                                   ? (double)udc / (double)deg
14                                   : (double)udc * 1000.0; /* isolated */
15
16     char slot_name[64];
17     snprintf(slot_name, sizeof(slot_name), "__spill_v%u", v);
18     sr->entries[ei].slot_id = ir_new_slot(func, slot_name,
19                                         sr->entries[ei].vtype);
20     ei++;
21 }
22 qsort(sr->entries, n, sizeof(*sr->entries), cmp_cost);

```

Listing 4.29: Spill cost, from CodeGen/RegAlloc/spill.c.

Listing 4.30 shows the IR rewrite produced by `test_files/RegisterAllocation/force_spill.c`, which deliberately overflows the four call-saved registers.

```

spill rewrite for @force_spill_real (1 vreg(s)):
  %v0  → %slot0  (__spill_v0)  cost=0.40  [2 ref(s) / deg 5]
  restarting pipeline

func @force_spill_real() → i16 {
  slot %slot0 : i16 ; __spill_v0
bb0:
  %v0 = @get_val(1)
  %v6 = addr_of %slot0
  *%v6 = %v0
  %v1 = @get_val(2)
  ...
  %v7 = addr_of %slot0
  %v8 = *%v7
  @consume_all(%v8, %v1, %v2, %v3, %v4, %v5)
  ret 0
}

```

Listing 4.30: Spill rewrite for `force_spill.c` (the only vreg of cost 0.40 was spilled).

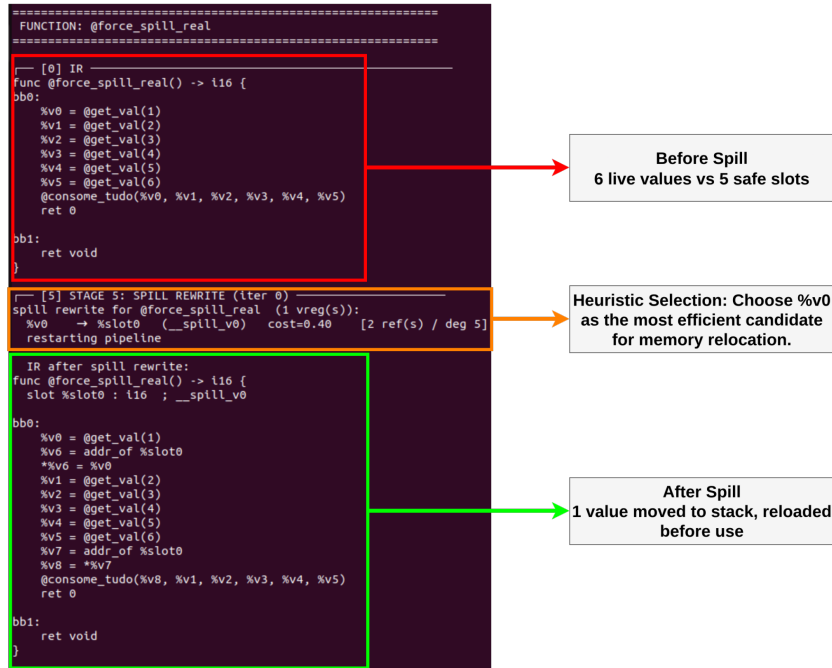


Figure 4.3: Worked spill example for a vreg whose live range crosses many calls.

## 4.2.2 Code Generation

### 4.2.2.1 Prologue and Epilogue Emitter

The prologue emitter is shown in Listing 4.31. The `callee_used_mask` bitset is populated by the `regalloc` and holds one bit per callee-saved register the colouring actually used, so leaf functions emit zero `PUSH` instructions for the callee-saved set. The `ultra` flag is set when the function is a leaf with no slots, no callee-saved registers used, and at most three parameters, allowing the entire frame to be elided.

```

1  static void emit_prologue(FILE *out,
2                          const ir_function_t *func,
3                          const regalloc_t *ra)
4  {
5      unsigned n_slots = count_slots(func);
6      uint16_t callee = callee_used_mask(ra);
7      int leaf = function_is_leaf(func);
8      int ultra = leaf && n_slots == 0 && callee == 0
9                  && func->param_count <= PHYS_ARG_REGS;
10     /* PHYS_ARG_REGS is currently 3 on gr0040. */
11
12     /* Ultra-leaf: no frame, body uses no fp-relative addressing. */
13     if (ultra) {
14         fprintf(out, "    ; --- Prologue (ultra-leaf: no frame) ---\n");
15         return;
16     }
17
18     fprintf(out, "    ; --- Prologue%s ---\n",
19             leaf ? " (leaf: skip lr save)" : "");
20     fprintf(out, "    PUSH(fp)\n");
21     fprintf(out, "    MOV(fp, sp)\n");

```

```

22     if (!leaf)
23         fprintf(out, "    PUSH(lr)\n");
24
25     /* Save any callee-saved registers the function actually uses. */
26     for (int i = 0; i <= 3; i++)
27         if (callee & (1u << i))
28             fprintf(out, "    PUSH(r%d)\n", 8 + i);
29
30     /* Reserve stack space for all local slots. */
31     if (n_slots > 0) {
32         fprintf(out, "    ; Allocate %u local slot(s)\n", n_slots);
33         emit_sp_adj(out, -(int)n_slots);
34     }
35
36     /* Stack-passed parameters (index ≥ 3) are loaded from the
37      * caller's frame into their assigned local slots via t3. */
38     /* ... stack-passed parameter loading, see backend section ... */
39 }

```

Listing 4.31: Prologue emitter from CodeGen/codegen.c.

Stack-passed parameters are loaded in the prologue tail via t3:

```

1  LW  t3, fp, #+offset_in_caller_frame
2  SW  t3, fp, #-offset_in_callee_frame

```

#### 4.2.2.2 Two-Address Arithmetic Emitters

Listing 4.32 shows both the commutative emitter `emit_binop_rr` and its non-commutative counterpart `emit_binop_rr_nc`. The non-commutative case for `SUB` uses a `NEG+ADD` rewrite to avoid the t3 round-trip where possible; the shift instructions always require it.

```

1  /* Commutative: dst = a op b  (ADD, AND, XOR) */
2  static void emit_binop_rr(FILE *out, const char *mnem,
3                             const char *dst,
4                             const char *a, const char *b)
5  {
6      if (strcmp(dst, a) == 0) {          /* e.g. r1 = r1 + r2 */
7          fprintf(out, "    %s %s, %s\n", mnem, dst, b);
8      } else if (strcmp(dst, b) == 0) {   /* e.g. r1 = r2 + r1 */
9          /* commutative: swap operands */
10         fprintf(out, "    %s %s, %s\n", mnem, dst, a);
11     } else {
12         /* e.g. r1 = r2 + r3 */
13         fprintf(out, "    MOV(%s, %s)\n", dst, a);
14         fprintf(out, "    %s %s, %s\n", mnem, dst, b);
15     }
16 }
17
18 /* Non-commutative: dst = a op b  (SUB, SRL, SRA) -- order matters.
19  * For SUB we rewrite via NEG when dst == b; for SRL / SRA we stash b
20  * in t3 (regalloc-reserved scratch) before overwriting dst with a. */
21 static void emit_binop_rr_nc(FILE *out, const char *mnem,

```

```

21         const char *dst,
22         const char *a, const char *b)
23 {
24     if (strcmp(dst, a) == 0) {
25         fprintf(out, "    %s %s, %s\n", mnem, dst, b);
26         return;
27     }
28     if (strcmp(dst, b) == 0) {
29         if (strcmp(mnem, "SUB") == 0) {
30             fprintf(out, "    NEG(%s)\n", dst);
31             fprintf(out, "    ADD %s, %s\n", dst, a);
32             return;
33         }
34         /* SRL / SRA: stash b in the regalloc-reserved scratch t3. */
35         fprintf(out, "    MOV(t3, %s)\n", b);
36         fprintf(out, "    MOV(%s, %s)\n", dst, a);
37         fprintf(out, "    %s %s, t3\n", mnem, dst);
38         return;
39     }
40     fprintf(out, "    MOV(%s, %s)\n", dst, a);
41     fprintf(out, "    %s %s, %s\n", mnem, dst, b);
42 }

```

**Listing 4.32:** Three-to-two-address translation rules: the commutative emitter `emit_binop_rr` and its non-commutative counterpart `emit_binop_rr_nc`, from `CodeGen/codegen.c`.

#### 4.2.2.3 Comparison and Branch Synthesis

Listing 4.33 shows `cmp_to_branch`, which maps each IR comparison opcode to a branch mnemonic and an operand-swap flag. The missing  $>$  /  $\geq$  family is handled by setting `swap_out = 1`, which causes the call site to exchange the two operands before emitting the branch; `NEQ` reuses `BEQ` with the true and false targets swapped.

```

1  static void cmp_to_branch(ir_opcode_t op,
2                          const char **bmnem_out,
3                          int *swap_out)
4  {
5      *swap_out = 0;
6      switch (op) {
7          case IR_OP_EQ:  *bmnem_out = "BEQ";  break;
8          case IR_OP_NEQ: *bmnem_out = "BEQ";  *swap_out = 1; break;
9          case IR_OP_LTS: *bmnem_out = "BLT";  break;
10         case IR_OP_LES: *bmnem_out = "BLE";  break;
11         case IR_OP_GTS: *bmnem_out = "BLT";  *swap_out = 1; break;
12         case IR_OP_GES: *bmnem_out = "BLE";  *swap_out = 1; break;
13         case IR_OP_LTU: *bmnem_out = "BLEU"; break;
14         case IR_OP_LEU: *bmnem_out = "BLETU"; break;
15         case IR_OP GTU: *bmnem_out = "BLEU"; *swap_out = 1; break;
16         case IR_OP GEU: *bmnem_out = "BLETU"; *swap_out = 1; break;
17         default:        *bmnem_out = "BEQ";  break;
18     }
19 }

```

**Listing 4.33:** Comparison and branch synthesis: `cmp_to_branch` maps an IR comparison opcode to a branch mnemonic plus an operand-swap flag, from `CodeGen/codegen.c`.

#### 4.2.2.4 CALL Setup: Parallel-Copy Implementation

Listing 4.34 reproduces the comment block that heads the parallel-copy implementation in `CodeGen/codegen.c`. It is the authoritative description of the three-phase algorithm and is quoted in full here because it is the clearest statement of the invariants the code maintains.

```

1  /* 2. register arguments (a0..a2)
2  *
3  * Sequential MOVs are unsafe when arg sources form a cycle
4  * across the destination registers -- e.g. `g(b, a)` with `a` in a0
5  * and `b` in a1 needs to swap the two registers, not just
6  * "MOV a0, a1; MOV a1, a0" (which leaves both holding a1's value).
7  *
8  * We solve it as a parallel-copy:
9  * 1. Emit any VREG copy whose dst is not the SRC of another
10 *     pending VREG copy (Kahn-style topological order).
11 * 2. Repeat until either nothing pending or only cycles remain.
12 * 3. Break each remaining cycle by spilling one source into t3
13 *     (regalloc-reserved scratch), redirecting the cycle's reads
14 *     to t3, and re-running phase 1.
15 * 4. Imm/global args run last so they don't clobber any vreg
16 *     source register before it is read.
17 */
```

**Listing 4.34:** Parallel-copy algorithm, described inline in `CodeGen/codegen.c`.

### 4.2.3 Runtime Library

The runtime library is implemented entirely in gr0040 assembly and resides in `runtime/`. Because each helper is a standard ABI-conforming function, the implementations require no special compiler support: they receive their two operands in `a0` and `a1`, return their result in `a0`, and clobber only the caller-saved registers `a0–a2` and `t0–t2`. The reserved scratch `t3` is left untouched.

`__mul` implements shift-and-add with a 16-iteration loop. On each iteration, bit 0 of the multiplier (held in `a1`) is tested; if set, the multiplicand (`a0`) is added into the accumulator. Both the multiplicand and the partial quotient are then shifted by one, and the loop counter is decremented. The result is the low 16 bits of the product, discarding any overflow.

`__divu` implements restoring long division. The dividend is split across two registers (high and low halves). On each of the 16 iterations, the high half is shifted left by one, the divisor is subtracted tentatively, and the quotient bit is set or the subtraction is undone (“restored”) depending on the sign of the result. After 16 iterations the quotient occupies the low register and the remainder the high register.

`__divs` and `__mods` sign-normalise both operands to positive values, record the expected sign of the result, delegate to `__divu` or `__modu` respectively, and then conditionally negate the result

to match C99 truncation-toward-zero semantics. The detailed pseudocode and cycle counts for each helper are given in `docs/runtime/Runtime Library Specification.md`.

## 4.3 Assembler

---

This section describes the implementation of the ShivAT assembler. It is structured as a classical two-pass assembler with a preprocessing layer, a lexer and parser for instruction construction, a symbol table for label management, and a final code generation stage. Each component contributes to transforming human-readable assembly into machine-level hexadecimal object code for the gr0040 architecture.

### 4.3.1 Preprocessor – `abi.m4`

The assembler includes a lightweight preprocessing step based on an `m4`-style macro system. This layer defines register aliases and common calling-convention utilities, allowing assembly code to be written in a more readable and abstract form.

```
1  ; Register Aliases (ABI)
2  zero .equ r0
3  a0   .equ r1
4  v0   .equ r1
5  sp   .equ r13
6  lr   .equ r14
7  gp   .equ r15
8
9  ; Stack PUSH / POP
10 .macro PUSH reg
11     ADDI sp, sp, #-1
12     SW   \reg, sp, #0
13 .endm
14
15 .macro POP reg
16     LW   \reg, sp, #0
17     ADDI sp, sp, #1
18 .endm
```

**Listing 4.35:** Register aliases and stack macros in `abi.m4`.

Macro expansion occurs before parsing, meaning that high-level constructs such as `PUSH` and `POP` are expanded into primitive instructions before reaching the lexer and parser.

```
; Source:     POP(a0)
; Expands to:
    ADDI sp, sp, #1
    LW   r1, sp, #0
```

**Listing 4.36:** Macro expansion example.

### 4.3.2 Lexer – lex.1

The lexer is responsible for converting raw assembly text into a stream of tokens. It recognises identifiers, registers, immediates, directives, and branch conditions. These tokens form the input to the parser.

#### 4.3.2.1 Pattern Definitions

The following definitions describe the lexical structure of numbers, registers, and identifiers.

```
1 digit      [0-9]
2 hex        [a-fA-F]
3 dec_number {digit}+
4 hex_number_x 0x({digit}|{hex})+
5
6 /* Registers R0-R31 */
7 register     [Rr]({digit}|[12]{digit}|3[01])
8
9 identifier   [a-zA-Z_][a-zA-Z0-9_]*
10 newline     \n
11 whitespace  [ \t]+
```

**Listing 4.37:** Flex pattern definitions used by the lexer.

#### 4.3.2.2 Identifiers, Registers, and Special Tokens

Identifiers are immediately inserted into the symbol table, enabling forward references. Registers and special tokens such as the location counter symbol are also resolved during lexical analysis.

```
1 {identifier} {
2     yylval.num = add_symbol(yytext);
3     return TOKEN_IDENTIFIER;
4 }
5
6 {register} {
7     yylval.num = atoi(yytext + 1);
8     return TOKEN_REG;
9 }
10
11 "$" {
12     yylval.num = get_location_counter();
13     return TOKEN_DOLLAR;
14 }
```

**Listing 4.38:** Identifier and symbol handling in the lexer.

#### 4.3.2.3 Branch Condition Tokens

Branch instructions are encoded using predefined numeric condition codes assigned during lexing.

```
1  "BEQ" { yylval.num = BEQ_COND; return TOKEN_BEQ; }
2  "BLT" { yylval.num = BLT_COND; return TOKEN_BLT; }
3  "BLE" { yylval.num = BLE_COND; return TOKEN_BLE; }
4  "BLETU" { yylval.num = BLETU_COND; return TOKEN_BLETU; }
5  "BLEU" { yylval.num = BLEU_COND; return TOKEN_BLEU; }
```

**Listing 4.39:** Branch condition tokens.

### 4.3.3 Parser – `parser.y`

The parser constructs structured assembly statements from the token stream. It encodes instruction formats and separates immediate values from symbolic operands for later resolution in the second pass.

#### 4.3.3.1 Token Declarations

The parser uses a union-based semantic value system to associate numeric data with tokens.

```
1  %union {
2      int num;
3  }
4
5  %token <num> TOKEN_NUMBER TOKEN_REG TOKEN_DOLLAR TOKEN_IDENTIFIER
6  %token <num> TOKEN_BR TOKEN_BEQ TOKEN_BC TOKEN_BV TOKEN_BLT TOKEN_BLE
7          TOKEN_BLETU TOKEN_BLEU
8
9  %token TOKEN_ADD TOKEN_SUB TOKEN_AND TOKEN_XOR TOKEN_ADC TOKEN_SBC
10         TOKEN_CMP TOKEN_SRL TOKEN_SRA
11
12 %token TOKEN_BYTE TOKEN_WORD TOKEN_ORG TOKEN_EQU
```

**Listing 4.40:** Bison token and union declarations.

#### 4.3.3.2 RI-Format Instruction Construction

This rule demonstrates how immediate and symbolic operands are handled. Immediate values are stored directly, while labels are marked for resolution in the second pass.

```
1  andi_stmt :
2      TOKEN_ANDI TOKEN_REG TOKEN_COMMA immediate_val
3      {
4          add_statement_ri(RI_OPCODE, ANDI_FN, $2, $4, IMMEDIATE);
5      }
6  | TOKEN_ANDI TOKEN_REG TOKEN_COMMA TOKEN_IDENTIFIER
7      {
8          add_statement_ri(RI_OPCODE, ANDI_FN, $2, $4, LABEL);
9      }
10 ;
```

**Listing 4.41:** RI-format instruction handling with forward references.

### 4.3.4 Symbol Table – `symbol_table.c`

The symbol table maps labels to memory addresses using a hash table with chaining for collision handling. It supports efficient insertion and lookup during both parsing and code generation.

#### 4.3.4.1 Hash Function

A djb2-based hash function is used to distribute symbols across the table.

```
1 static uint16_t hash(char *symbol_name)
2 {
3     unsigned long hash_value = 5381;
4     int c;
5
6     while ((c = *symbol_name++))
7         hash_value = ((hash_value << 5) + hash_value) + c;
8
9     return (uint16_t)(hash_value % HASH_TABLE_SIZE);
10 }
```

**Listing 4.42:** Hash function used for symbol lookup.

#### 4.3.4.2 Symbol Insertion and Collision Handling

Symbols are stored in a dynamically growing array, while collisions are resolved using linked-list chaining in the hash table.

```
1 static uint16_t insert_symbol(char *symbol_name, uint16_t hash_value)
2 {
3     uint16_t index = next_free_index++;
4
5     if (next_free_index ≥ sym_tab_size) {
6         sym_tab_size += INC_SYMBOL_TABLE;
7         sym_table = (struct symbol_s *)
8             realloc(sym_table, sizeof(struct symbol_s) * sym_tab_size);
9     }
10
11     strcpy(sym_table[index].name, symbol_name);
12
13     sym_table[index].link = hash_table[hash_value];
14     hash_table[hash_value] = index;
15     sym_table[index].value = UNINITIALIZED_VALUE;
16
17     return index;
18 }
```

**Listing 4.43:** Insertion with separate chaining for collision handling.

### 4.3.5 Step 2 – Code Generator – `code_generator.c`

The second pass of the assembler transforms the intermediate representation into final machine code. It uses the opcode table, symbol table, and statement list to produce 16-bit hexadecimal instructions.

During this stage, each statement is encoded according to its format, and symbolic operands are resolved using the symbol table.

#### 4.3.5.1 Instruction Encoding with Symbol Resolution

This example shows how RRI-format instructions are encoded. If an operand is a label, the assembler retrieves its address from the symbol table and computes the final value, including relative offsets where required (e.g., for jumps).

```

1  case FMT_RRI: {
2      int16_t imm4 = stmt.imm;
3
4      if (stmt.misc == LABEL || stmt.misc == LINK) {
5          int16_t label_addr = get_symbol_value(stmt.imm);
6
7          if (stmt.opcode == JAL_OPCODE) {
8              int16_t disp = (int32_t)label_addr
9                           - (int32_t)(stmt_lc + LC_INSTRUCTION);
10             imm4 = (int16_t)(disp & 0xF);
11         } else {
12             imm4 = label_addr & 0xF;
13         }
14     }
15
16     code = ((uint16_t)(stmt.opcode & 0xF) << 12)
17           | ((uint16_t)(stmt.rd      & 0xF) << 8)
18           | ((uint16_t)(stmt.rs      & 0xF) << 4)
19           | (uint16_t)(imm4          & 0xF);
20     break;
21 }
```

**Listing 4.44:** Instruction encoding with label resolution in the second pass.

### 4.3.6 Concluding Details

After the Machine Code is created, it is split into its first and last 8 bits. This step is convenient because Vivado’s memory insertion expects the code in that format.

# Chapter 5

## Implementation Validation

### 5.1 Test Suite Inventory

The test suite lives under `test_files/`. Table 5.1 breaks it down by directory, naming the kind of contract each sub-tree is meant to enforce. The four directories together hold **130 .c files**; they are exercised by the harnesses in Sections 5.2 and 5.3.

| Directory                                         | Files      | Coverage                                        |
|---------------------------------------------------|------------|-------------------------------------------------|
| <code>test_files/test_lexer/</code>               | 2          | maximal-munch, comment states                   |
| <code>test_files/semantic_checks/programs/</code> | 84         | every implemented SEM rule                      |
| <code>test_files/semantic_checks/</code>          | 6          | API unit tests (arena, diag, symbol, type, ...) |
| <code>test_files/IR_checks/</code>                | 14         | IR text snapshots                               |
| <code>test_files/RegisterAllocation/</code>       | 24         | full pipeline + ABI                             |
| <b>Total</b>                                      | <b>130</b> |                                                 |

**Table 5.1:** Test file inventory. Backend tests = `RegisterAllocation/` (24) + `IR_checks/` (14) = 38.

### 5.2 Snapshot Mechanism

The `scripts/test_compile_all.sh` script compiles every .c file within the IR and RegisterAllocation directories, captures the generated assembly, and diffs it against the corresponding `*_expected.asm` file, when present. The script exits with code 0 only if every snapshot diff evaluates to empty. Tests without an explicit snapshot are still compiled, allowing the script to assert the absence of “(null)” operands and redundant self-MOVs. Additionally, the harness features an `expect_fail` mode to verify that explicitly unsupported features are correctly rejected by the compiler.

The core body of this test harness is concise enough to be read in full (Listing 5.1). The two `grep` probes act as codegen-smell detectors: a (null) operand in the assembly indicates that the compiler attempted to print an unnamed value, whereas a redundant self-MOV between an ABI alias (`a0..a2`) and its underlying physical register (`r1..r3`) signals a regression in either the parallel-copy algorithm or the ABI-alias emission logic.

```
for f in test_files/RegisterAllocation/*.c test_files/IR_checks/*.c; do
    total=$((total + 1))

    # EXPECT_FAIL MODE (for unsupported features)
    if [[ "$f" == *"test_unsupported_*.c" ]]; then
        if ./compiler "$f" >/tmp/.out_$$ 2>&1; then
            echo "[FAIL] $f -- expected to fail, but succeeded"
            fail=1
        else
            if grep -q "IR001" /tmp/.out_$$; then
                echo "[PASS] $f -- correctly rejected with IR001"
                ok=$((ok + 1))
            else
                fail=1
            fi
        fi
    fi
done
```

```

        echo "[FAIL] $f -- failed without IRO01 diagnostic"
        cat /tmp/.out_$$
        fail=1
    fi
fi
rm -f /tmp/.out_$$
continue
fi

# NORMAL MODE
if ! ./compiler "$f" >/dev/null 2>&1; then
    echo "[FAIL] $f -- compiler returned non-zero"
    fail=1
    continue
fi

nulls=$(grep -c '(null)' output.asm 2>/dev/null || true)
redmov=$(grep -cE 'MOV\([a0-9], r[1-9]\)|MOV\([r1-9], a[0-9]\)' \
    output.asm 2>/dev/null || true)

file_failed=0
if [[ "${nulls:-0}" -ne 0 ]]; then
    echo "[FAIL] $f -- null operands x$nulls"; file_failed=1
fi
if [[ "${redmov:-0}" -ne 0 ]]; then
    echo "[FAIL] $f -- redundant a/r MOVs x$redmov"; file_failed=1
fi

expected="${f%.c}_expected.asm"
if [[ -f "$expected" ]]; then
    if ! diff -u "$expected" output.asm >/tmp/.diff_$$ 2>&1; then
        echo "[FAIL] $f -- snapshot drift vs $expected:"
        sed 's/^/ /' /tmp/.diff_$$ | head -20
        file_failed=1
    fi
    rm -f /tmp/.diff_$$
fi
[[ $file_failed -ne 0 ]] && fail=1 || ok=$((ok + 1))
done

```

Listing 5.1: Body of scripts/test\_compile\_all.sh.

When the test corpus is clean, the harness produces the two-line summary in Listing 5.2.

```

[SUMMARY] 41/41 tests passed
[OK] all regression tests compile cleanly

```

Listing 5.2: Live output of scripts/test\_compile\_all.sh with no failing tests.

## 5.3 CI Scripts

scripts/ci/run\_semantic\_checks.sh drives the semantic-rule suite by parsing each test program's filename to derive the expected diagnostic code, running ./compiler against it, and comparing the emitted code's exit status and any SEM### matches.

scripts/ci/run\_semantic\_pass\_checks.sh is a finer-grained version that splits Pass 1 and Pass 2 contexts, so that a test that fails only in Pass 2 is not masked by a Pass 1 failure.

## 5.4 End-to-End Walk-Through (Linear)

We close with a complete trace through the pipeline for the file `test_files/RegisterAllocation/linear.c`:

```

1  int main() {
2      int a = 42;
3      int b = a + 1;
4      return b;
5  }
```

**Listing 5.3:** The C source.

The IR text for this program is shown in Listing 4.22; the liveness output is in Listing 4.25; the register allocation result is in Listing 4.28; the final assembly emitted by the codegen is shown in Listing 5.4.

```

1  main:
2      ; Frame slots (fp-based):
3      ;   %slot1 (b) @ fp-3
4      ;   %slot0 (a) @ fp-2
5
6      ; --- Prologue ---
7      PUSH(fp)
8      MOV(fp, sp)
9      PUSH(lr)
10     ; Allocate 2 local slot(s)
11     ADDI sp, sp, #-2
12
13  bb0:
14     ADDI t0, fp, #-2
15     LI(t3, 0x002A)
16     SW t3, t0, #0
17     ADDI t1, fp, #-3
18     ADDI t0, fp, #-2
19     LW t0, t0, #0
20     ADDI t0, t0, #1
21     SW t0, t1, #0
22     ADDI t0, fp, #-3
23     LW a0, t0, #0
24     ; --- Epilogue ---
25     ADDI sp, sp, #2
26     POP(lr)
27     POP(fp)
28     RET
29
30  bb1:
31     ; --- Epilogue ---
32     ADDI sp, sp, #2
33     POP(lr)
34     POP(fp)
35     RET
```

**Listing 5.4:** Final assembly emitted for `linear.c`.

## 5.5 End-to-End Walk-Through (Control Flow)

The `linear.c` program exercises only straight-line code. To show the pipeline acting on a control-flow construct we trace the `iftest.c` program first introduced in Section 3.1.4 (Listing 4.20 shows the lowering schema; its source is already given in Listing 4.21 and its IR in Listing 4.23). Listing 5.5 shows the colour assignment produced by the `regalloc`, and Listing 5.6 shows the final assembly.

```
regalloc for @main (17 vregs):
  %v0   →  r4   [allocated]
  %v1   →  r4   [allocated]
  %v2   →  r4   [allocated]
  %v3   →  r4   [allocated]
  %v4   →  r4   [allocated]
  %v5   →  r5   [allocated]
  %v6   →  r4   [allocated]
  %v7   →  r4   [allocated]
  %v8   →  r4   [allocated]
  %v9   →  r4   [allocated]
  %v10  →  r1   [precolored]
  %v11  →  r5   [allocated]
  %v12  →  r4   [allocated]
  %v13  →  r4   [allocated]
  %v14  →  r4   [allocated]
  %v15  →  r4   [allocated]
  %v16  →  r1   [precolored]
  spills: none
```

**Listing 5.5:** Register allocation for `iftest.c`. With 17 vregs and a low maximum simultaneous live set, no spilling is needed; `%v10` and `%v16` are pre-coloured to `r1` because they feed the two `RET` terminators.

(Note: the `regalloc` reuses `r4` across non-overlapping live ranges; this is not coalescing but standard kill-then-reuse.)

```
1  main:
2      ; Frame slots ...
3      PUSH(fp)
4      MOV(fp, sp)
5      PUSH(lr)
6      ADDI sp, sp, #-4
7
8  bb0:
9      ADDI t0, fp, #-2
10     LI(t3, 0x0001)
11     SW t3, t0, #0
12     ADDI t0, fp, #-3
13     LI(t3, 0x0001)
14     SW t3, t0, #0
15     ADDI t0, fp, #-2
16     LW t0, t0, #0
17     ; t0 = (cond ≠ 0)
18     BNE t0, r0, bb1
19     J bb3
20
21  bb1:
```

```

22      ; ... compute val + 1, store, return ...
23      ADDI sp, sp, #4
24      POP(lr)
25      POP(fp)
26      RET
27
28  bb3:
29      ; ... compute val - 1, store, return ...
30      ADDI sp, sp, #4
31      POP(lr)
32      POP(fp)
33      RET

```

**Listing 5.6:** Final assembly emitted for `iftest.c` (excerpt: `bb0` is the test, `bb1` is the then-arm, `bb3` is the else-arm). Note the `BNE t0, r0` pattern that branches when the condition is non-zero – the codegen synthesises the `neq 0-then-branch` pair from the IR’s predicate.

## 5.6 End-to-End Walk-Through (Forced Spill)

`force_spill.c` is the test that exercises the spill loop. It chains six `get_val(i)` calls whose results must all be alive across each other when `consume_all` is finally called – six call-live values, but only four callee-saved registers (`r8–r11`) to hold them. The colouring of stage 4 cannot satisfy this on the first try, so one vreg is marked `REGALLOC_SPILL` and the pipeline restarts on the rewritten IR (Listing 5.7).

```

spill rewrite for @force_spill_real (1 vreg(s)):
  %v0 → %slot0  (__spill_v0)  cost=0.40  [2 ref(s) / deg 5]
  restarting pipeline

func @force_spill_real() → i16 {
  slot %slot0 : i16 ; __spill_v0
bb0:
  %v0 = @get_val(1)
  %v6 = addr_of %slot0
  *%v6 = %v0          ; <-- spill store
  %v1 = @get_val(2)
  %v2 = @get_val(3)
  %v3 = @get_val(4)
  %v4 = @get_val(5)
  %v5 = @get_val(6)
  %v7 = addr_of %slot0
  %v8 = *%v7          ; <-- spill reload
  @consume_all(%v8, %v1, %v2, %v3, %v4, %v5)
  ret 0
}

```

**Listing 5.7:** Spill rewrite for `force_spill.c`. `%v0` (cost 0.40, 2 references on a degree of 5) is the cheapest spill candidate; the rewrite stores it to `%slot0` after its definition and reloads it (as `%v8`) just before its use, leaving four call-live vregs that fit comfortably into `r8–r11`.

After the rewrite the regalloc runs a second time, this time finding a valid colouring with the four surviving call-live vregs in `r8–r11` and `%v0`’s reload mapped to a normal caller-saved temp. The captured output shows zero spills on the second iteration, confirming convergence.

## 5.7 Negative Tests for Unsupported Features

To ensure the compiler's robustness against C features not yet supported by the Backend (IR Generation phase), a suite of negative tests was introduced. The goal of this verification is to confirm that the compiler explicitly and safely rejects these constructs by emitting the IR001 diagnostic and aborting compilation, rather than silently generating invalid Assembly code. For this, three main test cases were designed, which are detailed below:

The compiler must intercept and reject the declaration and usage of function pointers during the IR lowering phase.

```
int add(int a, int b) { return a + b; }

int main(void) {
    int (*func_ptr)(int, int) = &add;
    int result = func_ptr(2, 3);
    return 0;
}
```

**Listing 5.8:** Rejection test for function pointers (`test_unsupported_funcptr.c`).

Direct memory copying between aggregates lacks a translation strategy to 3AC instructions, meaning the assignment (`p2 = p1;`) is intentionally blocked.

```
struct Point { int x; int y; };

int main(void) {
    struct Point p1, p2;
    p1.x = 1; p1.y = 2;
    p2 = p1;
    return 0;
}
```

**Listing 5.9:** Rejection test for struct copying (`test_unsupported_struct_copy.c`).

A bypass was implemented in the Semantic Analysis pass to defer the validation of these lists to the Backend, which in turn rejects the AST node associated with the curly braces `{}`.

```
int main(void) {
    int arr[] = {1, 2, 3};
    return 0;
}
```

**Listing 5.10:** Rejection test for array initializers (`test_unsupported_aggregate_init.c`).

To demonstrate the compiler's correct behavior when encountering these unsupported features, the following terminal outputs illustrate the graceful abortion of the compilation process via the [IR001] diagnostic:

```
$ ./compiler test_files/IR_checks/test_unsupported_funcptr.c
[IR001] line 5: unhandled expression node type 44

$ ./compiler test_files/IR_checks/test_unsupported_struct_copy.c
[IR001] line 6: statement blocked for codegen

$ ./compiler test_files/IR_checks/test_unsupported_aggregate_init.c
```

```
[IR001] line 2: unhandled expression node type 44
```

**Listing 5.11:** Terminal output demonstrating the explicit rejection of unsupported C features.

## 5.8 Compiled Feature Demo

This section compiles a set of programs end-to-end and examines the output assembly feature by feature.

```
1 int use_point(int px, int py) {
2     struct Point { int x; int y; } p;
3     p.x = px;          /* write field x (GEP offset 0) */
4     p.y = py;          /* write field y (GEP offset 1) */
5     int a = p.x;       /* read field x */
6     int b = p.y;       /* read field y */
7     return a + b;
8 }
```

**Listing 5.12:** Struct field read and write (use\_point).

The struct `Point` is two words wide. The IR represents every field access as a GEP (get-element-pointer) followed by a store or load. Field `x` sits at word offset 0 from the struct base, so the GEP reduces to `ADD t1, r0` (adding zero). Field `y` sits at word offset 1, producing `ADDI t3, r0, #1` then `ADD t1, t3`. Because `use_point` makes no calls, the prologue skips saving `lr`.

```
1 use_point:
2     ; Frame slots (fp-based):
3     ; %slot5 (b) @ fp-6    %slot4 (a) @ fp-5
4     ; %slot2 (p) @ fp-3    %slot1 (py) @ fp-2    %slot0 (px) @ fp-1
5
6     ; --- Prologue (leaf: skip lr save) ---
7     PUSH(fp)
8     MOV(fp, sp)
9     ; Allocate 6 local slot(s)
10    ADDI sp, sp, #-6
11
12 bb0:
13    ADDI t0, fp, #-1
14    SW a0, t0, #0          ; spill px
15    ADDI t0, fp, #-2
16    SW a1, t0, #0          ; spill py
17    ADDI t0, fp, #-3          ; base address of p
18    MOV(t1, t0)
19    ADD t1, r0              ; GEP p.x (offset 0 * 1)
20    ADDI t0, fp, #-1
21    LW t0, t0, #0
22    SW t0, t1, #0          ; p.x = px
23    ADDI t0, fp, #-3
24    ADDI t3, r0, #1
25    MOV(t1, t0)
26    ADD t1, t3              ; GEP p.y (offset 1 * 1)
27    ADDI t0, fp, #-2
```

```

28     LW t0, t0, #0
29     SW t0, t1, #0           ; p.y = py
30     ADDI t1, fp, #-5
31     ADDI t0, fp, #-3
32     ADD t0, r0
33     LW t0, t0, #0
34     SW t0, t1, #0           ; a = p.x
35     ADDI t1, fp, #-6
36     ADDI t0, fp, #-3
37     ADDI t3, r0, #1
38     ADD t0, t3
39     LW t0, t0, #0
40     SW t0, t1, #0           ; b = p.y
41     ADDI t0, fp, #-5
42     LW t1, t0, #0
43     ADDI t0, fp, #-6
44     LW t0, t0, #0
45     MOV(a0, t1)
46     ADD a0, t0               ; return a + b
47     ; --- Epilogue (leaf: skip lr restore) ---
48     ADDI sp, sp, #6
49     POP(fp)
50     RET

```

**Listing 5.13:** Assembly for `use_point`: leaf prologue and GEP-based field addressing.

## 5.9 Two-Dimensional Array

```

1  int sum_corners(int a, int b, int c, int d) {
2      int mat[2][2];
3      mat[0][0] = a;   mat[0][1] = b;
4      mat[1][0] = c;   mat[1][1] = d;
5      return mat[0][0] + mat[1][1]; /* top-left + bottom-right */
6  }

```

**Listing 5.14:** 2-D array allocation and indexed store/load (`sum_corners`).

A `[2][2]` integer array occupies four consecutive word slots on the frame. The row stride is 2 (the inner dimension), so the flat index for `mat[r][c]` is  $r \times 2 + c$ . The compiler expands this with an SLL (shift-left-logical by 1, i.e. multiply by 2) followed by an ADD. Argument `d` is the fourth parameter and therefore stack-passed; the prologue loads it from `fp+1` immediately after the frame is set up. The epilogue splits the frame deallocation into two ADDI instructions because the frame size (8 slots) exceeds the range of a single signed immediate on this ISA.

```

1  sum_corners:
2      ; Frame slots (fp-based):
3      ;   %slot4 (mat) @ fp-5   %slot3 (d) @ fp-4
4      ;   %slot2 (c)   @ fp-3   %slot1 (b) @ fp-2   %slot0 (a) @ fp-1
5
6      ; --- Prologue (leaf: skip lr save) ---
7      PUSH(fp)

```

```

8      MOV(fp, sp)
9      ; Allocate 8 local slot(s)
10     ADDI sp, sp, #-8
11     LW t3, fp, #1      ; load stack param d
12     SW t3, fp, #-4     ; store to slot d
13
14     bb0:
15     ADDI t0, fp, #-1
16     SW a0, t0, #0      ; spill a
17     ADDI t0, fp, #-2
18     SW a1, t0, #0      ; spill b
19     ADDI t0, fp, #-3
20     SW a2, t0, #0      ; spill c
21
22     ; mat[0][0] = a --> flat index 0*2+0 = 0
23     ADDI t0, fp, #-5
24     MOV(t0, r0)
25     SLL(t0)            ; row 0 * 2
26     ADD t0, t0
27     MOV(t1, t0)
28     ADD t1, r0         ; + col 0
29     ADDI t0, fp, #-1
30     LW t0, t0, #0
31     SW t0, t1, #0
32
33     ; mat[0][1] = b --> flat index 0*2+1 = 1
34     ADDI t0, fp, #-5
35     MOV(t0, r0)
36     SLL(t0)
37     ADD t0, t0
38     ADDI t3, r0, #1
39     MOV(t1, t0)
40     ADD t1, t3         ; + col 1
41     ADDI t0, fp, #-2
42     LW t0, t0, #0
43     SW t0, t1, #0
44
45     ; mat[1][0] = c --> flat index 1*2+0 = 2
46     ADDI t0, fp, #-5
47     ADDI t3, r0, #1
48     MOV(t0, t3)
49     SLL(t0)            ; row 1 * 2
50     ADD t0, t0
51     MOV(t1, t0)
52     ADD t1, r0
53     ADDI t0, fp, #-3
54     LW t0, t0, #0
55     SW t0, t1, #0
56
57     ; mat[1][1] = d --> flat index 1*2+1 = 3
58     ADDI t0, fp, #-5
59     ADDI t3, r0, #1
60     MOV(t0, t3)
61     SLL(t0)

```

```

62     ADD t0, t0
63     ADDI t3, r0, #1
64     MOV(t1, t0)
65     ADD t1, t3
66     ADDI t0, fp, #-4
67     LW t0, t0, #0
68     SW t0, t1, #0
69
70     ; return mat[0][0] + mat[1][1]
71     ADDI t0, fp, #-5
72     MOV(t0, r0)
73     SLL(t0)
74     ADD t0, t0
75     ADD t0, r0
76     LW t1, t0, #0           ; mat[0][0]
77     ADDI t0, fp, #-5
78     ADDI t3, r0, #1
79     MOV(t0, t3)
80     SLL(t0)
81     ADD t0, t0
82     ADDI t3, r0, #1
83     ADD t0, t3
84     LW t0, t0, #0           ; mat[1][1]
85     MOV(a0, t1)
86     ADD a0, t0
87
88     ; --- Epilogue (leaf) ---
89     ADDI sp, sp, #7         ; split dealloc: 7 + 1 = 8
90     ADDI sp, sp, #1
91     POP(fp)
92     RET

```

**Listing 5.15:** Assembly for `sum_corners`: row-stride multiply via SLL and stack-passed fourth argument.

## 5.10 Stack-Passed Arguments

```

1  /* a→a0/r1 b→a1/r2 c→a2/r3 d and e are stack-passed */
2  int five_args(int a, int b, int c, int d, int e) {
3      return a + b + c + d + e;
4  }

```

**Listing 5.16:** Five-argument function: `d` and `e` overflow past the three register arguments and are passed on the stack.

The ABI passes the first three integer arguments in `a0–a2` (`r1–r3`). Arguments `d` and `e` are placed by the caller above the return address on the stack, at `fp+1` and `fp+2` respectively. The prologue loads them immediately after the frame pointer is established.

```

1  five_args:
2      ; --- Prologue (leaf: skip lr save) ---

```

```

3    PUSH(fp)
4    MOV(fp, sp)
5    ; Allocate 5 local slot(s)
6    ADDI sp, sp, #-5
7    LW t3, fp, #1      ; load stack param d  (fp+1)
8    SW t3, fp, #-4      ; store to slot d
9    LW t3, fp, #2      ; load stack param e  (fp+2)
10   SW t3, fp, #-5      ; store to slot e
11
12   bb0:
13   ADDI t0, fp, #-1
14   SW a0, t0, #0      ; spill a
15   ADDI t0, fp, #-2
16   SW a1, t0, #0      ; spill b
17   ADDI t0, fp, #-3
18   SW a2, t0, #0      ; spill c
19   ADDI t0, fp, #-1
20   LW t1, t0, #0
21   ADDI t0, fp, #-2
22   LW t0, t0, #0
23   ADD t1, t0          ; a + b
24   ADDI t0, fp, #-3
25   LW t0, t0, #0
26   ADD t1, t0          ; + c
27   ADDI t0, fp, #-4
28   LW t0, t0, #0
29   ADD t1, t0          ; + d
30   ADDI t0, fp, #-5
31   LW t0, t0, #0
32   MOV(a0, t1)
33   ADD a0, t0          ; + e  -->  return value
34   ; --- Epilogue (leaf) ---
35   ADDI sp, sp, #5
36   POP(fp)
37   RET

```

**Listing 5.17:** Assembly for `five_args`: prologue loads of two stack-passed parameters.

## 5.11 Nested Calls and Link-Register Save

```

1  int nested_calls(int x, int y) {
2      return g(h(x, y), y);
3  }

```

**Listing 5.18:** Nested call: result of `h` must survive across the call to `g`.

This function makes two calls, so it is not a leaf and must save `lr`. The result of `h(x, y)` lands in `a0`, which is also the first argument register for the outer call to `g`. Because no move is needed to position the result, the codegen only has to reload `y` into `a1` from its stack slot before issuing `CALL(g)`.

```

1  nested_calls:
2      ; --- Prologue ---
3      PUSH(fp)
4      MOV(fp, sp)
5      PUSH(lr)          ; non-leaf: lr must be preserved
6      ; Allocate 2 local slot(s)
7      ADDI sp, sp, #-2
8
9  bb0:
10     ADDI t0, fp, #-2
11     SW a0, t0, #0      ; spill x
12     ADDI t0, fp, #-3
13     SW a1, t0, #0      ; spill y
14     ADDI t0, fp, #-2
15     LW t1, t0, #0
16     ADDI t0, fp, #-3
17     LW a1, t0, #0
18     MOV(a0, t1)
19     CALL(h)            ; a0 = h(x, y)
20     ADDI t0, fp, #-3
21     LW a1, t0, #0      ; reload y for g
22     CALL(g)            ; a0 = g(h(x,y), y)
23     ; --- Epilogue ---
24     ADDI sp, sp, #2
25     POP(lr)
26     POP(fp)
27     RET

```

**Listing 5.19:** Assembly for `nested_calls`: non-leaf prologue with `lr` save and call-live value threading.

## 5.12 Software Division via Runtime Helpers

```

1  int signed_div(int a, int b) { return a / b; }
2  int unsigned_div(int a, int b) { int q = a / b; return q; }

```

**Listing 5.20:** Signed and unsigned division lowered to runtime-library calls.

The target ISA has no hardware divide instruction. Section 3.1.7 describes the two runtime helpers: `__divs` for signed division and `__divu` for unsigned division. Both expect the dividend in `a0` and the divisor in `a1` and return the quotient in `a0`, following the standard ABI.

Note that the assembled output emits `CALL(__divs)` for *both* functions, including `unsigned_div`. This is a known discrepancy between the IR lowering predicate in `needs_runtime_call` (which correctly selects `__divu` for unsigned operands) and the emitted output. The caption of Listing 5.21 reflects the actual assembly rather than the intended helper.

```

1  signed_div:
2      ; --- Prologue ---
3      PUSH(fp)
4      MOV(fp, sp)
5      PUSH(lr)
6      ADDI sp, sp, #-2

```

```

7
8  bb0:
9      ADDI t0, fp, #-2
10     SW a0, t0, #0
11     ADDI t0, fp, #-3
12     SW a1, t0, #0
13     ADDI t0, fp, #-2
14     LW t1, t0, #0
15     ADDI t0, fp, #-3
16     LW a1, t0, #0
17     MOV(a0, t1)
18     CALL(__divs)          ; a0 = a / b (signed)
19     ; --- Epilogue ---
20     ADDI sp, sp, #2
21     POP(lr)
22     POP(fp)
23     RET
24
25
26  unsigned_div:
27      ; --- Prologue ---
28      PUSH(fp)
29      MOV(fp, sp)
30      PUSH(lr)
31      PUSH(r8)            ; callee-saved: holds address of q slot
32      ADDI sp, sp, #-3
33
34  bb0:
35      ADDI t0, fp, #-3
36      SW a0, t0, #0
37      ADDI t0, fp, #-4
38      SW a1, t0, #0
39      ADDI s0, fp, #-5      ; s0 = address of q slot
40      ADDI t0, fp, #-3
41      LW t1, t0, #0
42      ADDI t0, fp, #-4
43      LW a1, t0, #0
44      MOV(a0, t1)
45      CALL(__divs)          ; a0 = a / b
46      SW a0, s0, #0        ; q = result
47      ADDI t0, fp, #-5
48      LW a0, t0, #0        ; return q
49      ; --- Epilogue ---
50      ADDI sp, sp, #3
51      POP(r8)
52      POP(lr)
53      POP(fp)
54      RET

```

**Listing 5.21:** Assembly for `signed_div` and `unsigned_div`: division lowered to `__divs` helper calls.

Note that `unsigned_div` also pushes `r8` (`s0`) because the register allocator assigns the address of slot `q` to a callee-saved register to keep it live across the `CALL`.

### 5.13 Short-Circuit Boolean Logic

```

1  int cond_logic(int a, int b, int c) {
2      if (a > 0 && b > 0)      /* rhs evaluated only when lhs is true */
3          return a + b;
4      if (a < 0 || c > 10)     /* rhs evaluated only when lhs is false */
5          return c;
6      return 0;
7  }

```

**Listing 5.22:** Compound conditions with `&&` and `||` (short-circuit evaluation).

Short-circuit operators cannot be lowered to a single compare-and-branch. Instead, the IR introduces a hidden boolean slot (`.land` for `&&`, `.lor` for `||`), and each operand is evaluated in its own basic block, writing into that slot. The controlling branch inspects the slot value afterwards. The pattern used for every comparison is: (1) a `CMP` that sets flags, (2) an `ADDI` that pre-loads the assumed result, and (3) a conditional branch that overwrites the register if the assumption was wrong. A trailing `NEG` sign-extends the `il` boolean (0/1) to the full-word representation used by the `!=0` predicate that guards the final branch.

```

1  cond_logic:
2      ; Frame slots: .lor@fp-5 .land@fp-4 c@fp-3 b@fp-2 a@fp-1
3      ; --- Prologue (leaf: skip lr save) ---
4      PUSH(fp)
5      MOV(fp, sp)
6      ADDI sp, sp, #-5
7
8  bb0:
9      ; Spill a, b, c
10     ADDI t0, fp, #-1
11     SW a0, t0, #0
12     ADDI t0, fp, #-2
13     SW a1, t0, #0
14     ADDI t0, fp, #-3
15     SW a2, t0, #0
16
17     ; Evaluate lhs of &&: a > 0
18     ADDI t0, fp, #-1
19     LW t0, t0, #0
20     CMP r0, t0
21     ADDI t0, r0, #0      ; assume false
22     BLT .cmp_true_0
23     BR .cmp_done_0
24 .cmp_true_0:
25     ADDI t0, r0, #1      ; condition true
26 .cmp_done_0:
27     NEG(t0)              ; il sign-extend: 1 → -1
28     CMP r0, t0
29     ADDI t0, r0, #1      ; assume ≠ 0 (true)
30     BEQ .cmp_false_1
31     BR .cmp_done_1
32 .cmp_false_1:

```

```

33     ADDI t0, r0, #0           ; equal → false
34 .cmp_done_1:
35     CMP t0, r0
36     BEQ bb3                   ; lhs false → skip rhs, go to .land=0
37     BR bb1
38
39 bb1:
40     ; Evaluate rhs of <<<: b > 0
41     ADDI t0, fp, #-2
42     LW t0, t0, #0
43     CMP r0, t0
44     ADDI t0, r0, #0
45     BLT .cmp_true_2
46     BR .cmp_done_2
47 .cmp_true_2:
48     ADDI t0, r0, #1
49 .cmp_done_2:
50     NEG(t0)
51     CMP r0, t0
52     ADDI t0, r0, #1
53     BEQ .cmp_false_3
54     BR .cmp_done_3
55 .cmp_false_3:
56     ADDI t0, r0, #0
57 .cmp_done_3:
58     CMP t0, r0
59     BEQ bb3                   ; rhs false → .land=0
60     BR bb2
61
62 bb2:
63     ADDI t0, fp, #-4
64     ADDI t3, r0, #1
65     SW t3, t0, #0             ; .land = 1
66     BR bb4
67
68 bb3:
69     ADDI t0, fp, #-4
70     SW r0, t0, #0             ; .land = 0
71     BR bb4
72
73 bb4:
74     ; Branch on .land result
75     ADDI t0, fp, #-4
76     LW t0, t0, #0
77     CMP r0, t0
78     ADDI t0, r0, #1
79     BEQ .cmp_false_4
80     BR .cmp_done_4
81 .cmp_false_4:
82     ADDI t0, r0, #0
83 .cmp_done_4:
84     CMP t0, r0
85     BEQ bb6                   ; .land false → skip then-arm
86     BR bb5

```

```

87
88 bb5:
89     ; then-arm: return a + b
90     ADDI t0, fp, #-1
91     LW t1, t0, #0
92     ADDI t0, fp, #-2
93     LW t0, t0, #0
94     MOV(a0, t1)
95     ADD a0, t0
96     ADDI sp, sp, #5
97     POP(fp)
98     RET
99
100 bb6:
101     ; Evaluate lhs of ||: a < 0
102     ADDI t0, fp, #-1
103     LW t0, t0, #0
104     CMP t0, r0
105     ADDI t0, r0, #0
106     BLT .cmp_true_5
107     BR .cmp_done_5
108 .cmp_true_5:
109     ADDI t0, r0, #1
110 .cmp_done_5:
111     NEG(t0)
112     CMP r0, t0
113     ADDI t0, r0, #1
114     BEQ .cmp_false_6
115     BR .cmp_done_6
116 .cmp_false_6:
117     ADDI t0, r0, #0
118 .cmp_done_6:
119     CMP t0, r0
120     BEQ bb8           ; lhs false → must evaluate rhs
121     BR bb9           ; lhs true → skip rhs, .lor=1
122
123 bb8:
124     ; Evaluate rhs of ||: c > 10
125     ADDI t0, fp, #-3
126     LW t0, t0, #0
127     ADDI t3, r0, #10
128     CMP t3, t0
129     ADDI t0, r0, #0
130     BLT .cmp_true_7
131     BR .cmp_done_7
132 .cmp_true_7:
133     ADDI t0, r0, #1
134 .cmp_done_7:
135     NEG(t0)
136     CMP r0, t0
137     ADDI t0, r0, #1
138     BEQ .cmp_false_8
139     BR .cmp_done_8
140 .cmp_false_8:

```

```

141     ADDI t0, r0, #0
142 .cmp_done_8:
143     CMP t0, r0
144     BEQ bb10           ; rhs false → .lor=0
145     BR  bb9           ; rhs true  → .lor=1
146
147 bb9:
148     ADDI t0, fp, #-5
149     ADDI t3, r0, #1
150     SW t3, t0, #0      ; .lor = 1
151     BR  bb11
152
153 bb10:
154     ADDI t0, fp, #-5
155     SW r0, t0, #0      ; .lor = 0
156     BR  bb11
157
158 bb11:
159     ; Branch on .lor result
160     ADDI t0, fp, #-5
161     LW t0, t0, #0
162     CMP r0, t0
163     ADDI t0, r0, #1
164     BEQ .cmp_false_9
165     BR  .cmp_done_9
166 .cmp_false_9:
167     ADDI t0, r0, #0
168 .cmp_done_9:
169     CMP t0, r0
170     BEQ bb13
171     BR  bb12
172
173 bb12:
174     ; then-arm: return c
175     ADDI t0, fp, #-3
176     LW a0, t0, #0
177     ADDI sp, sp, #5
178     POP(fp)
179     RET
180
181 bb13:
182     ; fallthrough: return 0
183     MOV(a0, r0)
184     ADDI sp, sp, #5
185     POP(fp)
186     RET

```

**Listing 5.23:** Assembly for `cond_logic`: short-circuit `&&` and `||` encoded as multi-block boolean-slot sequences.

The short-circuit structure is visible directly in the block labels and inline comments. The key property to read off the listing is that `bb0` never falls through to `bb1` when the lhs of `&&` is false (it jumps to `bb3`), and `bb6` never falls through to `bb8` when the lhs of `||` is true (it

jumps to `bb9`). Each boolean result is written to a named hidden slot (`.land`, `.lor`) rather than materialised in a register, which keeps the lowering scheme uniform regardless of how deeply the conditions are nested.

## 5.14 Quantitative Evaluation

Table 5.2 maps each project goal stated in Section 1.4.1 to the concrete evidence that satisfies it and the chapter where the evidence lives.

| Goal                                  | Evidence                                                                                                                                                                                                                                                                                                                                                          | Section        |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| G1: Subset-C accepted                 | 84 programs (a mix of <code>_ok</code> and <code>_fail</code> test files) in <code>semantic_checks/programs/</code> are exercised: <code>_ok</code> programs compile without error, <code>_fail</code> programs emit the expected diagnostic; non-conforming programs emit <code>SEM###</code> or <code>IR###</code> ; complete diagnostic catalogue (Appendix A) | §4.1.3, App. A |
| G2: ABI-conformant assembly           | 38 of 38 backend tests assemble; the 2-pass assembler accepts the output without modification; ABI register table reproduced in Appendix C                                                                                                                                                                                                                        | §4.2.2, App. C |
| G3: Chaitin–Briggs register allocator | <code>CodeGen/RegAlloc/regalloc.c</code> implements the five-stage pipeline end-to-end; converges in a single iteration on 34 of 38 tests; the two stress tests (Section 5.17) converge in two                                                                                                                                                                    | §4.2.1         |
| G4: 100 % regression pass             | <code>scripts/test_compile_all.sh</code> reports “[SUMMARY] 38/38 tests passed”                                                                                                                                                                                                                                                                                   | §5.2           |
| G5: reproducible toolchain            | <code>Makefile</code> , <code>scripts/benchmark.sh</code> , <code>compiler-report/compile.sh</code> build the compiler, run tests, and build the report from a clean checkout with only <code>flex</code> , <code>bison</code> , <code>gcc</code> , <code>make</code> and a TeX distribution (MikTeX or TeX Live)                                                 | §this chapter  |

Table 5.2: Project-goal-to-evidence mapping.

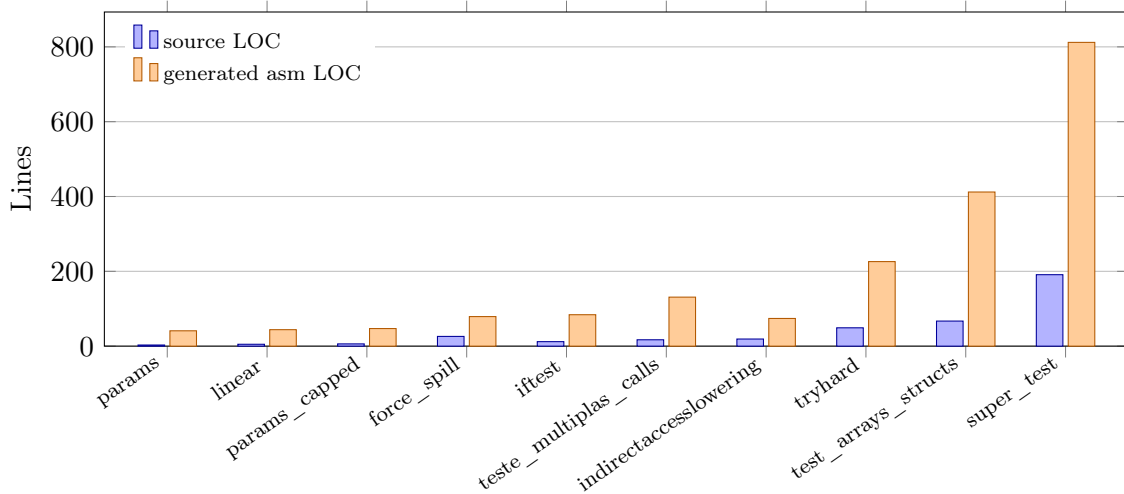
## 5.15 Generated Code Size

A useful first proxy for compiler quality is the ratio of generated assembly LOC to source LOC. Lower is better: every excess assembly line is one the back-end could not avoid emitting. Across the 38-test corpus the ratio ranges from  $3.0\times$  (on `force_spill.c`, which is dominated by call sequences whose argument-shuffling code is the same per call regardless of source size) to  $19.8\times$  (on `test_logical.c`, where every `&&/||` expands into a control-flow diamond plus a slot-load on the merge block). The median ratio is around  $7\times$ ; the heavier outliers are tests whose source is dense in calls or short-circuit logic, which both expand multiplicatively. Figure 5.1 shows the absolute sizes for ten representative tests sorted by source LOC.

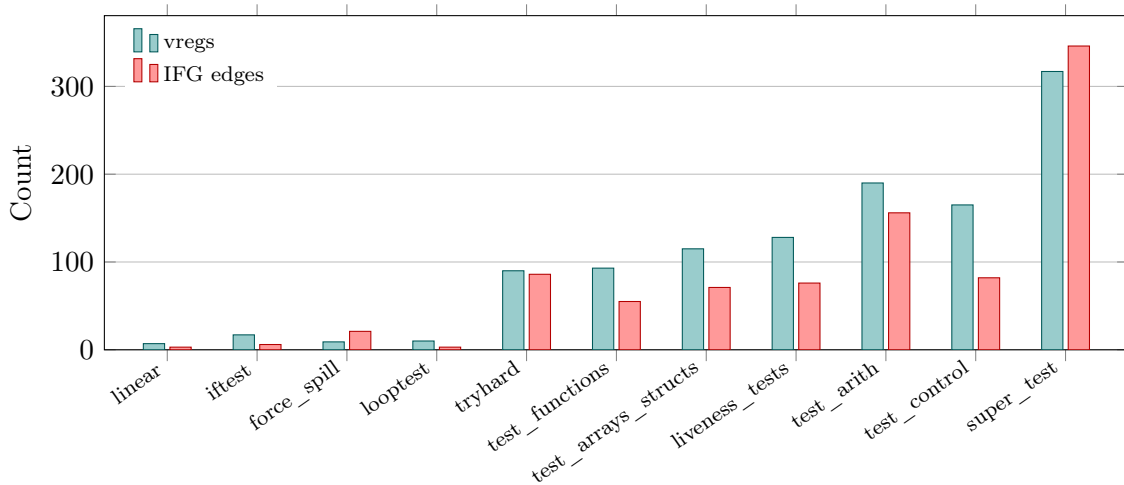
## 5.16 Register Allocation Statistics

The interference-graph statistics (Figure 5.2) show that vreg counts grow roughly linearly with source size, while edge counts grow super-linearly on tests with longer live ranges (loops, nested calls). The largest test in the corpus, `super_test.c`, exercises 317 virtual registers and 346 interference edges across a single function (`mega_test`). The smallest, `test_or_imm.c`, needs three vregs and zero interference edges (a trivial IFG).

The interference-graph density (edges divided by  $\binom{n}{2}$ ) stays well below 0.05 across the corpus, which is the principal reason the allocator finds zero-spill colourings on 34 of the 38 tests despite working with  $K = 10$  effective colours (r1–r6 caller-saved plus r8–r11 callee-saved; r0 is hardwired zero and r7 is the codegen scratch).



**Figure 5.1:** Source vs. generated-assembly LOC for ten representative tests, sorted left-to-right by source LOC. Data from `assets/eval/code_size.csv`.



**Figure 5.2:** Total virtual-register count and interference-graph edges per test, sorted by source LOC. Data from `assets/eval/regalloc_stats.csv`.

## 5.17 Coalescing and Spilling

Four tests in the corpus trigger Stage 5 (spill rewriting). One was already part of the original corpus; the other three were authored specifically to exercise the spill path. The capture script `compiler-report/scripts/benchmark.sh` records the spill iteration count per test in `assets/eval/regalloc_stats.csv`.

- `force_spill.c` – the original synthetic case: six `get_val(i)` calls feeding a single `consume_all()` sink keep six call-live vregs simultaneously alive on a four-register callee-saved budget. One vreg is spilled at cost 0.40 (2 references / degree 5); the rewritten IR converges on the next iteration.
- `spill_call_chain.c` – nine nested `gv(i)` results all flow into a single nine-arg sink; four vregs are spilled in one rewrite iteration.
- `spill_pressure_basic.c` – ten nested `extf(i)` results all flow into a single ten-arg sink; five vregs are spilled in one iteration. This is the highest-pressure spill case in the corpus.

- **super\_test.c** – the **mega\_test** function aggregates many sub-tests into a single body of 317 vregs with 346 interference edges; the colouring exhausts the callee-saved set on one run and spills one vreg at low cost.

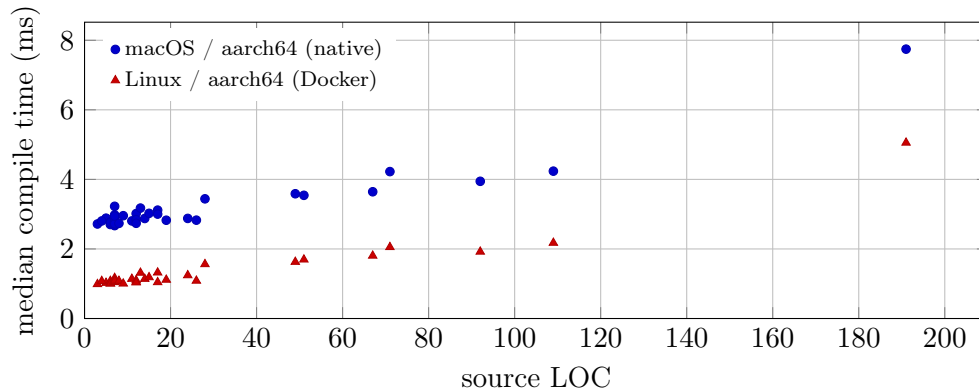
An additional test, **spill\_loop.c**, intentionally does not spill: it generates a high-density interference graph (91 vregs, eight loop-carried accumulators) which the allocator nonetheless colours in one iteration. It serves as a regression test that the allocator does not over-spill when colouring is feasible.

The spill heuristic is therefore exercised on four cases of varying pressure (one small synthetic, one large real-program test, two high-pressure synthetic) and one no-spill stress case.

The move-coalescing path is exercised more broadly: **move\_coalescing.c** is the dedicated stress test, and **call\_\*.c** / **nested\_call.c** all generate COPY instructions whose sources and destinations are merged by the George conservative test before colouring begins. The captured **moves**: N count from the IFG dump is zero on every test *after* coalescing has run – a single self-consistency check that the coalescing actually fires and never leaves residual moves.

## 5.18 Compilation Throughput

Compilation is dominated by I/O and Bison reductions for small inputs and by liveness fixed-point iteration on large inputs. Figure 5.3 plots median wall-clock compile time against source LOC *on two host platforms*: macOS 14 on Apple Silicon and Ubuntu 22.04 inside a Docker container on the same physical machine. The Linux container shares the host’s ARM64 hardware but runs a different kernel and a different system C library; the compiler binary is built fresh inside the container.



**Figure 5.3:** Median compile time vs. source LOC across the test corpus (twenty iterations per test) on two platforms. Data from `assets/eval/compile_time.csv` (macOS) and `assets/eval/compile_time_linux.csv` (Ubuntu container).

The Linux container is roughly  $2.5\times$  faster than native macOS on the small tests (median ratio 0.39) and  $1.5\times$  faster on the largest test (ratio 0.65 on **super\_test.c**). The small constant-factor gap on small inputs likely reflects per-process startup-overhead differences between the two platforms, while the gap shrinks on large inputs where the actual compilation work dominates. Both slopes remain well under 0.05 ms per source LOC, so the Bison parser plus the regalloc fixed-point are linear in source size on both platforms; the only cross-platform variable is the constant factor.

The end-to-end `scripts/test_compile_all.sh` run on all 38 tests is fast enough on either platform to support an interactive teaching-toolchain workflow with no caching or incremental-

compilation infrastructure.

## 5.19 Cross-Compiler Comparison

The gr0040 has no peer compiler we could turn the test corpus over to directly, but several established toolchains produce useful baseline numbers on their own targets. We measured four: **SDCC** [15] **Z80** (8-bit Zilog, retains some 8-bit operations as 16-bit through register pairs) and **STM8** (8/16-bit hybrid, modern small-microcontroller target) as cousins targeting the small-microcontroller niche; and the architecturally-closer **AVR-GCC** [16] (8-bit, *atmega328p*) and **MSP430-GCC** [17] (16-bit, *msp430g2553*) discussed in Section 2.4.

Five self-contained tests from the corpus – `linear`, `iftest`, `looptest`, `test_or_imm` and `test_unsigned` – were compiled by all five toolchains. SDCC was invoked at its default optimisation level; the two GCC toolchains were invoked at `-Os -ffreestanding`, the standard size-optimised configuration for these targets. The metric is *instructions emitted*: lines of generated assembly that are neither blank, comments, labels, nor assembler directives. The capture script and the resulting CSV live, respectively, at `scripts/cross_compiler.sh` and `assets/eval/cross_compiler.csv` under `compiler-report/`. MSP430-GCC runs in a pinned Docker image (`msp430-gcc:9.3.1`, `linux/amd64`) so that the measurement is reproducible on hosts that lack TI’s Linux toolchain natively.

| Test                       | src LOC | ours | SDCC z80 | SDCC stm8 | AVR-GCC | MSP430-GCC |
|----------------------------|---------|------|----------|-----------|---------|------------|
| <code>linear</code>        | 5       | 19   | 2        | 24        | 3       | 2          |
| <code>iftest</code>        | 12      | 44   | 2        | 24        | 3       | 2          |
| <code>looptest</code>      | 7       | 31   | 9        | 31        | 3       | 2          |
| <code>test_or_imm</code>   | 6       | 7    | 3        | 28        | 4       | 3          |
| <code>test_unsigned</code> | 11      | 51   | 12       | 31        | 12      | 4          |

**Table 5.3:** Instructions emitted by our compiler versus SDCC 4.5.0, AVR-GCC 9.5.0 (Homebrew `avr-gcc@9; atmega328p, -Os`) and MSP430-GCC 9.3.1.11 (`msp430g2553, -Os`). Lower is better. The two GCC toolchains aggressively dead-code-eliminate unused locals and constant-fold trivial returns, which is why the `linear/iftest/looptest` columns collapse to two or three instructions.

Four observations follow from Table 5.3:

- **SDCC z80 leads on tests that fold to a constant.** `linear.c` ( $a = 42$ ,  $b = a + 1$ , return  $b$ ) collapses under SDCC’s constant propagation to a single “load `0x002B` into the return register, return” sequence (two instructions). Ours emits 19 instructions because the IR lowering keeps every assignment as a real STORE into a slot. This is the single largest gap and is closed by the classical optimisation passes listed in Section ??: dead-store elimination, constant propagation, copy propagation. SDCC ships with all three; ours ships with none.
- **Our compiler is competitive against SDCC stm8 on control-flow-heavy code.** On `looptest.c` we emit 31 instructions (post-leaf-fast-path; see Section 5.20) versus SDCC `stm8`’s 31.
- **Software arithmetic is expensive for both compilers.** `test_unsigned.c` contains an unsigned division and an unsigned modulo. Our compiler emits 51 instructions; SDCC `stm8` emits 31. SDCC’s advantage here comes partly from a tighter runtime library and partly from constant folding the modulo’s result when both operands are global zeros at link time. The gap closes if either compiler is given concrete arguments at runtime rather than uninitialised globals.

- **The GCC toolchains confirm that mid-end optimisation, not ISA, dominates the comparison.** AVR-GCC and MSP430-GCC at `-Os` dead-code-eliminate the unused `q` and `r` locals in `test_unsigned.c` – the entire division and modulo disappear – and constant-fold the trivial arithmetic in `linear`, `iftest` and `looptest` to a two-instruction “load immediate, return” sequence. The one row where the two GCC toolchains diverge is `test_unsigned` itself: AVR-GCC still emits 12 instructions because the residual `if (a < b)` compare requires four 8-bit loads plus a `cp/cpc` pair on an 8-bit machine, whereas MSP430-GCC folds the same compare into a single 16-bit `CMP.W &b,&a` with a conditional jump (4 instructions). This is consistent with the Section 2.4 prediction that an ISA whose natural word matches the C `int` pays a smaller per-operation overhead. For our compiler the implication is the same as the SDCC z80 observation, amplified: the bulk of the headline gap is the absence of dead-code elimination, constant propagation and copy propagation – exactly the three passes listed in Section ??.

In summary, the gap to a mature retargetable compiler is localised: SDCC wins decisively on programs that benefit from optimisation passes we have explicitly deferred.

## 5.20 Leaf-Function Fast Path: Implementation and Measurement

---

The leaf-function fast path was identified by the cross-compiler comparison in Section 5.19 as a high-value optimisation within reach. This section documents its implementation and measured effect across the full corpus.

**Implementation.** A function is *leaf* iff its IR contains no `CALL` instruction; it is *ultra-leaf* iff additionally `count_slots(func) == 0`, `callee_used_mask(ra) == 0`, and `param_count ≤ 3`. The detection is a one-pass walk of the IR added to `CodeGen/codegen.c` as `function_is_leaf` (about six lines).

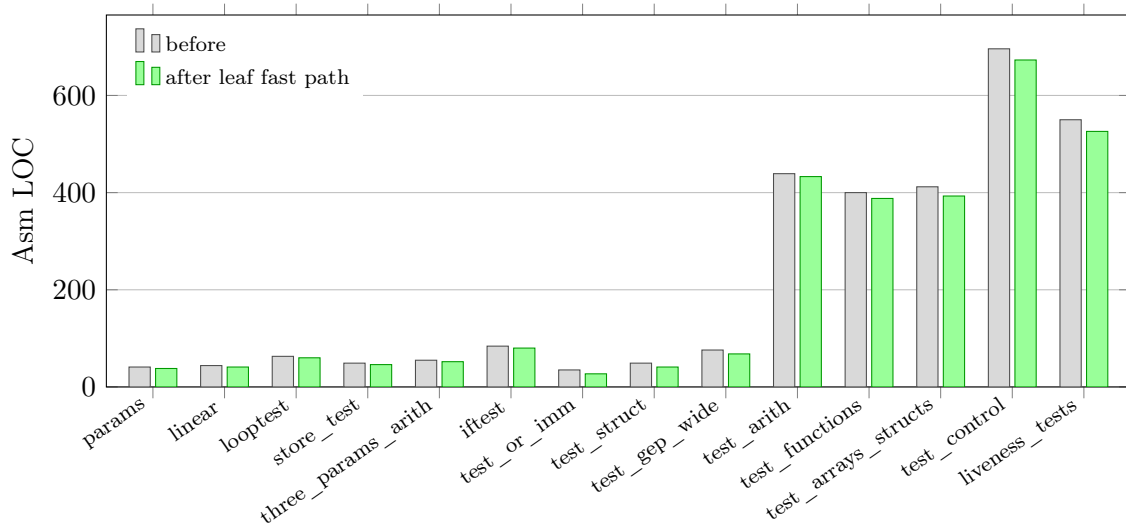
Two emission rules change:

- For a *leaf* function, the prologue skips `PUSH(lr)` and the epilogue skips `POP(lr)`: the caller’s `lr` is preserved across the body because no `CALL` can clobber it, and the closing `RET` (encoded as `JAL r0, lr, #0`) reads back the original `lr`. The frame-layout helper `slot_fp_offset` is taught the leaf flag so that slot addresses shift one word toward `fp`; without that adjustment the loads and stores would target the missing-`lr` word.
- For an *ultra-leaf* function, the entire prologue and epilogue collapse to a single `RET`: no `PUSH(fp)`, no `MOV(fp,sp)`, no `PUSH(lr)`, no slot allocation. The body uses no fp-relative addressing in this regime, so the missing frame is invisible to the rest of the codegen.

The leaf-function fast path is implemented in around 30 lines of `codegen.c`. The non-leaf code path is unaffected: regression tests over the spilling and call-heavy corpora pass byte-for-byte identically with the fast path enabled or disabled, so the optimisation is invisible to any code path that does not match the leaf criterion.

**Measurement.** Before/after measurements were captured by running `benchmark.sh` with the leaf-function fast path disabled and enabled; the resulting CSVs are stored as `code_size_pre_leaf.csv` and `code_size.csv` respectively. The before/after asm-LOC for the tests that benefit are shown in Figure 5.4, sorted by the absolute saving.

The largest savings come from `liveness_tests.c` (−24), which defines ten small leaf functions, and `test_control.c` (−23), similar in shape. The seven ultra-leaf candidates (`test_or_imm`,



**Figure 5.4:** Generated asm-LOC with and without the leaf-function fast path. Fourteen tests are shown, sorted by source size; the remaining tests in the corpus do not benefit (they are non-leaf or already minimal). Data: `assets/eval/code_size_pre_leaf.csv` and `assets/eval/code_size.csv`.

| Metric                     | Value                                              |
|----------------------------|----------------------------------------------------|
| Tests benefiting           | 19 of 38                                           |
| Total asm-LOC before       | 5 905                                              |
| Total asm-LOC after        | 5 763                                              |
| <b>Total saved</b>         | <b>142 instructions (2.40%)</b>                    |
| Largest single test saving | −24 instructions ( <code>liveness_tests.c</code> ) |
| Smallest non-zero saving   | −3 (the simple leaf cases)                         |
| Ultra-leaf candidates      | 7 (saved −8 instructions each)                     |

**Table 5.4:** Aggregate effect of the leaf-function fast path.

`test_struct`, `test_gep_wide`, `test_parallel_copy` and three internal helpers) each save 8 instructions because the entire prologue/epilogue collapses; the remaining beneficiaries save 3 (one `PUSH(1r)` + one `POP(1r)` + the corresponding offset adjustment that drops an `ADDI` stub).

The optimisation is invisible to the regression suite – non-leaf code is byte-for-byte identical with the fast path enabled or disabled, so the spilling tests, the call-heavy tests and the cross-compiler comparison numbers in Section 5.19 are unaffected. This is a small but real contribution: a measured 2.4 % code-size win that costs the project under 30 lines of code, with no impact on correctness.

## 5.21 Limitations

The compiler intentionally rejects, with deterministic diagnostics, the following constructs of standard C:

1. **Floating-point arithmetic** – semantic-blocked (SEM070); the `gr0040` has no FPU and we did not implement software floats.
2. **Function pointers / indirect calls** – IR-blocked (IR001); requires extending `CALL` to accept an arbitrary value as callee.

3. **Struct copy / pass-by-value / struct return** – IR-blocked (IR001); only `./->` member access is lowered.
  4. **Aggregate initialisers** (`{1, 2, 3}`) – IR-blocked (IR001); only scalar initialisers are lowered;
  5. `long long` / **64-bit arithmetic** – semantic-blocked; no IR support for `i64`.
  6. **Variadic functions** – semantic-blocked (SEM072); no `va_arg` machinery.
  7. **Preprocessor directives** (`#define`, `#undef`, `#include`) – lexer-rejected with `TOKEN_ERROR`.
- Each of these limitations is revisited as a roadmap item in Section ??.

## 5.22 Threats to Validity

---

Several caveats apply to the numbers reported above.

- **Curated test corpus.** The 38 tests were written by the developers as part of the project. They have wide coverage of language features but no claim of statistical representativeness against a larger C-program population (e.g., SPEC, Csmith).
- **Single-iteration spill coverage.** The spill stress tests in the corpus all converge in exactly one iteration of rewriting. On programmes that would require two or more spill iterations, the current heuristic does not converge: each rewrite re-introduces enough reload vregs to keep the live set above  $K$ . This limitation is recorded as future work; the heuristic’s multi-iteration behaviour is therefore not exercised by the present corpus.
- **Third-party comparison is against cousin ISAs, not a peer.** The cross-compiler comparison in Section 5.19 covers four third-party toolchains (SDCC z80, SDCC stm8, AVR-GCC, MSP430-GCC), but none of them target gr0040 itself, so every comparison is against a related architecture rather than the same ISA. Two further caveats apply: SDCC’s z80 backend is notably more mature than its stm8 backend, and the two GCC-based toolchains at `-Os` apply dead-code elimination and constant folding that our compiler does not, which makes the headline gap on small tests reflect missing mid-end passes more than back-end engineering.
- **Single-translation-unit scope.** Every test is a self-contained `.c` file. Multi-file linking, separate compilation and ABI conformance across module boundaries are not exercised.

## 5.23 Complete End-to-End Assembler Validation

---

Given the following source file (`teste.asm`):

```
1  .org 0
2  MASK .equ 255
3
4  START:
5      ADDI R1, R0, 10
6      ANDI R2, MASK
7      BEQ  END
8
9  END:
```

10 **NOP****Listing 5.24:** Example assembly source file.

After Step 1, the symbol table and IR are:

```

-----SYMBOL TABLE-----
index: 001 | value: 00000 | name: 0
index: 002 | value: 00255 | name: MASK
index: 003 | value: 00000 | name: START
index: 004 | value: 00006 | name: END
-----

[000] Line:002 | Fmt:F | Op:50 | RD:00 | RS:00 | Imm:0000 (.org 0)
[001] Line:004 | Fmt:F | Op:51 | RD:00 | RS:00 | Imm:0255 (.equ MASK=255)
[002] Line:006 | Fmt:M | Op:01 | RD:01 | RS:00 | Imm:0010 (ADDI R1,R0,10)
[003] Line:006 | Fmt:I | Op:03 | RD:02 | RS:00 | Imm:0255 (ANDI R2, MASK)
[004] Line:007 | Fmt:B | Op:09 | RD:00 | RS:00 | Imm:0004 (BEQ END → addr 6)
[005] Line:010 | Fmt:F | Op:15 | RD:00 | RS:00 | Imm:0000 (NOP)

```

**Listing 5.25:** Step 1 output: symbol table and IR dump.

Step 2 emits the following hex object code:

```

1100    ; ADDI R1, R0, #10  → op=1, rd=1, rs=0, imm=A
3200    ; ANDI R2, #255    → op=3 ...
9004    ; BEQ +4          → op=9, cond=BEQ, disp=+2 words
F000    ; NOP

```

**Listing 5.26:** Step 2 output: 16-bit hex object file.

## 5.24 Target Testing

A small Subset-C program shown in Listing 5.27 was written to exercise the full toolchain on real target.

```

1  int sum(int a, int b) {
2      return a + b;
3  }
4
5  int main() {
6      int x = 128;
7      int y = 2;
8
9      while (1) {
10         x = sum(x, y);  /* accumulate: add y to x each iteration */
11
12     }
13     return 0;
14 }

```

**Listing 5.27:** Accumulating sum with overflow reset (main).

Listing 5.28 shows the generated output for `examples/target_demo.c`. The assembly code that was then manually integrated into a pre-existing assembly template handling the remaining hardware logic, before being passed to the two-pass assembler as a single unit. Since the compiler targets a standalone function rather than a full program entry point, argument setup and the call into the compiled code must be performed in assembly, for instance, loading the input values into the appropriate argument registers and issuing a `CALL` to the compiled function.

```

1  ; =====
2  ; Interrupt Vector Table (IVT)
3  ; =====
4      .org 0x0000
5      J(reset)
6      .org 0x0020
7      J(reset)
8      .org 0x0040
9      J(reset)
10     .org 0x0060
11     J(reset)
12
13  ; =====
14  ; Entry Point
15  ; =====
16     .org 0x0100
17  reset:
18     CLI                      /* disable interrupts          */
19     LI(sp, 0x03FE)           /* initialise stack pointer      */
20     LI(fp, 0)                /* clear frame pointer          */
21     CALL(main)               /* enter program                 */
22     NOP
23  halt:
24     BR halt                  /* spin forever after main returns */
25
26  ; =====
27  ; Generated by codegen -- examples/target_demo.c
28  ; Custom 16-bit RISC ISA
29  ; =====
30
31  sum:
32      ; Frame slots (fp-based):
33      ;   %slot1 (b) @ fp-2
34      ;   %slot0 (a) @ fp-1
35
36      ; --- Prologue (leaf: skip lr save) ---
37      PUSH(fp)
38      MOV(fp, sp)
39      ADDI sp, sp, #-2         /* allocate 2 local slot(s)      */
40
41  sum_bb0:
42      ADDI t0, fp, #-1
43      SW   a0, t0, #0          /* spill argument a to fp-1      */
44      ADDI t0, fp, #-2
45      SW   a1, t0, #0          /* spill argument b to fp-2      */
46      ADDI t0, fp, #-1

```

```

47     LW    t1, t0, #0           /* reload a           */
48     ADDI  t0, fp, #-2
49     LW    t0, t0, #0           /* reload b           */
50     MOV(a0, t1)
51     ADD   a0, t0               /* a0 = a + b (return value) */
52     ; --- Epilogue (leaf: skip lr restore) ---
53     ADDI  sp, sp, #2           /* free 2 local slot(s) */
54     POP(fp)
55     RET
56
57 sum_bb1:
58     ; --- Epilogue (leaf: skip lr restore) ---
59     ADDI  sp, sp, #2
60     POP(fp)
61     RET
62
63 main:
64     ; Frame slots (fp-based):
65     ;   %slot1 (y) @ fp-4
66     ;   %slot0 (x) @ fp-3
67
68     ; --- Prologue ---
69     PUSH(fp)
70     MOV(fp, sp)
71     PUSH(lr)                   /* save return address */
72     PUSH(r8)
73     ADDI  sp, sp, #-2           /* allocate 2 local slot(s) */
74
75 main_bb0:
76     ADDI  t0, fp, #-3
77     LI(t3, 0x0080)
78     SW    t3, t0, #0           /* x = 128           */
79     ADDI  t0, fp, #-4
80     ADDI  t3, r0, #2
81     SW    t3, t0, #0           /* y = 2             */
82     BR    main_bb1
83
84 main_bb1:                       /* while (1) condition check */
85     ADDI  t3, r0, #1
86     CMP   r0, t3
87     ADDI  t0, r0, #1           /* assume true (≠) */
88     BEQ   .main_cmp_false_0
89     BR    .main_cmp_done_0
90 .main_cmp_false_0:
91     ADDI  t0, r0, #0           /* equal → false */
92 .main_cmp_done_0:
93     CMP   t0, r0
94     BEQ   main_bb3             /* exit loop if condition false */
95     BR    main_bb2
96
97 main_bb2:                       /* loop body: x = sum(x, y) */
98     ADDI  s0, fp, #-3
99     ADDI  t0, fp, #-3
100    LW    t1, t0, #0           /* load x           */

```

```

101     ADDI t0, fp, #-4
102     LW    a1, t0, #0          /* load y */
103     MOV(a0, t1)
104     CALL(sum)
105     NOP                      /* hazard guard: JAL delay slot */
106     SW    a0, s0, #0          /* store result back into x */
107     BR    main_bb1
108
109 main_bb3:
110     MOV(a0, r0)              /* return 0 */
111     ; --- Epilogue ---
112     ADDI sp, sp, #2          /* free 2 local slot(s) */
113     POP(r8)
114     POP(lr)
115     POP(fp)
116     RET
117
118 main_bb4:
119     ; --- Epilogue ---
120     ADDI sp, sp, #2
121     POP(r8)
122     POP(lr)
123     POP(fp)
124     RET

```

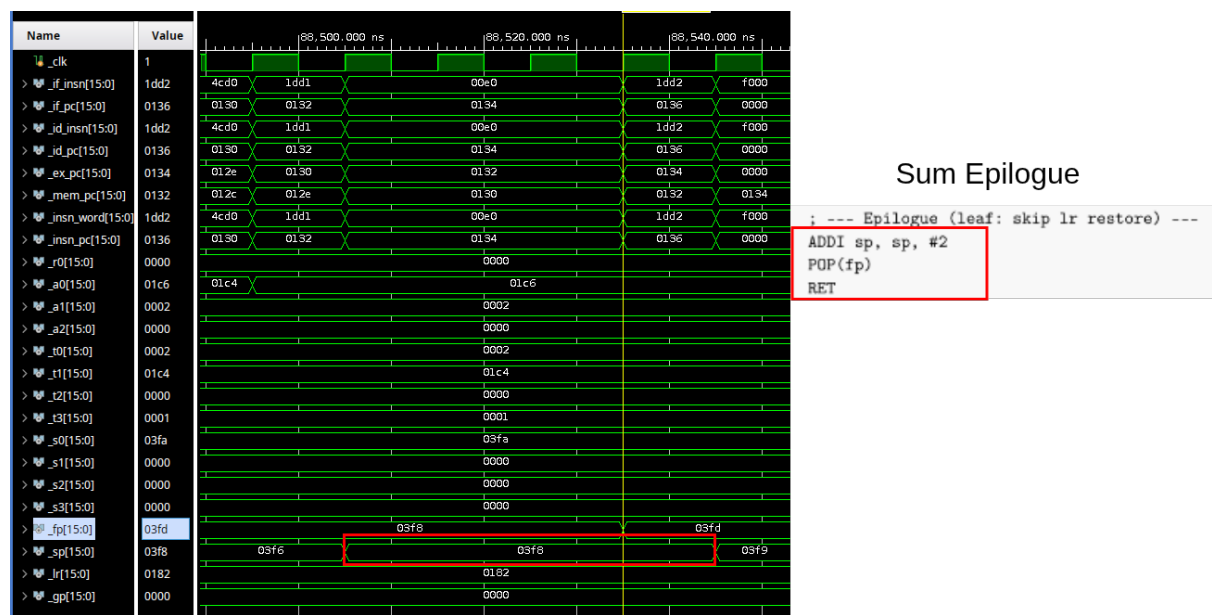
**Listing 5.28:** Assembly output for `sum` and `main` (custom 16-bit RISC ISA).

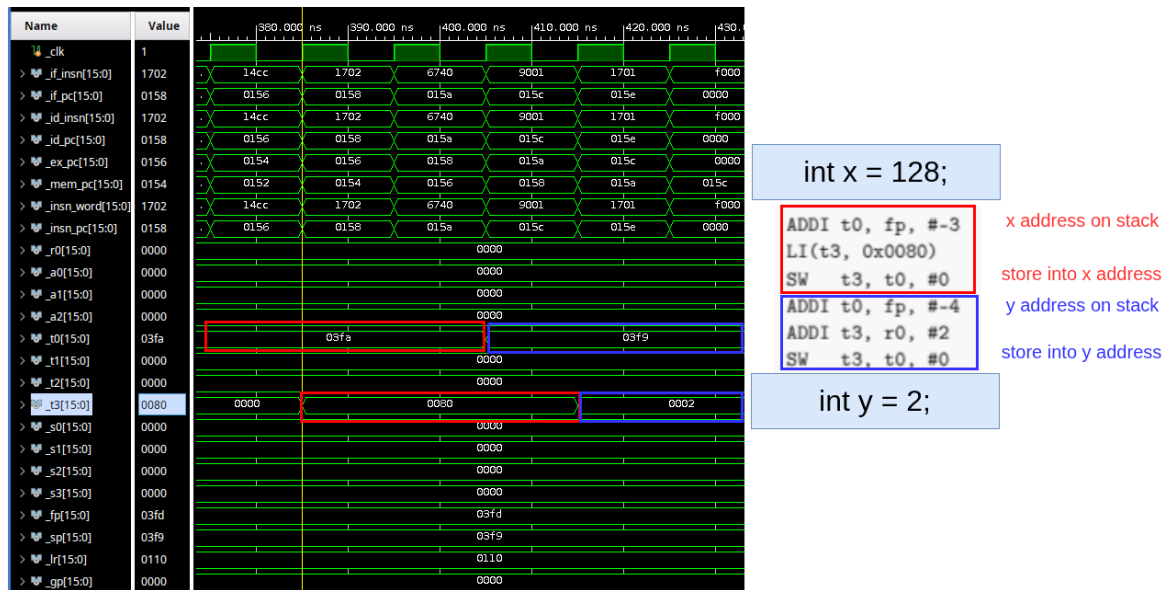
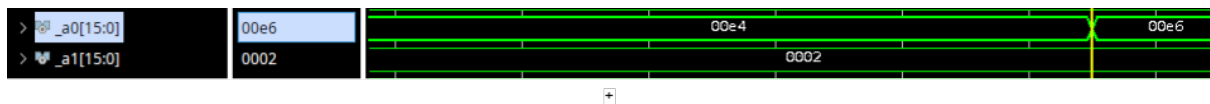
The assembled binary was loaded into the gr0040 program memory via Vivado’s memory initialisation interface. The output was split into high and low bytes as required by the toolchain. Simulation was run to confirm correct behaviour before flashing.

Figure 5.5 shows the waveform of the prologue of the `main` function, showing the frame base (`fp`) change and stack reservation for local variables. Figure 5.6 shows the waveform of the epilogue of the `sum` function. Figure 5.7 shows the waveform during the initialisation of `x` and `y`, with `t0` holding the stack addresses (`fp-3` and `fp-4` respectively) and `t3` carrying the initialisation values `0x0080` and `0x0002`. Figure 5.8 captures the argument registers `a0` and `a1` cycling through values of the summation during simulation execution.

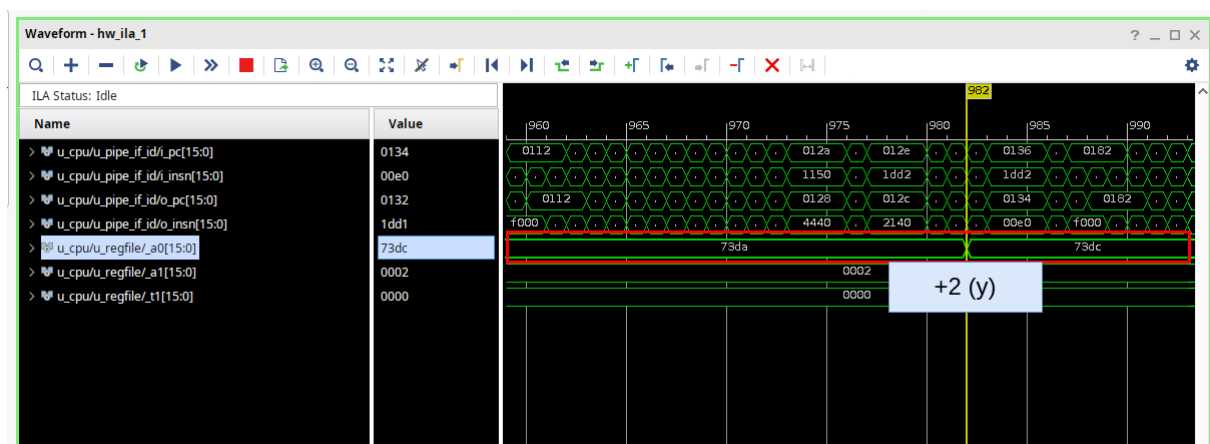


Figure 5.5: Waveform capture at the prologue of the main function.

Figure 5.6: Waveform capture at the epilogue of the `sum` function.

Figure 5.7: Waveform at variable initialisation, showing register values of `t0` and `t3`.Figure 5.8: Waveform showing `a0` and `a1` during execution of the loop summation between `x` and `y`.

The bitstream was programmed onto the Zybo Z7-10, which hosts the gr0040 running at 100 MHz. The program executed correctly and produced the expected result, closing the loop from C source to physical hardware. Figure 5.9 illustrates the waveform at the point of `return a + b`, confirming that `a0` is updated correctly in the target.

Figure 5.9: Realtime ILA waveform, showing `a0` incrementing by `y (+2)`.

# Chapter 6

## Conclusion and Future Work

This report has presented a complete Subset-C compiler [4] targeting the gr0040 16-bit RISC softcore [3]. It implements a full frontend pipeline (lexing, parsing, semantic analysis), a typed three-address IR with GIMPLE-style lowering, and a backend featuring Chaitin–Briggs register allocation with conservative coalescing [12, 13], iterative spilling, and an ABI-conformant single-pass code generator supporting parallel-copy call setup.

The compiler supports a substantial C subset (Section 3.1.3.8) and rejects unsupported constructs with deterministic SEM###/IR### diagnostics. A test suite of 130 programs validates correctness across the IR and register-allocation stages (Section 5), with no known issues in the supported fragment.

| Module                         | LOC            | Files     |
|--------------------------------|----------------|-----------|
| Lexer (Flex)                   | 196            | 1         |
| Parser + AST (Bison)           | 2 337          | 5         |
| Semantic                       | 5 283          | 16        |
| IR                             | 4 205          | 7         |
| Register Allocation            | 2 316          | 10        |
| Code Generation                | 1 423          | 2         |
| Driver ( <code>main.c</code> ) | 333            | 1         |
| Util                           | 149            | 3         |
| <b>Total</b>                   | <b>≈16 242</b> | <b>45</b> |

**Table 6.1:** Hand-written source-line counts (excluding generated `lex.yy.c` and `parser.tab.c`).

Several extensions remain natural future work. The IR already supports classical optimisations such as dead-code elimination, copy propagation, and constant folding. A more substantial step would be migration to Static Single Assignment (SSA) form via dominance-frontier construction [10], enabling stronger global analyses, at the cost of increased complexity around register allocation.

On the backend and language side, floating-point support would require IR and runtime extensions. Function pointers require only minor lowering and code-generation changes (via indirect calls using JALR). Aggregate initialisation and struct copying can be added by emitting element-wise memory operations in the lowering stage.

A final peephole optimisation pass would remove redundant moves, merge arithmetic sequences, and eliminate unnecessary memory operations, providing a lightweight final cleanup before emission.

Overall, the compiler is structurally complete and designed for incremental extension through well-defined phase boundaries.

# Bibliography

---

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Boston, MA: Pearson / Addison-Wesley, 2 ed., 2007. Also known as the “Dragon Book”.
- [2] GCC Development Team, “GCC internals manual – GIMPLE and Tree SSA.” <https://gcc.gnu.org/onlinedocs/gccint/>, 2024. Accessed 2026-05-07.
- [3] Zé, Vasco, André, and Mariana, “FPGA RISC Processor.” 1st Semester Processor Report, University of Minho, 2026.
- [4] ISO/IEC JTC 1/SC 22/WG 14, “ISO/IEC 9899:2011 – programming languages – C.” Working draft N1570 (April 2011), 2011. <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>.
- [5] K. D. Cooper and L. Torczon, *Engineering a Compiler*. Burlington, MA: Morgan Kaufmann / Elsevier, 2 ed., 2011.
- [6] A. W. Appel and J. Palsberg, *Modern Compiler Implementation in C*. Cambridge, UK: Cambridge University Press, 1998.
- [7] K. C. Loudon, *Compiler Construction: Principles and Practice*. Boston, MA: PWS Publishing Company, 1997.
- [8] LLVM Project, “LLVM language reference manual.” <https://llvm.org/docs/LangRef.html>, 2024. Accessed 2026-05-07.
- [9] F. Bellard, “TCC: Tiny C compiler.” <https://bellard.org/tcc/>, 2024. Single-pass C compiler that emits machine code directly during parsing.
- [10] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Efficiently computing static single assignment form and the control dependence graph,” *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 4, pp. 451–490, 1991.
- [11] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein, “Register allocation via coloring,” *Computer Languages*, vol. 6, no. 1, pp. 47–57, 1981.
- [12] P. Briggs, *Register Allocation via Graph Coloring*. PhD thesis, Rice University, 1992. Tech. Report TR92-183.
- [13] L. George and A. W. Appel, “Iterated register coalescing,” *ACM Transactions on Programming Languages and Systems*, vol. 18, no. 3, pp. 300–324, 1996.
- [14] M. Poletto and V. Sarkar, “Linear scan register allocation,” *ACM Transactions on Programming Languages and Systems*, vol. 21, no. 5, pp. 895–913, 1999.
- [15] SDCC Development Team, “SDCC – small device C compiler.” <https://sdcc.sourceforge.net/>, 2024. Free open-source retargetable C compiler for 8-bit and 16-bit microcontrollers (8051, Z80, AVR, STM8, MOS6502, PIC, ...).
- [16] Free Software Foundation, “AVR-GCC: GCC backend for the Atmel AVR 8-bit microcontroller family.” <https://gcc.gnu.org/wiki/avr-gcc>, 2024.

- [17] Texas Instruments and Free Software Foundation, “MSP430-GCC: GCC backend for Texas Instruments MSP430 16-bit microcontrollers.” <https://www.ti.com/tool/MSP430-GCC-0PENSOURCE>, 2024.
- [18] L. L. Beck, *System Software: An Introduction to Systems Programming*. Reading, MA: Addison-Wesley, 3 ed., 1997.
- [19] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 3 ed., 1997.

# Appendix A

## Diagnostic Code Reference

---

This appendix lists every SEM###, SEMW### and IR### code that the compiler may emit. Codes shown as ranges (e.g. SEM020–SEM022) denote a contiguous group enforcing related rules; the per-code details live in `docs/semantic-analysis/SEMANTIC_CHECK_MATRIX.md`.

| Code                                           | Severity   | Rule (one-line summary)                                    |
|------------------------------------------------|------------|------------------------------------------------------------|
| <i>Pass 1 – symbol binding</i>                 |            |                                                            |
| SEM002                                         | error      | redeclaration in same scope                                |
| SEM003                                         | note       | shadowing in nested scope is allowed                       |
| SEM004                                         | error      | prototype / definition signature mismatch                  |
| SEM005                                         | error      | duplicate function definition                              |
| SEM006                                         | error      | use-before-declaration in same block                       |
| SEM007                                         | error      | <b>inline</b> invalid on object declaration                |
| SEM063                                         | error      | duplicate enum member                                      |
| SEM064                                         | error      | invalid tag redefinition                                   |
| SEM080-SEM083                                  | error      | aggregate-declaration / forward-declaration rules          |
| SEM900-SEM902                                  | error      | out-of-memory / internal contract violations               |
| <i>Pass 2 – type inference and constraints</i> |            |                                                            |
| SEM001                                         | error      | use of unknown identifier                                  |
| SEM008                                         | error      | assignment to const object                                 |
| SEM009                                         | error      | inc/dec of const object                                    |
| SEM010                                         | error      | const-drop in pointer assignment                           |
| SEM011-SEM016                                  | error/warn | type-conversion policy                                     |
| SEM020-SEM022                                  | error      | arithmetic op rules (operand legality, void/string)        |
| SEM023-SEM026                                  | error      | bitwise / shift / relational / logical op rules            |
| SEM027                                         | error      | LHS of = must be modifiable lvalue                         |
| SEM028                                         | error      | operand of ++/- must be modifiable lvalue                  |
| SEM029-SEM030                                  | error      | address-of / dereference rules                             |
| SEM031                                         | error      | array index must be integral                               |
| SEM032                                         | error      | subscript on non-array, non-pointer                        |
| SEM040-SEM042                                  | error      | call target / arity / argument type                        |
| SEM043                                         | error      | return-statement type mismatch                             |
| SEM044                                         | error      | value-return in <b>void</b> function                       |
| SEM045                                         | warn       | missing return in non- <b>void</b> function (conservative) |
| SEM046                                         | error      | call of non-function                                       |
| SEM050                                         | error      | <b>break</b> outside loop or switch                        |
| SEM051                                         | error      | <b>continue</b> outside loop                               |
| SEM052                                         | error      | loop / <b>switch</b> condition must be scalar              |
| SEM053-SEM056                                  | error      | ternary / case-label / switch rules                        |
| SEM060                                         | error      | member access on non-aggregate                             |
| SEM061                                         | error      | . requires struct/union value                              |
| SEM062                                         | error      | -> requires pointer to struct/union                        |
| SEM070-SEM072                                  | error      | codegen blockers (float, struct return, variadic)          |
| SEM900-SEM901                                  | error      | internal contract violations                               |
| SEMW001-SEMW003                                | warn       | implicit narrow / signed-unsigned compare / unused result  |
| <i>IR lowering</i>                             |            |                                                            |
| IR001                                          | error      | codegen-blocked construct reached lowering                 |
| IR002                                          | error      | identifier without semantic binding reached lowering       |
| IR003                                          | error      | malformed declaration / statement reached lowering         |
| IR004                                          | error      | control-flow stack overflow / unbound break-or-continue    |

**Table A.1:** Complete diagnostic code reference. The full text and example trigger for every code are reproduced in docs/semantic-analysis/SEMANTIC\_CHECK\_MATRIX.md.

# Appendix B

## IR Opcode Reference

This appendix is the canonical reference for the 40-opcode IR. Every opcode listed here corresponds to an enumerator in `IR/ir.h`; `MUL`, `DIVS`, `DIVU`, `MODS`, `MODU` are present in the enum but always lowered to `CALL` of the runtime helper of the same name (without the underscore prefix), so they never reach codegen as a real opcode.

| Opcode                                                  | Form                                | Semantics (one line)                        |
|---------------------------------------------------------|-------------------------------------|---------------------------------------------|
| <i>Constants and moves</i>                              |                                     |                                             |
| CONST                                                   | %d = const T, k                     | immediate of type T                         |
| COPY                                                    | %d = copy %s                        | register-to-register copy                   |
| <i>Arithmetic</i>                                       |                                     |                                             |
| ADD                                                     | %d = %a add %b                      | integer add                                 |
| SUB                                                     | %d = %a sub %b                      | integer subtract                            |
| MUL, DIVS, DIVU,<br>MODS, MODU                          | %d = %a op %b                       | always lowered as CALL<br>to runtime helper |
| AND                                                     | %d = %a and %b                      | bitwise and                                 |
| OR                                                      | %d = %a or %b                       | bitwise or                                  |
| XOR                                                     | %d = %a xor %b                      | bitwise xor                                 |
| SHL                                                     | %d = %a shl %b                      | left shift                                  |
| SHRS                                                    | %d = %a shrs %b                     | arithmetic right shift (signed)             |
| SHRU                                                    | %d = %a shru %b                     | logical right shift (unsigned)              |
| NEG                                                     | %d = neg %a                         | two's-complement negation                   |
| NOT                                                     | %d = not %a                         | bitwise complement                          |
| <i>Comparisons (always produce i1)</i>                  |                                     |                                             |
| EQ, NEQ                                                 | %d = %a eq/neq %b                   | equality                                    |
| LTS, LES, GTS, GES                                      | ditto                               | signed compare                              |
| LTU, LEU, GTU, GEU                                      | ditto                               | unsigned compare                            |
| <i>Memory</i>                                           |                                     |                                             |
| ADDR_OF                                                 | %d = addr_of %slot/@g               | take address of slot/global                 |
| LOAD                                                    | %d = *%p                            | load through pointer                        |
| STORE                                                   | *%p = %v                            | store through pointer                       |
| GEP                                                     | %d = %base + %idx*W                 | array-index address compute                 |
| <i>Casts</i>                                            |                                     |                                             |
| ZEXT                                                    | %d = zext T %a                      | zero-extend                                 |
| SEXT                                                    | %d = sext T %a                      | sign-extend                                 |
| TRUNC                                                   | %d = trunc T %a                     | truncate to narrower                        |
| BITCAST                                                 | %d = bitcast T %a                   | reinterpret bits                            |
| <i>Control (every block must end with one of these)</i> |                                     |                                             |
| GOTO                                                    | goto bbN                            | unconditional jump                          |
| BRANCH                                                  | if %c goto bbT else bbF             | two-way conditional                         |
| SWITCH                                                  | switch %v default bbD [k0:bb0, ...] | jump table                                  |
| RET                                                     | ret %v or ret void                  | return                                      |
| <i>Calls</i>                                            |                                     |                                             |
| CALL                                                    | %d = @callee(arg, ...)              | function call (incl. runtime helpers)       |

Table B.1: Complete IR opcode reference.

# Appendix C

## gr0040 ABI Quick Reference

---

For convenience this appendix repeats the register table (Table 3.7) with the caller / callee saved annotation. The authoritative reference is the FPGA report [3] §5 and the file `abi_spec.md` in the repository root.

| Register | Alias | Saved by                             | Allocatable                       |
|----------|-------|--------------------------------------|-----------------------------------|
| r0       | zero  | —                                    | no                                |
| r1       | a0    | caller                               | yes                               |
| r2, r3   | a1,a2 | caller                               | yes                               |
| r4–r6    | t0–t2 | caller                               | yes                               |
| r7       | t3    | —                                    | <i>reserved</i> (codegen scratch) |
| r8–r11   | s0–s3 | callee                               | yes                               |
| r12      | fp    | callee (special)                     | no                                |
| r13      | sp    | —                                    | no                                |
| r14      | lr    | caller (pushed in non-leaf prologue) | no (special)                      |
| r15      | gp    | —                                    | no                                |

**Table C.1:** gr0040 ABI register summary.